

中药材热风烘干过程的热湿耦合建模与坐标正则化谱方法求解

摘要

热风烘干是中药材产地加工中能耗最高、也最易造成药效成分损失的工序，其品质取决于药材内部温度场与水分场的协同演化。本题的难点有三：圆柱轴心的 $1/r$ 坐标奇异使常规网格离散复杂化；表面为对流传质而非平衡边界；问题 4 的尺寸收缩构成移动边界，会使固定网格上的守恒律失真。本文以导热理论、Fick 第二定律与表观体积热容假设为基础，建立中药材热风烘干过程的圆柱轴对称热湿耦合模型；基于附件 1 的 241 组实测数据建立一阶惯性环境边界模型，温度与含湿量的拟合决定系数 R^2 分别为 0.9957 与 0.9901；控制方程经坐标正则化 $u = (r/R)^2$ 变换后在 Chebyshev–Gauss–Lobatto 节点上谱配置离散，时间域采用变阶变步长隐式 BDF 推进。

针对问题一：常物性下热湿两场解耦，难点在于温度场要在指数型环境边界下给出可核验的基准解。本文将温度场作准静态分解，得到 **Bessel–Duhamel 级数闭式解**；水分场在正则坐标上求解。 $t = 1800\text{ s}$ 时中心温度、表面温度和表面含水率分别为 34.0425°C 、 37.1989°C 和 1.5109 kg/kg ，谱解与闭式解的最大差为 $1.14 \times 10^{-10}\text{ K}$ 。

针对问题二：难点在于物性随温度与含水率同时变化，两场不再解耦。本文将附录 3 的变物性关系代入控制方程，并按附件 1 刻画两阶段烘房环境，通过 $D(C, T_K)$ 与热物性的相互依赖描述双向耦合。 $t = 3\text{ h}$ 时中心温度、中心含水率和表面含水率分别为 49.9051°C 、 1.7673 kg/kg 和 1.0083 kg/kg 。

针对问题三：难点在于判据是逐点「处处低于 0.15 kg/kg 」而非平均值，且最后干燥点的位置事先未知。本文以全域最大含水率定义终止事件，逐时刻在全部 $N + 1$ 个节点上取极值，得烘干时长 57.1043 h (2.38 天)，与题目 2—3 天量级独立吻合；若改用截面平均值判定，会把时长低估 37.3%。

针对问题四：难点在于移动边界会使固定 r 坐标下的守恒律失真。本文引入物质坐标 $\xi = r/R(t)$ ，再取 $u = \xi^2$ 消去轴心奇异项，并证明固定 r 坐标下必须保留附加对流项。采用附录 4 物性并计入实测收缩后得烘干时长 50.7717 h ；为隔离收缩效应补算受控基准 128.9223 h ：物性改变使时长增加 125.8%，而单独考虑尺寸收缩效应时缩短 60.6%，二者方向相反，故问题 3 到问题 4 的 11.1% 净差异不能归因于收缩。

检验与灵敏度：以闭式解为绝对基准，空间加密至 $N = 8$ 即达 10^{-10} K 量级偏差平台，时间容差收紧至 10^{-12} 时偏差单调降至 $1.1 \times 10^{-10}\text{ K}$ ；三族独立离散互差不超过 2×10^{-6} ，六种积分器与容差配置给出的烘干时长极差仅 0.007 s 。12 组参数扰动表明活化温度 E 最敏感 ($\pm 2\%$ 使时长变化 $+23.9\%/ - 18.6\%$)；环境口径对问题 1 温度场影响达 0.467°C ，而对烘干时长仅 4.4 s 。

关键词：热湿耦合；Robin 边界；坐标正则化；谱配置；移动边界；受控对照

一、问题重述

1.1 问题背景

干燥是中药材产地加工与炮制过程中的关键工序，直接决定成品有效成分保留率、外观等级与贮藏稳定性。热风烘干因设备简单、成本低而应用最广。该过程分为两个阶段：先经历「预热平衡阶段」，药材由环境温度被加热到接近烘房温度、内部建立稳定的温度梯度；随后进入「恒温干燥阶段」，在近似恒定的温湿度环境下水分由内部迁至表面并被气流带走。工艺参数选择不当会导致干燥过慢（能耗高、周期长）、表面「结壳」（外干内湿）或温度过高（破坏热敏性成分）。传统的「试错——检测」优化模式周期长、成本高，近年来针对中药材热风干燥的热湿耦合建模 [3] 与分段温湿度调控 [4] 已有系统研究，为本文提供了对照；因此有必要用数理模型刻画药材内部温度与水分浓度的时空演化，以预测干燥时长、指导工艺设计。

1.2 题目条件

某中药材形状近似为圆柱：长 $L = 25\text{ cm}$ 、半径 $R_0 = 2\text{ cm}$ 。烘干开始时药材温度 $T_0 = 28^\circ\text{C}$ ，水分浓度（干基含水率） $C_0 = 2.55\text{ kg/kg}$ 。烘房温度与水分浓度随时间的变化由附件 1 给出（ $0 \sim 14400\text{ s}$ ，每 60 s 一点，共 241 点）；干燥过程中药材半径变化由附件 2 给出（ $0 \sim 259200\text{ s}$ ，每 1800 s 一点，共 145 点）。附录 2–4 分别给出问题 1、问题 2–3、问题 4 所需的物性经验关系。

1.3 需要解决的问题与量子子任务

问题	输入	输出	判定标准
一	附录 2 物性、附件 1 环境、初值 ($28^\circ\text{C}, 2.55$)	$t = 100 \sim 1800\text{ s}$ 、 $r = 0 \sim 2\text{ cm}$ 的温度与含水率； 1800 s 内每 1 s 、每 0.1 cm 全结果	表 3、表 4 保留 4 位小数； result1.xlsx
二	附录 3 变物性、两阶段工艺	3 h 内每 0.5 h 的温度与含水率；全结果	表 5、表 6；result2.xlsx
三	同问题二，时间跨度至干燥结束	烘干时长；每 6 h 、每 0.5 cm 的含水率	处处 $C < 0.15\text{ kg/kg}$ ； 表 7；result3.xlsx
四	附录 4 物性、附件 2 的 $R(t)$	烘干时长；每 6 h 、每 0.5 cm 的含水率	同问题三；表 8； result4.xlsx

1.4 数据探索

如图 1 所示，附件 1 的两个序列均呈明显一阶惯性上升形态：温度由 28°C 升至约 50.2°C 后稳定，含湿量由 0.01963 kg/kg 升至约 0.0509 kg/kg 后稳定，在 $t \approx 4\text{ h}$ 处进入平台——这一形态直接决定了后续的环境建模方式（一阶惯性式）与工艺分段（ $t = 14400\text{ s}$ ）。附件 2 的半径序列（图 2）单调不增，由 $R(0) = 2.000\text{ cm}$ 迅速下降，至 $t > 207000\text{ s}$

后稳定在 $R \approx 1.198 \sim 1.200 \text{ cm}$ ($R/R_0 = 0.599$)，90% 的收缩在前 10 h 内完成， $|dR/dt|$ 在 $t = 0$ 处最大。

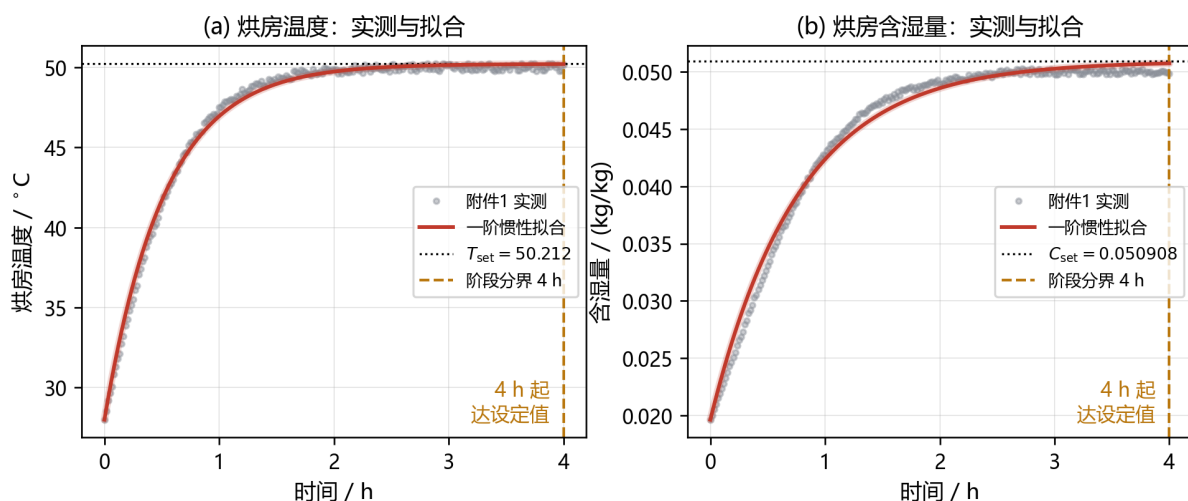


图 1 附件 1 烘房温度与含湿量的实测序列及一阶惯性拟合

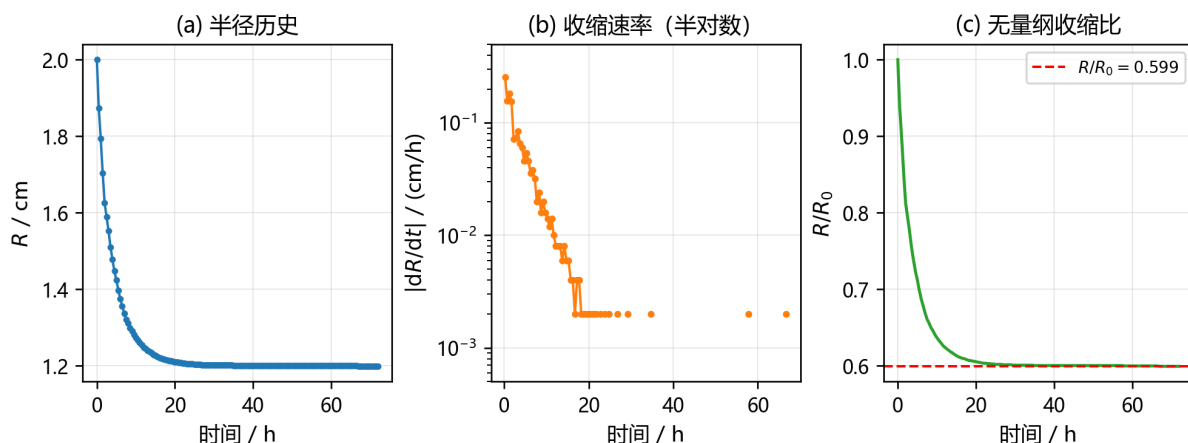


图 2 附件 2 药材半径历史、收缩速率与无量纲收缩比（其中 (b) 为逐区间速率 $|\Delta R/\Delta t|$ ；实测半径按 0.001 cm 的步进下降，半径不变的区间速率为零，在半对数轴上不绘出）

二、问题分析

2.1 总体思路与建模理由链

本题的核心是一个变物性、非线性、带 Robin 边界、且（问题 4 中）求解域随时间收缩的轴对称扩散问题。四个问题共用同一套控制方程，差别只在物性关系、环境口径与几何是否收缩。因此采取「一套方程、一套求解框架、四组参数」的总体思路，把工作量集中在「把模型建准」与「把数值解算到可信」两件事上。技术路线如图 3 所示。

理由链（以问题 1 为例）：机理——热风对流的边界驱动 + 内部导热/扩散；模型形式——圆柱坐标下的抛物型耦合方程组 + Robin 边界；可辨识参数—— ρ, c_p, k, h, h_m 由附录 2 给出， $D(C)$ 由附录 2 的经验式给出，环境由附件 1 标定，均不含不可辨识参数；求解方法——温度场与水分场解耦 $\Rightarrow$ 温度场可闭式求解 (§6.1)，水分场用谱配置；可验证性——温度场有闭式解作绝对基准，水分场可用两族离散互证。

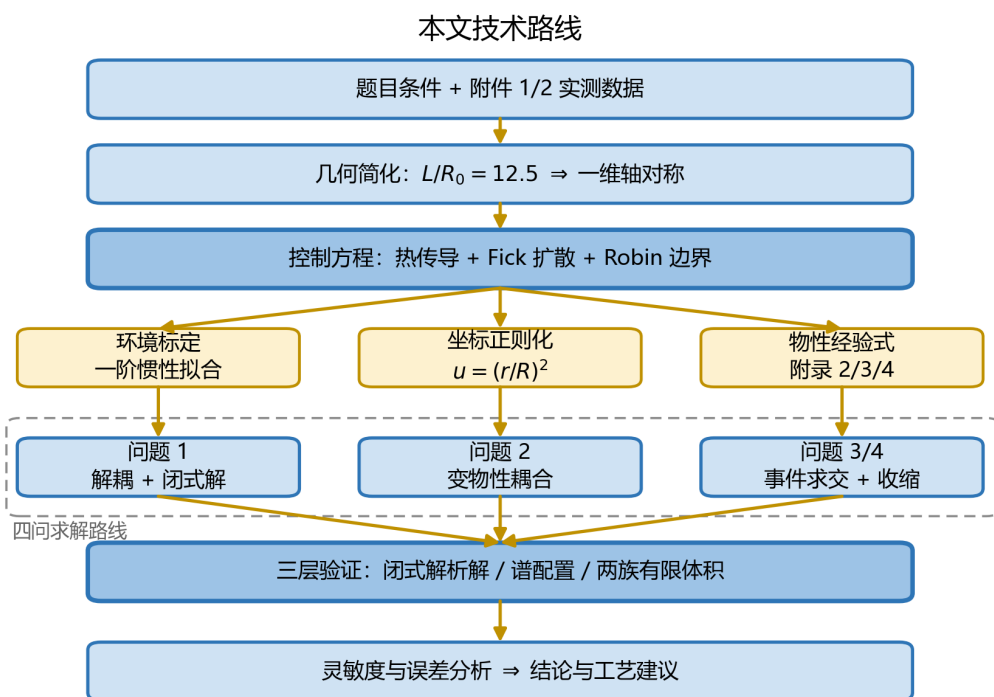


图 3 本文技术路线图

2.2 各问关键点

- **问题 1:** 关键在于识别「两场解耦」。附录 2 给的是常数物性、 D 只依赖 C ，温度方程与水分方程可独立求解；又因烘房温度恰为指数型，温度场可用准静态分解化为闭式级数解。物理预期为「热快质慢」： $Fo_T(1800\text{ s}) = 0.760$ 而 $Fo_D = 0.0222$ 。
- **问题 2:** 关键是两阶段工艺刻画与变物性耦合。附录 3 的 D 同时依赖 C 与 T ，两场真正耦合；温度 2 h 内基本平衡，此后药材近似等温体，水分剖面由平缓转为陡峭。
- **问题 3:** 关键是判据的数学表述与极值点位置。判据是逐点的「处处低于」，而非平均值；由圆柱几何对称性，中心扩散路径最长，必然是最后干燥点。本文不预设这一点，而逐时刻对全场节点取最大。
- **问题 4:** 关键是移动边界。各向同性收缩下若在固定 r 坐标直接套用扩散方程会漏掉一个对流项，必须改用物质坐标；同时表格列名是「当前物理距离」，与求解坐标不是一回事，采样须逐时刻换算。

三、模型假设

1. **轴对称一维简化。** 药材为有限长圆柱，但忽略轴向梯度和端面效应，温度与干基含水率只依赖 r 和 t 。长径比 $L/R_0 = 12.5$ ，轴向与径向特征时间之比 $(L/R_0)^2 = 156$ ；烘房气流沿轴向近似均匀。这一简化的误差在模型评价中单独说明。
2. **局部热平衡。** 骨架与内部水分在同一位置瞬时达到热平衡，用单一温度描述。题面未给出固液两相换热参数，故采用单温度近似； $Le = D/\alpha = 2.9 \times 10^{-2}$ 仅说明热扩散快于水分扩散，不作为局部热平衡的充分证明。
3. **干物质不迁移。** 干燥中水分迁移，干物质骨架只随整体收缩运动。固定半径时干物质体积密度 ρ_d 取常数；问题 4 中 $\rho_d R(t)^2$ 沿材料点守恒。
4. **表面传热传质为对流型。** 由 h 与 h_m 控制，且过程中保持常数。依据：题目直接给

定 $h = 25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ 。

5. **物性经验关系适用**。三组物性式在其适用区间内成立。因题目未给出标定区间，本文在 § 十二、中对 E 与 a 做 $\pm 2\%$ 扰动评估外推风险。
6. **烘房环境可分段描述**。 $0 \sim 14400 \text{ s}$ 按一阶惯性式变化，其后恒定（烘房持续排湿）。依据：附件 1 在 $t \approx 4 \text{ h}$ 达到设定值并稳定。
7. **收缩各向同性**。同一物质点在任意时刻满足 $r(t) = \xi R(t)$ 。依据：题目只给出半径方向的收缩数据；§ 九、证明该假设下物质坐标对流项严格为零。
8. **采用表观热容近似**。能量方程只保留显热传导，忽略水分迁移携带的显焓、蒸发潜热和体积变形功。题目未提供含湿焓、汽化潜热与孔隙率等闭合数据，本文在 § 十三、中将其列为适用边界。

四、符号说明

符号	含义	单位
r	到药材中心的物理距离	m
$R(t)$	当前半径（问题 4 变化，其余恒为 R_0 ）	m
R_0, L	初始半径、药材长度	m
$T(r, t)$	药材温度	$^{\circ}\text{C}$
$C(r, t)$	药材干基含水率（kg 水/kg 干物质）	kg/kg
$T_{\text{air}}, C_{\text{air}}$	烘房温度、空气含湿量	$^{\circ}\text{C}$, kg/kg
ρ_d	单位当前体积内的干物质质量	kg/m^3
ρ, c_p, k	湿物料表观密度、比热容、导热系数	kg/m^3 , $\text{J}/(\text{kg} \cdot \text{K})$, $\text{W}/(\text{m} \cdot \text{K})$
D	以干基含水率梯度为驱动力的有效扩散系数	m^2/s
h, h_m	对流换热系数、对流传质系数	$\text{W}/(\text{m}^2 \cdot \text{K})$, m/s
α	热扩散率 $k/(\rho c_p)$	m^2/s
Bi, Bi_m	热/传质 Biot 数	—
Fo, Fo_D	热/传质 Fourier 数	—
Le	Lewis 数 D/α	—
ξ	物质坐标 $r/R(t)$	—
u	正则化坐标 ξ^2	—
T_K	Arrhenius 关系使用的绝对温度, $T_K = T + 273.15$	K
E	活化温度参数 E_a/R_g （本题为 3850 K ）	K
a	活化浓度指数	kg/kg
t_{dry}	烘干时长（处处 $C \leq 0.15$ 的首达时刻）	s

五、模型的建立

5.1 几何简化

药材为半径 $R_0 = 2 \text{ cm}$ 、长 $L = 25 \text{ cm}$ 的圆柱。轴向导热特征时间 $L^2/\alpha \approx 3.70 \times 10^5 \text{ s}$ （约 4.3 天），径向 $R_0^2/\alpha \approx 2369 \text{ s}$ ，两者之比恰为 $(L/R_0)^2 = 156$ ，相差约 2.2 个数量级；同时烘房气流沿轴向均匀送风。故端面效应与轴向梯度可以忽略，问题化为轴对称一维径向问题（图 4）；这类圆柱湿物料的热风干燥有成熟的有限差分分解与实测对照 [1]，同一类「温度场—水分场同时求解」的建模范式在果蔬 [2] 与中药材 [3] 上亦被广泛采用。

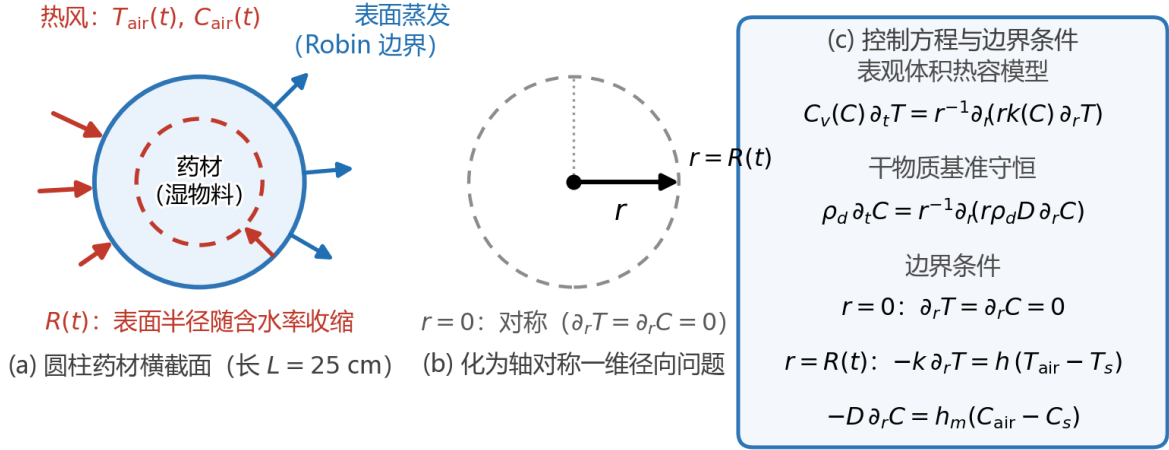


图 4 物理模型与坐标系示意图: (a) 圆柱药材横截面, 热风对流驱动、表面水分蒸发、半径随含水率收缩; (b) 取 1/4 截面对称化后的一维径向坐标 $r \in [0, R(t)]$; (c) 控制方程与 $r=0$ 对称条件、 $r=R(t)$ 处 Robin 边界条件

5.2 传热控制方程: 表观体积热容模型

题目给出的 $\rho(C)$ 、 $c_p(C)$ 和 $k(C)$ 视为湿物料的表现热物性, 记 $C_v(C) = \rho(C)c_p(C)$ 。在局部热平衡和显热传导近似下, 温度满足

$$C_v(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(rk(C) \frac{\partial T}{\partial r} \right). \quad (1)$$

式 (1) 是以温度为状态变量的表观热容闭合式, 不等同于对 $C_v T$ 直接求时间导数。它在每个时刻根据局部含水率更新 C_v 和 k , 但不计含湿焓交叉项与蒸发潜热。若这些热力学数据可获得, 则应改用体积焓方程 $\partial_t H(C, T) = r^{-1} \partial_r (rk \partial_r T) - Q_{\text{evap}}$ 。完备的多孔介质模型还含热湿交叉通量与相变项 [5]、气相压力与两相动量方程 [6]; 题目未给出相应闭合数据, 故本文只取其对 Fourier-Fick 极限的简化形式。

5.3 传质控制方程: 干物质基准守恒

以干基含水率 C 为变量, 单位当前体积内的水分质量为 $\rho_d C$, 相对于干物质骨架的水分通量取为 $j_r = -\rho_d D(C, T_K) \partial_r C$ 。问题 1-3 的半径固定, 干物质骨架不发生宏观运动, 且 ρ_d 在求解域内取常数。环形壳层的水分衡算为

$$\rho_d \frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r \rho_d D \frac{\partial C}{\partial r} \right).$$

约去常数 ρ_d 后得到

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C, T_K) \frac{\partial C}{\partial r} \right). \quad (2)$$

这里的 ρ_d 是干物质体积密度, 不与经验湿物料密度 $\rho(C)/(1+C)$ 混用; D 被解释为以干基含水率梯度为驱动力的有效扩散系数。若令 ρ_d 随空间或时间变化, 则必须保留其在守恒式中的位置, 不能直接约去。

5.4 边界条件与初始条件

中心对称： $\partial_r T(0, t) = \partial_r C(0, t) = 0$ 。表面 Robin 条件（ $r = R$ ）：

$$-k \frac{\partial T}{\partial r} \Big|_{r=R} = h(T_s - T_{\text{air}}(t)), \quad -D \frac{\partial C}{\partial r} \Big|_{r=R} = h_m(C_s - C_{\text{air}}(t)), \quad (3)$$

其中 $T_s = T(R, t)$ 、 $C_s = C(R, t)$ 。 $C_s - C_{\text{air}}$ 是题目给定经验边界中的有效传质驱动力；固相干基含水率与气相含湿量的分母基准不同，本式不表示二者满足直接相平衡。题面未给出吸附等温线或界面分配系数，故不再增加无法标定的界面参数。初始条件为 $T(r, 0) = 28^\circ\text{C}$ ， $C(r, 0) = 2.55 \text{ kg/kg}$ 。

5.5 烘房环境数据的标定

附件 1 是带噪实测序列，直接作边界会把测量噪声注入解。由图 1 的一阶惯性形态，先作一阶惯性拟合，初值固定为实测初值：

$$T_{\text{air}}(t) = T_{\text{set}} - (T_{\text{set}} - 28)e^{-t/\tau_T}, \quad C_{\text{air}}(t) = C_{\text{set}} - (C_{\text{set}} - 0.01963)e^{-t/\tau_C}. \quad (4)$$

用 Levenberg–Marquardt 非线性最小二乘（初值固定，每式只有 2 个自由参数）得

表 1 烘房环境一阶惯性标定结果

参数	拟合值	标准误	RMSE	R^2
$T_{\text{set}} / ^\circ\text{C}$	50.2121	0.0303	0.336 $^\circ\text{C}$	0.9957
τ_T / s	1877.49	12.74		
$C_{\text{set}} / (\text{kg/kg})$	0.050908	9.3×10^{-5}	8.1×10^{-4}	0.9901
τ_C / s	2776.34	31.71		

计算中把标定参数按其有效位数取用（ $T_{\text{set}} = 50.212$ 、 $\tau_T = 1877.5$ 、 $\tau_C = 2776.3$ ）。该取舍在 $T(R, 1800 \text{ s})$ 上仅造成约 $6 \times 10^{-5} ^\circ\text{C}$ 的变化，比标定标准误小三个数量级。

工艺分段：由附件 1 可见烘房在 $t \approx 4 \text{ h}$ 达到设定值并稳定，故取

$$(T_{\text{air}}, C_{\text{air}}) = \begin{cases} \text{式(4)}, & 0 \leq t \leq 14400 \text{ s}, \\ (50.212 ^\circ\text{C}, 0.050908 \text{ kg/kg}), & t > 14400 \text{ s}. \end{cases} \quad (5)$$

该分段在 $t = 14400 \text{ s}$ 处不连续：一阶惯性式在该点的取值为 $50.2016 ^\circ\text{C}$ 与 0.050733 kg/kg ，与设定值分别相差 $0.0104 ^\circ\text{C}$ 与 $1.75 \times 10^{-4} \text{ kg/kg}$ 。相对于全程变化量（温度约 $22 ^\circ\text{C}$ 、含湿量约 0.031 kg/kg ）分别只有 5×10^{-4} 与 6×10^{-3} ，也小于环境拟合自身的 RMSE，故直接切换到设定值不会带来可见误差。若需严格连续，则应把分段点后移，或对惯性式作尾部截断。

5.6 物性经验关系与量纲检查

表 2 三组物性经验关系（附录 2–4）

	问题 1（附录 2）	问题 2、3（附录 3）	问题 4（附录 4）
$\rho / (\text{kg/m}^3)$	820	$650 + 128C$	$760 + 90C$
$c_p / (\text{J}/(\text{kg}\cdot\text{K}))$	2600	$1450 + 2736 \frac{C}{C+1}$	$1850 + 2150 \frac{C}{C+1}$
$k / (\text{W}/(\text{m}\cdot\text{K}))$	0.36	$0.21 + 0.38 \frac{C}{C+1}$	$0.12 + 0.20 \frac{C}{C+1}$
$D / (\text{m}^2/\text{s})$	$7 \times 10^{-9} e^{-0.89/C}$	$2.4 \times 10^{-3} e^{-0.45/C} e^{-3850/T_K}$	$4.2 \times 10^{-4} e^{-0.30/C} e^{-3850/T_K}$

e^{-3850/T_K} 为 Arrhenius 因子，对应活化能 $E_a = 3850R_g = 32.0 \text{ kJ/mol}$ 。量纲检查：3850 K 量纲为温度， $T_K = T + 273.15$ 须取绝对温度； a 与 C 同量纲 (kg/kg)，故 a/C 无量纲。

注 1 (经验式的读法自检) 附录 2 写成 $D = 7 \times 10^{-9} e^{-0.89/C}$ ，指数部分有两种读法。若读作 $e^{-0.89C}$ ，则 C 由 2.55 降到 0.15 时 D 反而增大 8.5 倍 ($e^{-0.89 \times 0.15} / e^{-0.89 \times 2.55} = 8.47$)，与「越干越难扩散」的降速干燥事实矛盾；读作 $e^{-0.89/C}$ 时 D 下降 266 倍，正是降速干燥所要求的。附录 3 同判： $e^{-0.45/C}$ 使 D 由 $C = 2.55$ 到 $C = 0.15$ 下降 16.8 倍。故三个 D 式中的指数一律读作 $e^{-a/C}$ 。

温度取绝对温度：由附录 3 在 $C = 2.55$ 、 $T = 50^\circ\text{C}$ 处得 $D = 1.347 \times 10^{-8} \text{ m}^2/\text{s}$ ，与已有中药材干燥实测的有效水分扩散系数 $3.90 \times 10^{-9} \sim 2.04 \times 10^{-8} \text{ m}^2/\text{s}$ [7] 属同一数量级（该系数随材料、尺寸与干燥方式变化，比较仅用于量级校验）；若把 T 误按摄氏度代入， $e^{-3850/50} \approx 0$ ， D 退化为零，显然不合理。

5.7 无量纲分析

定义 $\text{Bi} = hR_0/k$ 、 $\text{Bi}_m = h_m R_0/D$ 、 $\text{Fo} = \alpha t/R_0^2$ 、 $\text{Le} = D/\alpha$ 。取问题 1 参数： $\alpha = 1.6886 \times 10^{-7} \text{ m}^2/\text{s}$ ， $\text{Bi} = 1.3889$ ， $D(C_0) = 4.9377 \times 10^{-9} \text{ m}^2/\text{s}$ ， $\text{Bi}_m = 3.2404$ ， $\text{Le} = 2.9 \times 10^{-2}$ ，于是 $\text{Fo}_T(1800 \text{ s}) = 0.760$ 、 $\text{Fo}_D(1800 \text{ s}) = 0.0222$ （图 5）。两条先验结论：其一 $\text{Fo}_D \ll \text{Fo}_T$ ，温度先于水分达到准平衡；其二 Bi_m 量级为 1，内部扩散阻力与外部对流传质阻力相当，必须保留 Robin 条件而不能把表面当作平衡边界。

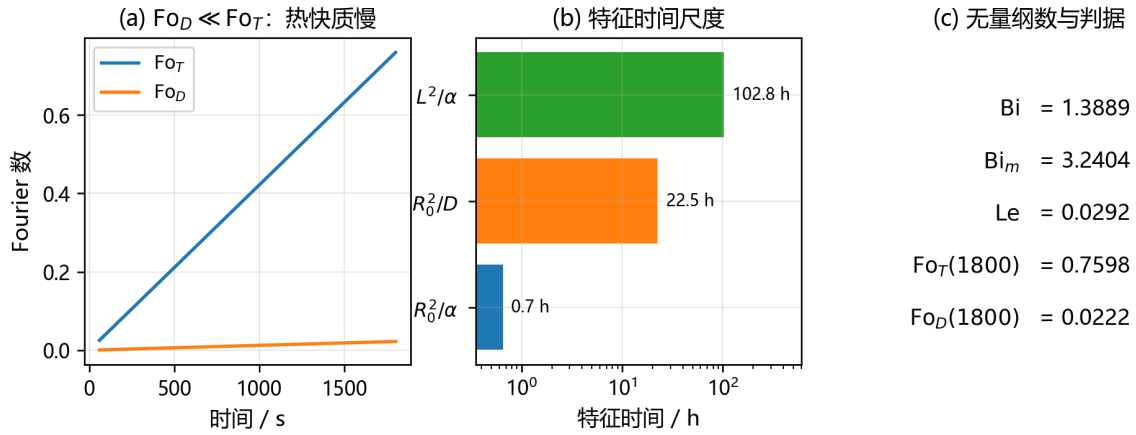


图 5 无量纲分析：Fourier 数、特征时间尺度与无量纲判据

5.8 新方法的出发点：坐标正则化

5.8.1 轴心奇异结构

把式 (1)、(2) 的算子展开：

$$\mathcal{L}\phi := \frac{1}{r} \frac{\partial}{\partial r} \left(r a \frac{\partial \phi}{\partial r} \right) = a \frac{\partial^2 \phi}{\partial r^2} + \frac{a}{r} \frac{\partial \phi}{\partial r} + \frac{\partial a}{\partial r} \frac{\partial \phi}{\partial r}, \quad (6)$$

第二项含 $1/r$ 。虽然由对称性 $\partial_r \phi(0) = 0$ 使极限有限，但离散格式必须显式处理这一奇异——常规节点中心有限体积要把最内控制体取成 $[0, h/2]$ 并令其测度为 $h^2/8$ ，界面差分还要借助对称性做单侧外推。这正是常规 r 坐标离散的主要复杂度来源。

5.8.2 正则化变换

命题 1 (圆柱 Laplace 算子的正则化) 令 $u = \xi^2 = (r/R)^2$, 则对任意光滑 $a(u)$,

$$\frac{1}{r} \frac{\partial}{\partial r} \left(r a \frac{\partial \phi}{\partial r} \right) = \frac{4}{R^2} \frac{\partial}{\partial u} \left(u a \frac{\partial \phi}{\partial u} \right), \quad (7)$$

且右端在 $u = 0$ 取有限值 $\partial_u(ua\partial_u\phi)|_{u=0} = a(0)\partial_u\phi(0)$ 。

证明. 由 $r = R\sqrt{u}$ 得 $\partial r/\partial u = R/(2\sqrt{u})$, 故 $\partial/\partial r = \frac{2\sqrt{u}}{R} \frac{\partial}{\partial u}$ 。逐步计算:

$$a \frac{\partial \phi}{\partial r} = a \frac{2\sqrt{u}}{R} \frac{\partial \phi}{\partial u}, \quad r a \frac{\partial \phi}{\partial r} = R\sqrt{u} \cdot a \cdot \frac{2\sqrt{u}}{R} \frac{\partial \phi}{\partial u} = 2ua \frac{\partial \phi}{\partial u},$$

$$\frac{\partial}{\partial r} \left(2ua \frac{\partial \phi}{\partial u} \right) = \frac{2\sqrt{u}}{R} \frac{\partial}{\partial u} \left(2ua \frac{\partial \phi}{\partial u} \right),$$

$$\frac{1}{r} \frac{\partial}{\partial r} \left(r a \frac{\partial \phi}{\partial r} \right) = \frac{1}{R\sqrt{u}} \cdot \frac{2\sqrt{u}}{R} \cdot \frac{\partial}{\partial u} \left(2ua \frac{\partial \phi}{\partial u} \right) = \frac{4}{R^2} \frac{\partial}{\partial u} \left(u a \frac{\partial \phi}{\partial u} \right).$$

正则性由 $\partial_u(ua\partial_u\phi) = a\partial_u\phi + u\partial_u(a\partial_u\phi)$ 在 $u = 0$ 处第二项为零立即得到。

因此两个控制方程在 $u \in (0, 1)$ 上统一写成

$$\boxed{\frac{\partial C}{\partial t} = \frac{4}{R^2} \frac{\partial}{\partial u} \left(u D \frac{\partial C}{\partial u} \right), \quad \rho c_p \frac{\partial T}{\partial t} = \frac{4}{R^2} \frac{\partial}{\partial u} \left(u k \frac{\partial T}{\partial u} \right).} \quad (8)$$

在 $u = 1$ 处 $\partial_r = 2\partial_u/R$, Robin 条件化为

$$2D \left. \frac{\partial C}{\partial u} \right|_{u=1} = h_m R (C_{\text{air}} - C_1), \quad 2k \left. \frac{\partial T}{\partial u} \right|_{u=1} = hR (T_{\text{air}} - T_1). \quad (9)$$

$u = 0$ 处采用正则解类: $\partial_u C$ 与 $\partial_u T$ 有界, 于是 $\partial_r C = 2\sqrt{u} \partial_u C/R$ 和 $\partial_r T = 2\sqrt{u} \partial_u T/R$ 自动趋于零。离散时直接在端点配置正则化方程, 不再增加与正则性重复的独立边界值。

5.8.3 为什么谱方法在 u 坐标下成立

命题 2 (解析性) 设 $C(\xi, t)$ 关于 ξ 为偶函数且解析, 则 C 作为 $u = \xi^2$ 的函数在 $[0, 1]$ 上解析。

证明. 偶解析函数有 Taylor 展开 $C(\xi) = \sum_{k \geq 0} a_{2k} \xi^{2k}$, 代入 $u = \xi^2$ 即得 $C(u) = \sum_{k \geq 0} a_{2k} u^k$ 。

具体地, 线性问题特征函数 $J_0(\beta_n r/R) = \sum_{k \geq 0} \frac{(-1)^k}{k!k!} \left(\frac{\beta_n^2}{4} \right)^k u^k$ 是 u 的整函数。故在 Chebyshev–Gauss–Lobatto 节点上做配置可获得谱精度 (图 6)。本文选择 u 而非 ξ 作为配置变量, 是为了把偶对称结构直接编码到坐标中并简化轴心端点处理; 在 ξ 坐标上显式施加对称条件同样可以构造谱方法, 因此这里不把该变换表述为唯一选择。

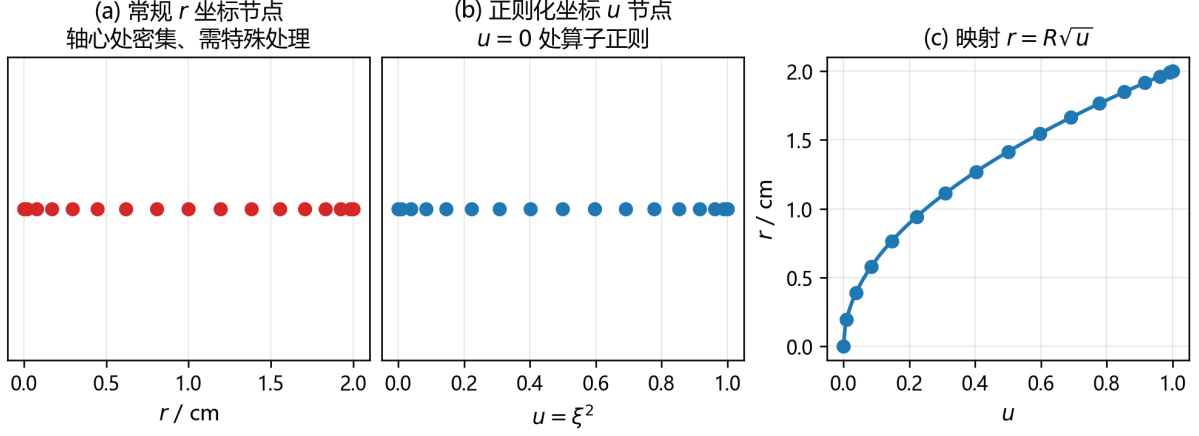


图 6 坐标正则化：常规 r 坐标与正则化坐标 $u = \xi^2$ 的节点分布对比

六、问题一：预热平衡阶段的求解

6.1 温度场的闭式解（Bessel–Duhamel 展开）

问题 1 的 ρ, c_p, k 为常数、 D 只依赖 C ，故两场完全解耦；又环境为指数型式 (4)，温度场可不离散地闭式求出。

记 $A = T_{\text{set}} - 28$ ， $T_{\text{air}}(t) = T_{\text{set}} - Ae^{-t/\tau_T}$ ，令 $w(r, t) = T(r, t) - T_{\text{air}}(t)$ 。由于 T_{air} 与 r 无关， $\nabla^2 T = \nabla^2 w$ ，于是

$$\frac{\partial w}{\partial t} = \alpha \nabla^2 w - \frac{A}{\tau_T} e^{-t/\tau_T}, \quad -k \left. \frac{\partial w}{\partial r} \right|_{r=R} = hw(R, t), \quad w(r, 0) = 0. \quad (10)$$

（初值： $w(r, 0) = 28 - (T_{\text{set}} - A) = 0$ 。）

准静态分解。设 $w = W(r)e^{-t/\tau_T} + \hat{w}(r, t)$ ，代入式 (10)：

$$\frac{\partial \hat{w}}{\partial t} = \alpha \nabla^2 \hat{w} + e^{-t/\tau_T} \left[\alpha \nabla^2 W + \frac{W}{\tau_T} - \frac{A}{\tau_T} \right].$$

令中括号为零，则 $\hat{w}$ 满足齐次热方程， W 满足

$$\nabla^2 W + \kappa^2 W = \kappa^2 A, \quad \kappa^2 := \frac{1}{\alpha \tau_T}, \quad -kW'(R) = hW(R), \quad (11)$$

且 $\hat{w}(r, 0) = -W(r)$ 。式 (11) 通解为 $W = A + BJ_0(\kappa r) + D_0 Y_0(\kappa r)$ ；由 $r = 0$ 处有界性去掉 Y_0 ，常数解 A 是 $\kappa^2 A$ 的特解。代入 Robin 条件 ($\frac{d}{dr} J_0(\kappa r) = -\kappa J_1(\kappa r)$) 得

$$W(r) = A + BJ_0(\kappa r), \quad B = \frac{hA}{\kappa \kappa J_1(\kappa R) - hJ_0(\kappa R)}, \quad \kappa = \frac{1}{\sqrt{\alpha \tau_T}}. \quad (12)$$

特征值问题。 $\hat{w}$ 满足齐次热方程与齐次 Robin 条件，分离变量得

$$\phi_n(r) = J_0\left(\frac{\beta_n r}{R}\right), \quad \beta_n J_1(\beta_n) = \text{Bi} J_0(\beta_n), \quad \gamma_n = \frac{\alpha \beta_n^2}{R^2}, \quad (13)$$

在权 r 下正交： $\int_0^R r \phi_n \phi_m dr = \frac{R^2}{2} [J_0^2(\beta_n) + J_1^2(\beta_n)] \delta_{nm}$ （该展开属经典导热解析框架 [8]），本文据此对指数型环境边界作准静态分解。

展开系数。设 $\hat{w} = \sum_n b_n \phi_n e^{-\gamma_n t}$ ，则 b_n 是 $-W$ 的带权投影系数。由 Green 恒等式

和 Robin 边界条件可得（完整推导见附录 A 的式 (25)）

$$\begin{aligned} b_n &= \frac{A\kappa^2 N_n}{(\beta_n^2/R^2) - \kappa^2}, \\ N_n &= \frac{2J_1(\beta_n)}{\beta_n[J_0^2(\beta_n) + J_1^2(\beta_n)]}, \\ T(r, t) &= T_{\text{air}}(t) + W(r)e^{-t/\tau_T} + \sum_{n \geq 1} b_n J_0\left(\frac{\beta_n r}{R}\right) e^{-\gamma_n t}. \end{aligned} \quad (14)$$

一致性自检： $t = 0$ 时 $T_{\text{air}}(0) = 28$ 且 $W(r) + \sum_n b_n \phi_n(r) = W(r) - W(r) = 0$ ，故 $T(r, 0) \equiv 28$ ； $t \rightarrow \infty$ 时 $T \rightarrow T_{\text{set}}$ 。**收敛速度：** $t = 1800$ s 时 $\gamma_n t$ 依次为 1.529, 13.191, 39.483, 80.742，即 $n \geq 2$ 项已压至 $e^{-13.2} \approx 1.8 \times 10^{-6}$ 以下（图 7），故式 (14) 的前两项已满足本文四位小数的报告精度；高精度验证仍按程序保留完整截断级数。

计算参数、前四个特征根与展开系数列于附录 A，并由图 7 给出衰减核验。

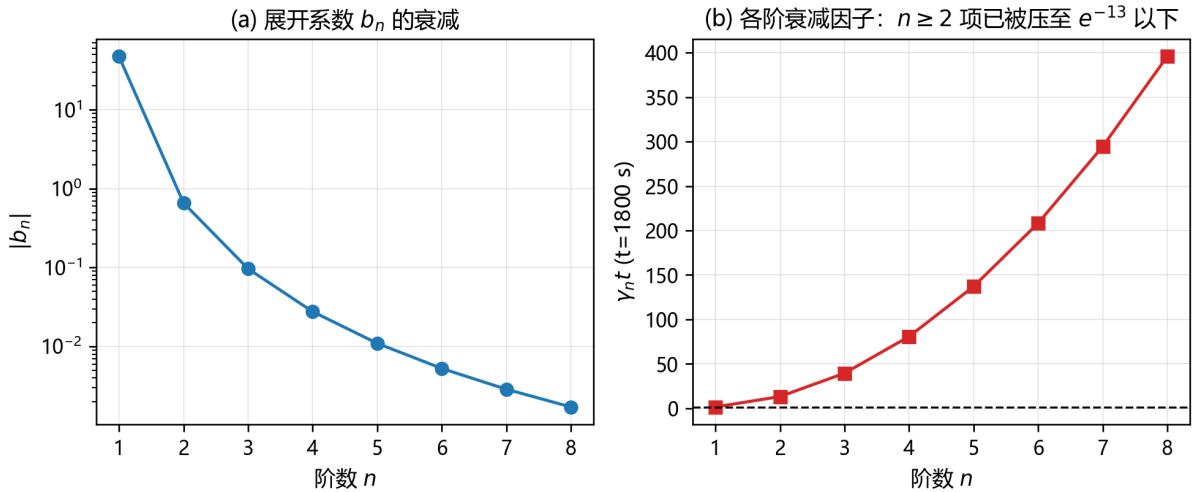


图 7 Bessel-Duhamel 展开：系数 b_n 的衰减与各阶衰减因子 $\gamma_n t$

6.2 水分场的谱配置求解

水分方程由式 (8) 给出，初值 $C \equiv 2.55$ ，表面通量由式 (9) 给出。注意初值 $C_s = 2.55$ 而 $C_{\text{air}} = 0.0196$ ，存在初值跳跃，须用自适应的首步抑制振荡。空间离散见 § 十、。

6.3 求解结果

表 3 30 分钟内药材的温度（单位：°C）

时间/s	$r = 0$	$r = 0.5$ cm	$r = 1.0$ cm	$r = 1.5$ cm	$r = 2.0$ cm
100	28.0001	28.0005	28.0054	28.0425	28.2219
300	28.0500	28.0776	28.1857	28.4550	29.0261
600	28.5456	28.6438	28.9618	29.5667	30.5576
900	29.5568	29.7074	30.1740	30.9976	32.2359
1200	30.9069	31.0882	31.6397	32.5824	33.9433
1500	32.4397	32.6353	33.2248	34.2150	35.6117
1800	34.0425	34.2411	34.8362	35.8249	37.1989

表 4 30 分钟内药材的水分浓度 (单位: kg/kg)

时间/s	$r = 0$	$r = 0.5 \text{ cm}$	$r = 1.0 \text{ cm}$	$r = 1.5 \text{ cm}$	$r = 2.0 \text{ cm}$
100	2.5500	2.5500	2.5500	2.5500	2.2471
300	2.5500	2.5500	2.5500	2.5492	2.0510
600	2.5500	2.5500	2.5500	2.5353	1.8773
900	2.5500	2.5500	2.5497	2.5046	1.7552
1200	2.5500	2.5500	2.5482	2.4646	1.6591
1500	2.5500	2.5499	2.5445	2.4207	1.5794
1800	2.5500	2.5497	2.5383	2.3756	1.5109

6.4 结果分析

温度场 (图 8、图 9): 表面在 100 s 内即上升 0.22°C , 1800 s 时达 37.1989°C ; 中心滞后明显, 1800 s 时为 34.0425°C , 仅占表面与环境温差 $41.696 - 28 = 13.696^\circ\text{C}$ 的 44%。

水分场: $r \leq 1.0 \text{ cm}$ 的三个位置几乎不变 (中心仍为 2.5500), 表面由 2.55 降到 1.5109, 降幅 40.7%, 与 $\text{Fo}_D = 0.0222$ 的数量级判断吻合。

脱水总量: 截面平均含水由 2.5500 降到 2.2937 kg/kg; 按附录 2 的 $\rho = 820 \text{ kg/m}^3$ 与初始含水 2.55 折算, 干物质质量 72.57 g、初始水量 185.04 g, 1800 s 内脱除水分约 18.6 g (占 10.1%)。

近表面梯度自检 (图 10): 1800 s 时表面处一阶导由密采样差分得 -302.4 kg/kg/m , 而该时刻 Robin 条件给出 $-h_m(C_s - C_{\text{air}})/D = -304.1 \text{ kg/kg/m}$, 两者相对差 0.6%, 余差来自差分的一阶截断误差。这验证了边界条件实现的正确性。

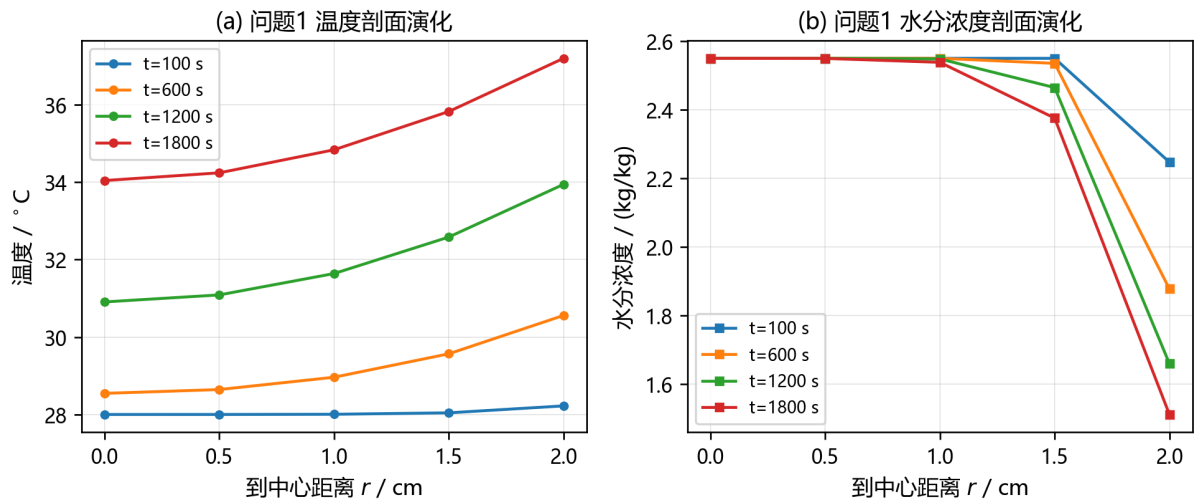


图 8 问题 1 温度与水分浓度剖面演化

七、问题二：整段烘干过程的求解

7.1 两阶段工艺与物性切换

依 §5.5, 预热平衡段 $0 \leq t \leq 14400 \text{ s}$ 环境按式 (4), 恒温干燥段 $t > 14400 \text{ s}$ 环境恒定; 物性改用附录 3 关系式; 方程与边界同式 (1)–(3)。此时 D 同时依赖 C 与 T , 两场真正耦合。

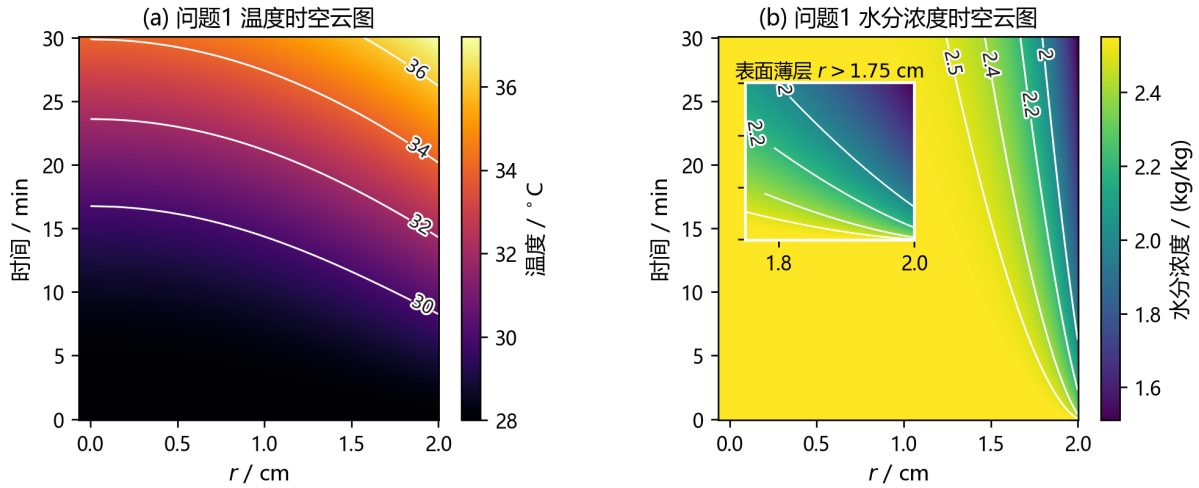


图9 问题1 温度与水分浓度的时空云图

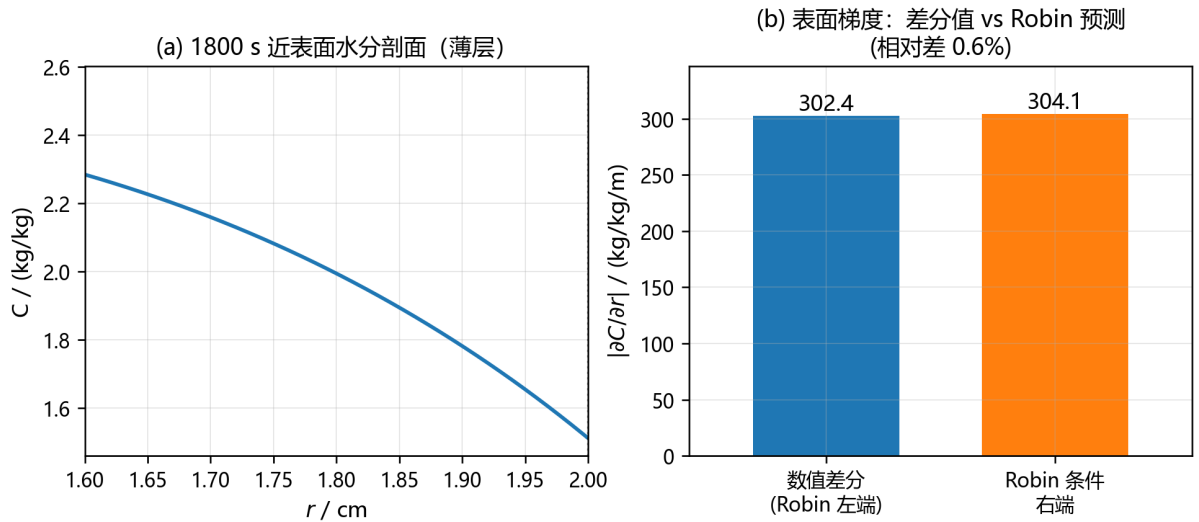


图10 近表面水分薄层与 Robin 条件的数值核对

7.2 求解结果

表5 3小时内药材的温度 (单位: °C)

时间/h	$r = 0$	$r = 0.5 \text{ cm}$	$r = 1.0 \text{ cm}$	$r = 1.5 \text{ cm}$	$r = 2.0 \text{ cm}$
0.5	32.5657	32.7628	33.3571	34.3574	35.7832
1.0	40.4221	40.5819	41.0549	41.8231	42.8629
1.5	45.5852	45.6720	45.9273	46.3366	46.8815
2.0	48.2254	48.2651	48.3815	48.5675	48.8138
2.5	49.4129	49.4293	49.4776	49.5546	49.6565
3.0	49.9051	49.9115	49.9302	49.9602	49.9999

7.3 结果分析

温度 2 h 内基本平衡: 1 h 中心 40.42 °C、2 h 48.23 °C、3 h 49.91 °C, 与环境温差仅 0.3 °C; 径向差由 1 h 的 2.44 °C 收缩到 3 h 的 0.09 °C。这说明恒温干燥阶段药材可近似视为等温体。

水分剖面由平缓转为陡峭 (图 11、图 12): 3 h 时中心 1.7673、 $r = 1.5 \text{ cm}$ 处 1.3332、

表 6 3 小时内药材的水分浓度（单位：kg/kg）

时间/h	$r = 0$	$r = 0.5 \text{ cm}$	$r = 1.0 \text{ cm}$	$r = 1.5 \text{ cm}$	$r = 2.0 \text{ cm}$
0.5	2.5499	2.5488	2.5247	2.3237	1.6538
1.0	2.5248	2.4934	2.3562	2.0227	1.4709
1.5	2.3860	2.3258	2.1349	1.8020	1.3444
2.0	2.1731	2.1105	1.9250	1.6256	1.2282
2.5	1.9589	1.9025	1.7369	1.4716	1.1157
3.0	1.7673	1.7174	1.5705	1.3332	1.0083

表面 1.0083 kg/kg，剖面斜率随 r 单调增大。这是 D 随 C 指数下降的直接后果：表面先失水，表层 D 随之下降低形成「干壳」，内部水分必须穿过阻力越来越大的外壳才能迁出。

失水速率呈降速特征 [9]（图 13）：表面传质通量 $|h_m(C_s - C_{\text{air}})|$ 由 6 h 的 3.87×10^{-7} 单调降到 72 h 的 $1.4 \times 10^{-9} \text{ kg}/(\text{m}^2 \cdot \text{s})$ ，跨越 3.2 个数量级。值得强调的是，本模型并未独立设定恒速期，该特征是自动涌现于 $D(C, T)$ 经验式的。问题 2 只要求输出 0 ~ 3 h，而这一衰减特征要到更长时段才充分展开；由于问题 2 与问题 3 的控制方程、边界条件和附录 3 物性完全相同，图 13 的两条曲线直接取同一模型积分到 72 h 的结果，故图与正文所指时段一致。

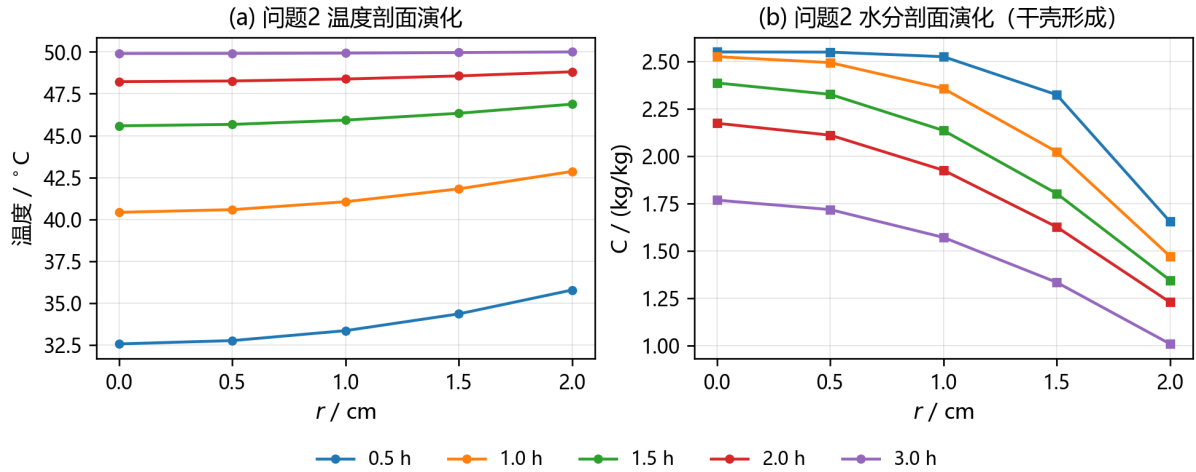


图 11 问题 2 温度与水分浓度剖面演化（干壳形成）

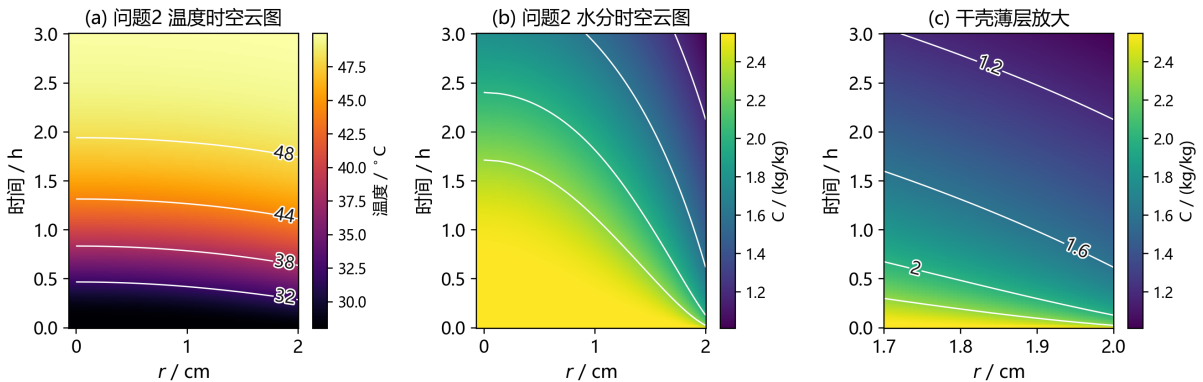


图 12 问题 2 温度与水分浓度时空云图（(c) 为 (b) 中干壳薄层的放大， $r > 1.7 \text{ cm}$ ）

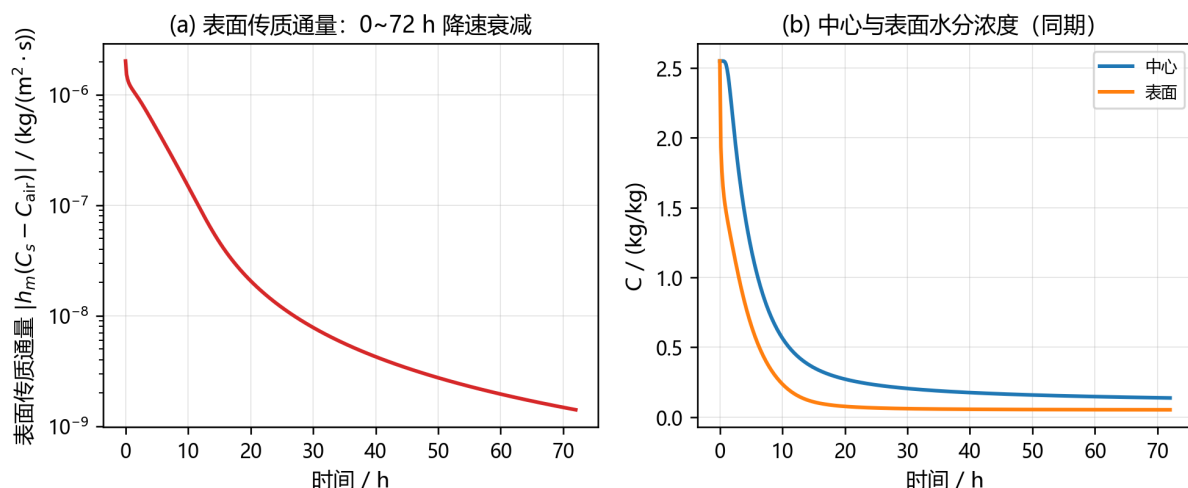


图 13 表面传质通量的降速特征 (0 ~ 72 h) 与中心/表面含水率对比

八、问题三：烘干时长的确定

8.1 判据的数学表述与关键判断

题目要求「药材各处的水分浓度应低于 0.15 kg/kg」，即

$$t_{\text{dry}} = \inf \left\{ t \geq 0 : \max_{0 \leq r \leq R_0} C(r, t) \leq 0.15 \right\}. \quad (15)$$

关键判断：中心点的扩散路径最长，预计是最后达到阈值的位置。本文用数值解逐时刻检验该判断，且最大值取在全部 $N + 1$ 个节点上，而不是只在输出位置上取最大（后者可能漏掉真正的极值点）。实测确认全部输出时刻 $\arg \max$ 都落在 $u = 0$ ，故 t_{dry} 由 $C(0, t)$ 首次达到 0.15 的时刻给出。数值上令事件函数 $g(t) = \max_j C_j(t) - 0.15$ ，用终端事件的稠密输出定位下降过零时刻。

平均值判据的误导性（图 14）：按 §11.6 的面积平均 $\bar{C}(t) = \int_0^1 C(u, t) du$ ， $\bar{C}$ 早在 35.8 h 就降到 0.15 kg/kg，但中心要到 57.1 h 才达标，滞后 21.3 h，以平均值判定会低估所需时间约 37.3%。此处必须用面积平均：若改用若干个物理半径采样点的算术平均（各点不等权），达标时刻会落到 35.6 h，与上述定义不符。

8.2 结果

$$t_{\text{dry}} = 205575.5 \text{ s} = 57.1043 \text{ h} = 2.3793 \text{ d}. \quad (16)$$

此时 $C(0, t_{\text{dry}}) = 0.1500$ （判据恰好满足），表面 $C_s = 0.0536 \text{ kg/kg}$ ，截面面积平均含水 $\bar{C} = 0.1243 \text{ kg/kg}$ 。表 7 给出每 6 h 的水分浓度分布。与题目「烘干过程一般持续 2~3 天」独立吻合且不贴边。

8.3 结果分析

降速期的定量特征（图 16、图 15）：用相邻表列值作中心差分， $\ln(-\dot{C}_0)$ 对 t 的斜率在 30 ~ 54 h 约为 -0.051 h^{-1} ，对应速率衰减的特征时间约 20 h；而同一区间内 C_0 自身的对数斜率只有 -0.011 h^{-1} 。速率衰减明显快于浓度衰减，这正是降速干燥期的标志。

表 7 药材烘干过程的水分浓度（单位：kg/kg）

时间/h	$r = 0$	$r = 0.5 \text{ cm}$	$r = 1.0 \text{ cm}$	$r = 1.5 \text{ cm}$	$r = 2.0 \text{ cm}$
6	1.0150	0.9863	0.9003	0.7548	0.5342
12	0.4547	0.4418	0.4020	0.3291	0.1651
18	0.2978	0.2906	0.2679	0.2247	0.0853
24	0.2370	0.2320	0.2161	0.1850	0.0673
30	0.2052	0.2012	0.1886	0.1637	0.0607
36	0.1853	0.1820	0.1713	0.1500	0.0575
42	0.1716	0.1687	0.1593	0.1404	0.0557
48	0.1614	0.1588	0.1503	0.1331	0.0546
54	0.1535	0.1511	0.1433	0.1274	0.0539
烘干结束	0.1500	0.1477	0.1402	0.1249	0.0536

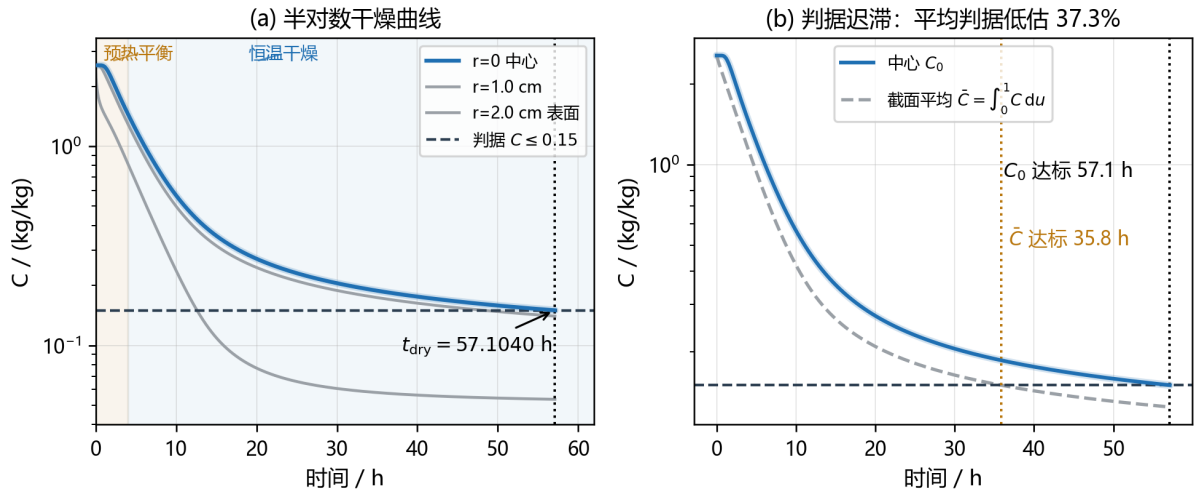


图 14 问题 3 半对数干燥曲线与判据迟滞效应

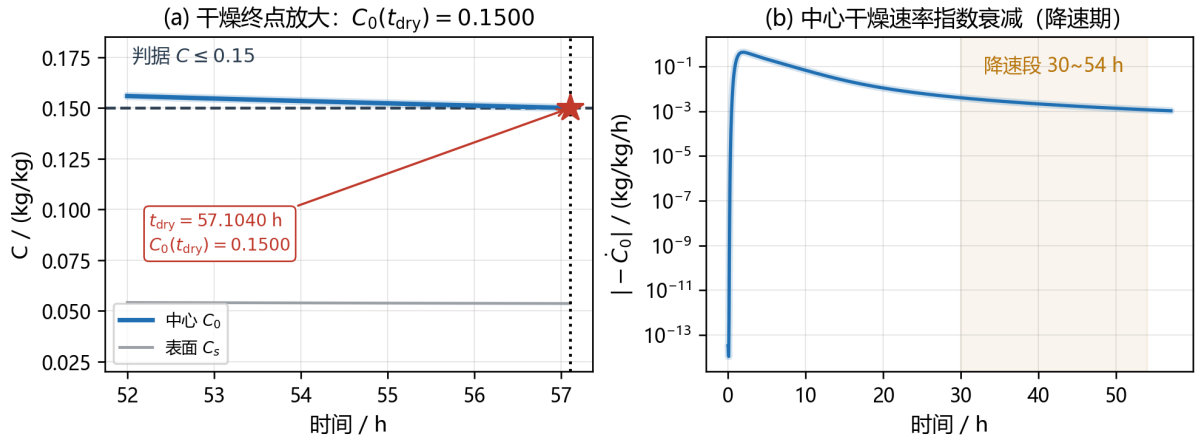


图 15 干燥终点附近放大与中心干燥速率的指数衰减

九、问题四：考虑尺寸收缩的求解

9.1 物质坐标与干物质守恒

当 $R = R(t)$ 时求解域随时间变化。引入物质坐标 $\xi = r/R(t) \in [0, 1]$, $r = \xi R(t)$ 。对各向同性收缩，同一物质点满足 $r(t) = \xi R(t)$ ，即 ξ 是材料标签；干物质不迁移，任

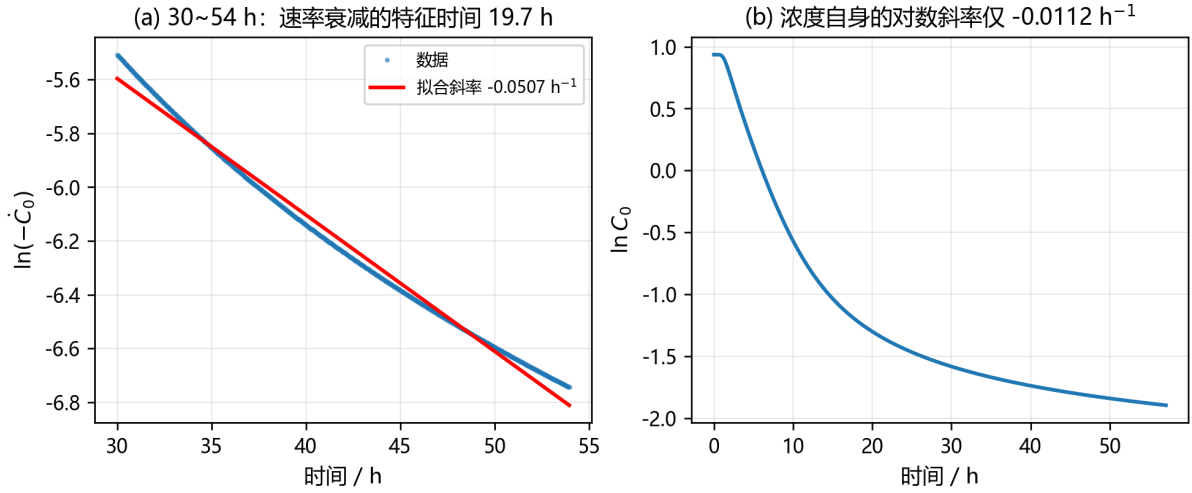


图 16 减速期的速率衰减与浓度衰减对比

一材料壳层的干物质质量保持不变。单位长度壳层的当前体积为 $2\pi r dr = 2\pi R^2 \xi d\xi$ ，故

$$\left. \frac{\partial}{\partial t} (\rho_d R^2) \right|_{\xi} = 0 \implies \rho_d(\xi, t) = \rho_{d0} \frac{R_0^2}{R(t)^2} \quad \text{与} \xi \text{ 无关,}$$

其中最后一步使用了初始干物质分布均匀这一条件。这里的 ρ_d 是干物质体积密度，不再用随含水率变化的经验湿物料密度 $\rho(C)/(1+C)$ 代替；经验 $\rho(C)$ 只用于式 (1) 中的表观体积热容。

9.2 水分守恒：材料导数吸收收缩运动

对固定材料区域 $[0, \xi]$ 做水分衡算。区域内水分只经由 ξ 处材料面与外界交换： $j_r = -\rho_d D \partial_r C = -(\rho_d D/R) \partial_{\xi} C$ ，材料面面积（单位长度） $2\pi \xi R$ ；利用 $\rho_d R^2$ 沿材料点守恒，有

$$2\pi R^2 \int_0^{\xi} \rho_d \xi' \frac{\partial C}{\partial t} d\xi' = -(j_r \cdot 2\pi \xi R) = 2\pi \xi \rho_d D \frac{\partial C}{\partial \xi}.$$

两边对 ξ 求导、约去 $2\pi \rho_d$ ：

$$\left. \frac{\partial C}{\partial t} \right|_{\xi} = \frac{4}{R(t)^2} \frac{\partial}{\partial u} \left(u D \frac{\partial C}{\partial u} \right), \quad u = \xi^2. \quad (17)$$

式中没有显式对流项，并非材料速度为零，而是骨架运动已包含在固定 ξ 的材料导数中。在表观热容近似下，温度方程写为

$$\left. \frac{\partial T}{\partial t} \right|_{\xi} = \frac{4}{R(t)^2 \rho(C) c_p(C)} \frac{\partial}{\partial u} \left(u k(C) \frac{\partial T}{\partial u} \right). \quad (18)$$

式 (18) 是显热温度模型在材料坐标下的延伸，不能解释为包含潜热、含湿焓输运和体积功的完整能量守恒。表面条件仍采用式 (9)，其中 $R = R(t)$ 。

9.3 一个必须避免的错误： r 坐标下不能直接写扩散方程

命题 3 固定 r 坐标下的正确方程是

$$\left. \frac{\partial C}{\partial t} \right|_r = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right) - \frac{\dot{R}}{R} r \frac{\partial C}{\partial r}, \quad (19)$$

不能删去最后一项。

证明. 设 $C(r, t) = \tilde{C}(\xi, t)$, $\xi = r/R(t)$, 则 $\partial_t C|_r = \partial_t \tilde{C}|_\xi - (\xi \dot{R}/R) \partial_\xi \tilde{C}$ 。代入式 (17) 并换回 r 导数 ($\partial_\xi = R \partial_r$): 第一项 = $(1/r) \partial_r (r D \partial_r C)$, 第二项 = $-(r/R)(\dot{R}/R)(R \partial_r C) = -(\dot{R}/R) r \partial_r C$ 。

物理检验: 取 $D = 0$ (水分完全被困在干物质中)。此时材料点含水率不变, $C(r, t) = f(r R_0 / R(t))$ 。由式 (19) 两端同等于 $-f'(\dot{R}/R) r (R_0 / R)$; 若删去对流项则得 $\partial_t C|_r = 0$, 即「含水率分布不随时间变化」, 与材料点含水率守恒不符。

9.4 输出坐标的约定

求解器工作在 $u = \xi^2$ 上, 而表格列名是当前物理距离 r , 两者由 $r = \xi R(t)$ 联系。故对每个输出时刻 t_i 与目标半径 r_j : $u_j = (r_j / R(t_i))^2$, 若 $r_j > R(t_i)$ 则该位置不存在 (留空)。若误在固定 ξ 剖面上插值, 会把 ξ 处的值标成 r 处的值: 以 $t = 6$ h、 $r = 1.0$ cm 为例, $\xi_{\text{误}} = 0.5$ 给出 $C = 1.3782$, 而真值为 1.0215, 误差 +35%。

9.5 求解结果

表 8 药材烘干过程的水分浓度 (问题 4, 含收缩; 列为当前物理距离, 单位 kg/kg)

时间/h	$R(t)$ /cm	$r = 0$	$r = 0.5$ cm	$r = 1.0$ cm	$r = 1.5$ cm	药材表面
6	1.374	1.7158	1.5346	1.0215	—	0.4228
12	1.248	0.7336	0.6513	0.4069	—	0.1677
18	1.214	0.4062	0.3665	0.2387	—	0.0899
24	1.204	0.2835	0.2596	0.1783	—	0.0681
30	1.201	0.2251	0.2084	0.1487	—	0.0603
36	1.200	0.1919	0.1788	0.1312	—	0.0568
42	1.200	0.1705	0.1597	0.1196	—	0.0550
48	1.200	0.1555	0.1462	0.1112	—	0.0539
烘干结束	1.200	0.1500	0.1413	0.1081	—	0.0535

注: 「—」表示该物理距离已位于收缩后的药材之外 ($r_j > R(t)$), 位置不存在故不填数值; $r = 1.5$ cm 列自 $t > 3.67$ h 起即为空。

$$t_{\text{dry}} = 182778.3 \text{ s} = 50.7717 \text{ h} = 2.1155 \text{ d}. \quad (20)$$

问题 3 与问题 4 同时改变了物性关系和半径, 不能用两者的直接差值识别纯收缩效应。为此补充受控场景: 保持问题 4 的附录 4 物性、边界条件、初值和算法不变, 仅令 $R(t) \equiv R_0$ 。 $N = 48$ 与 $N = 64$ 分别得到 464118.2 s 和 464120.2 s, 相对差为 4.4×10^{-6} 。三种场景汇总见表 9。

表 9 注: A 行取 $N = 128$ 连续极限值, B、C 两行取 $N = 64$ 生产值。B 行网格误差已量化: $N = 48$ 与 $N = 64$ 分别得 464118.2 s 与 464120.2 s, 相对差 4.4×10^{-6} , 换算到三行的相对变化上是第 5 位有效数字量级, 不影响「物性与收缩效应方向相反」这一结论。

表 9 物性变化与尺寸收缩的受控对照

场景	物性关系	半径	t_{dry}/s	t_{dry}/h
A: 问题 3	附录 3	R_0	205575.5	57.1043
B: 受控基准	附录 4	R_0	464120.2	128.9223
C: 问题 4	附录 4	$R(t)$	182778.3	50.7717

9.6 结果分析

物性效应与收缩效应方向相反（图 19）。在固定半径下，物性由附录 3 改为附录 4 后，烘干时长由 57.1043 h 增至 128.9223 h，增幅为

$$\frac{t_B - t_A}{t_A} = 125.8\%.$$

保持附录 4 物性不变并加入实测收缩后，烘干时长由 128.9223 h 降至 50.7717 h，纯几何收缩贡献为

$$\frac{t_C - t_B}{t_B} = -60.6\%.$$

问题 4 相对于问题 3 的直接差异为 $(t_C - t_A)/t_A = -11.1\%$ ，这是物性和几何共同变化后的净结果，不能表述为纯收缩贡献。半径由 2.000 cm 减至约 1.200 cm，扩散长度平方降至 $(R/R_0)^2 \approx 0.36$ ；尽管表面积同步减小，扩散距离缩短的加速作用仍占主导。**收缩完成后进入准等径干燥**： $t > 24$ h 后 $R \approx 1.20$ cm 基本不变，此后过程退化为半径 1.20 cm 的固定域问题。问题 4 物理坐标下随收缩移动的水分割面、收缩过程中的表面半径见图 17；物质坐标下的剖面与表 8 各物理距离处的干燥曲线见图 18。**表面在后段趋于准平衡**：表面浓度与环境平衡值之差 $|C_s - C_{\text{air}}|$ 在前段相当大——6 h 时为 0.372 kg/kg，12 h 时 0.117，24 h 时 0.017；直到约 48 h 才降到 0.003 kg/kg 以下（ t_{dry} 时为 0.0026）。也就是说「表面贴近环境平衡值」是干燥后期才成立的性质，不能表述为整个过程的性质；前期表面通量由这一较大的差值驱动，正是降速阶段的主要来源。

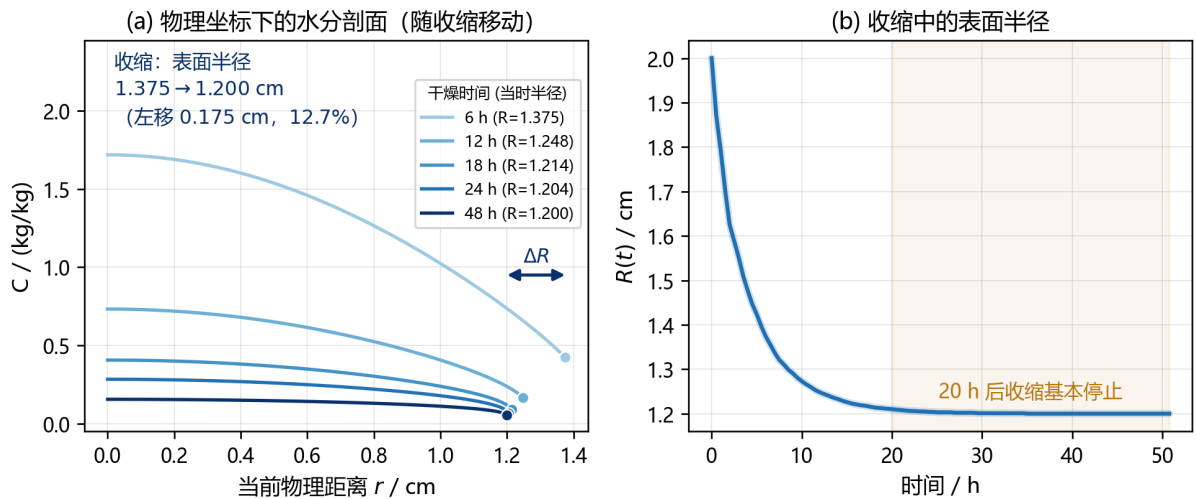


图 17 问题 4 物理坐标下的水分割面（随收缩移动）与表面半径变化

十、数值方法

本节按 §5.8 给出的正则化坐标 $u = \xi^2$ 展开：先说明离散格式，再交代时间积分与输出采样。

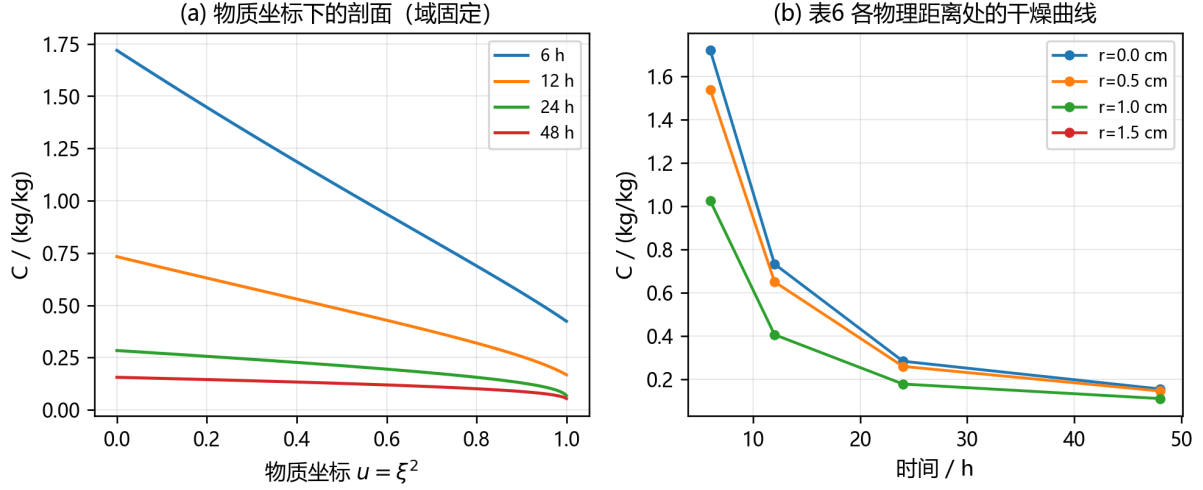


图 18 问题 4 物质坐标下的剖面与表 8 各物理距离处的干燥曲线

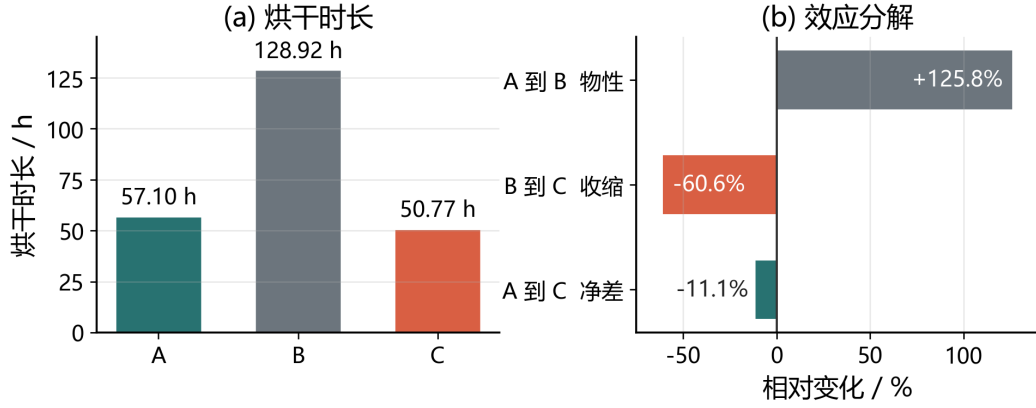


图 19 三种受控场景的烘干时长及物性、收缩效应分解

10.1 Chebyshev–Gauss–Lobatto 谱配置

取 $x_j = \cos(j\pi/N)$, $u_j = (1 - x_j)/2 \in [0, 1]$, 使 $u_0 = 0$ 、 $u_N = 1$ 。标准 Chebyshev 微分矩阵由

$$(D_x)_{ij} = \frac{c_i (-1)^{i+j}}{c_j x_i - x_j} \quad (i \neq j), \quad (D_x)_{ii} = -\sum_{j \neq i} (D_x)_{ij}, \quad c_0 = c_N = 2, \quad c_j = 1$$

给出；因 $u = (1 - x)/2$, 有 $D_u = -2D_x$ 。该构造是谱配置法处理边值问题的标准做法 [10]。

10.2 通量形式的配置

把式 (8) 写成通量散度形式。记节点通量 $g_j^{(C)} = u_j D_j (D_u C)_j$, 内部节点用谱微分给出的通量, 表面通量直接取自 Robin 物理条件 (9): $g_N^{(C)} = \frac{1}{2} h_m R (C_{\text{air}} - C_N)$, 于是

$$\frac{dC_j}{dt} = \frac{4}{R^2} (D_u \mathbf{g}^{(C)})_j, \quad j = 0, \dots, N. \quad (21)$$

温度通量为 $g_j^{(T)} = u_j k_j (D_u T)_j$, 表面取 $g_N^{(T)} = \frac{1}{2} h R (T_{\text{air}} - T_N)$, 半离散方程明确写为

$$\frac{dT_j}{dt} = \frac{4}{R^2 \rho(C_j) c_p(C_j)} (D_u \mathbf{g}^{(T)})_j, \quad j = 0, \dots, N. \quad (22)$$

注 2 (为何这种「混合通量」仍是谱精度) $\mathbf{g}$ 在除 u_N 外的节点上是插值多项式导数产生的通量, 在 u_N 上取物理边界通量 (解收敛时两者相等)。 $\mathbf{g}$ 因而逐点收敛到真通量 $g(u)$, 谱微分 $D_u \mathbf{g}$ 在 u_N 处谱收敛到 $g'(1)$; $u = 0$ 处 $u_0 = 0$ 使 $g_0 \equiv 0$ 自动满足对称性。 $\S$ 十一、的数值实验确认了这一论证。

10.3 时间积分与输出采样

时间方向采用变阶变步长隐式 BDF, 物性在每个右端项求值处按当前状态重新计算 (而非冻结在上一步)。生产设置: $N = 64$ 、 $\text{rtol} = 10^{-11}$ 、 $\text{atol}_C = 10^{-13}$ 、 $\text{atol}_T = 10^{-11}$; 另用 Radau 与 LSODA 交叉验证。输出用 CGL 节点上的重心插值

$$C(u_*) = \frac{\sum_j \frac{w_j}{u_* - u_j} C_j}{\sum_j \frac{w_j}{u_* - u_j}}, \quad w_j = (-1)^j \delta_j \quad (\delta_0 = \delta_N = \frac{1}{2}, \delta_j = 1),$$

数值稳定且与插值多项式等价 [11]。烘干时长用自适应积分器的终端事件机制, 由稠密输出做 Brent 求交, 不假设任何收敛阶。

十一、模型检验

本章按「解析验证 $\rightarrow$ 多方法互证 $\rightarrow$ 残差统计 $\rightarrow$ 收敛性 $\rightarrow$ 稳定性 $\rightarrow$ 守恒性」的顺序逐层检验。

11.1 解析解验证: 谱配置法对闭式解

问题 1 的温度场有闭式解式 (14), 可作为**绝对基准**。固定时间容差 10^{-12} 、只加密空间, 比较结果如表 10。

表 10 谱配置法与闭式解的比较 (问题 1 温度场, $t = 1800$ s)

N	4	6	8	12	24	64
全场最大误差 / K	2.6×10^{-10}	1.3×10^{-10}	1.09×10^{-10}	1.11×10^{-10}	1.14×10^{-10}	1.14×10^{-10}

误差在 $N = 8$ 就已落到时间积分误差平台上并不再随 N 下降——这正是谱精度成立的标志 (若边界通量处理有误, N 增大也不会收敛)。时间方向对应数据: $\text{rtol} = 10^{-8}$ 时误差 1.0×10^{-7} K、 10^{-10} 时 2.3×10^{-9} K、 10^{-12} 时 1.1×10^{-10} K (图 20)。

11.2 多方法互证: 三种独立路线

表 11 三条相互独立的路线互证 (问题 1)

量	A: 闭式解析解	B: 谱配置法 (本文)	C: 均匀网格守恒型有限体积
$T(0, 1800)$	34.0424989	34.042499	34.042498
$T(R_0, 1800)$	37.1989220	37.198922	37.198921
$C(R_0, 1800)$	—	1.51091810	1.51091918
$C(R_0, 1200)$	—	1.65914239	1.65914441

路线 C 采用与 A、B 都不同的技术: 均匀 r 网格、守恒型有限体积、等步长后向 Euler + Picard 迭代并在 $\Delta t \rightarrow 0$ 上做 Richardson 外推 (外推法的原始文献见 [12])。三法互差不超过 2×10^{-6} 。另在问题 2 的早时段 ($t = 0.5$ h、 $r = 0$) 与第三族对照格式 (节点中心 FV + θ 法) 比对: 谱解 32.565670279, 对照格式一阶外推 32.565670347, 两者相

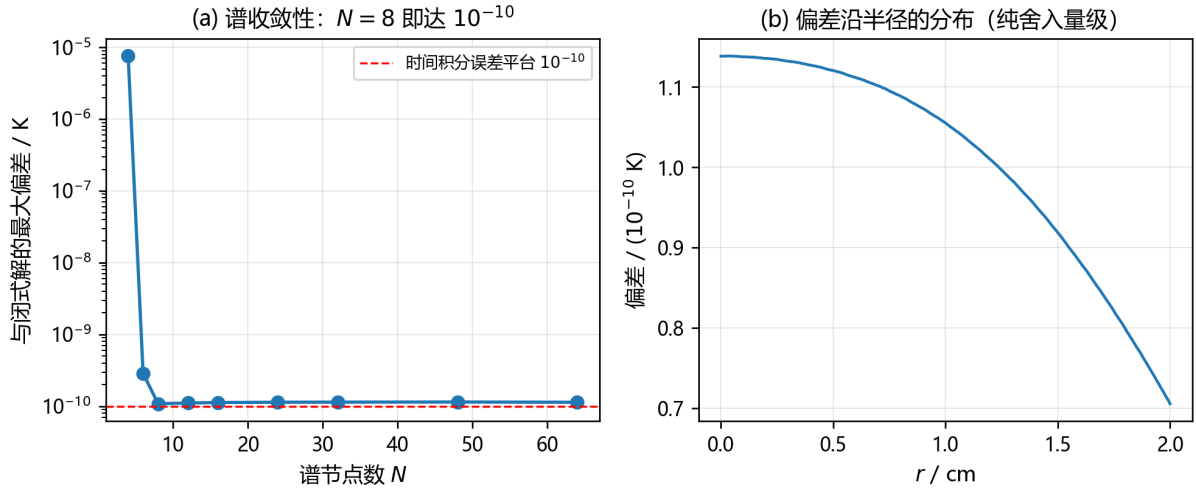


图 20 谱收敛性曲线与偏差沿半径的分布

差 $6.8 \times 10^{-8} ^\circ\text{C}$ (图 21)。三族方法互证说明: 三者解的是同一个 PDE, 且各自离散误差均已被压到目标精度以下。

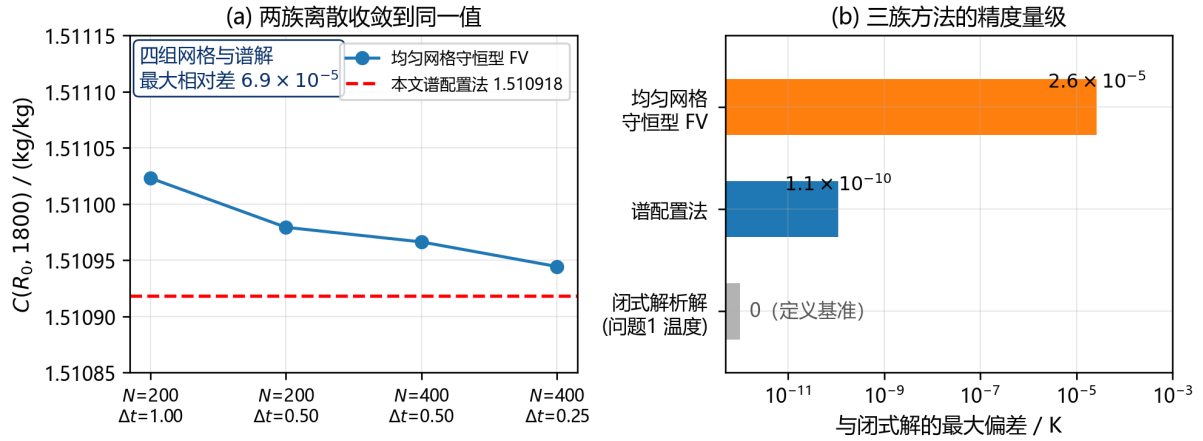


图 21 三族离散的互证与精度量级对比

11.3 残差统计特性

把 $N = 64$ 的谱解与闭式解逐点相减, 得到残差序列 (共 201 个空间点)。如图 22 所示, 残差量级为 10^{-10} K, 且全部落在 $7.1 \times 10^{-11} \sim 1.14 \times 10^{-10}$ K 的窄带内。需要明确的是, 这 201 个残差**全部同号** (Shapiro–Wilk 检验 $p \approx 3 \times 10^{-12}$), 因此它们**不是**随机舍入噪声, 而是时间积分器留下的**确定性离散偏差**; 其幅值随 N 增大不再下降 (表 10 中 $N \geq 8$ 后稳定在 10^{-10} K 量级), 说明空间方向已达谱精度、误差被时间积分平台接管。故这里能支持的结论是「空间离散没有残留的模型失配信号」, 而**不能**据此断言残差服从正态分布。

附件 1 的拟合残差同样做了统计诊断 (图 23): 温度残差标准差 $0.334 ^\circ\text{C}$ 、峰值残差 $0.96 ^\circ\text{C}$, 直方图近似正态; 但 Q-Q 图两端轻微偏离、滞后一阶自相关达 0.83, 说明除随机噪声外还存在**形式失配的系统性偏差**——这构成 § 十二、中环境口径灵敏度分析的动机。

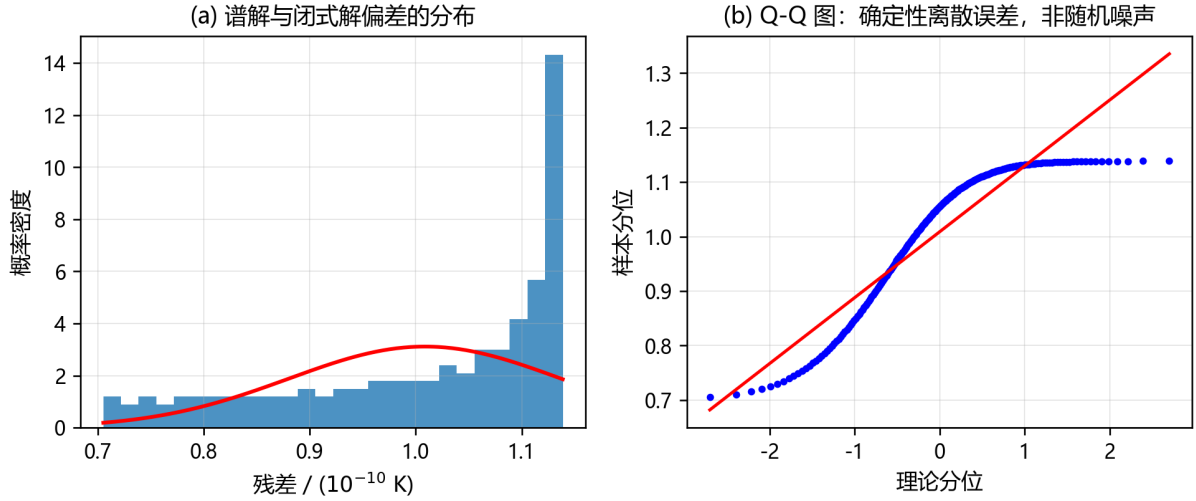


图 22 谱解与闭式解偏差的直方图与 Q-Q 图（偏差为确定性离散误差，非随机噪声）

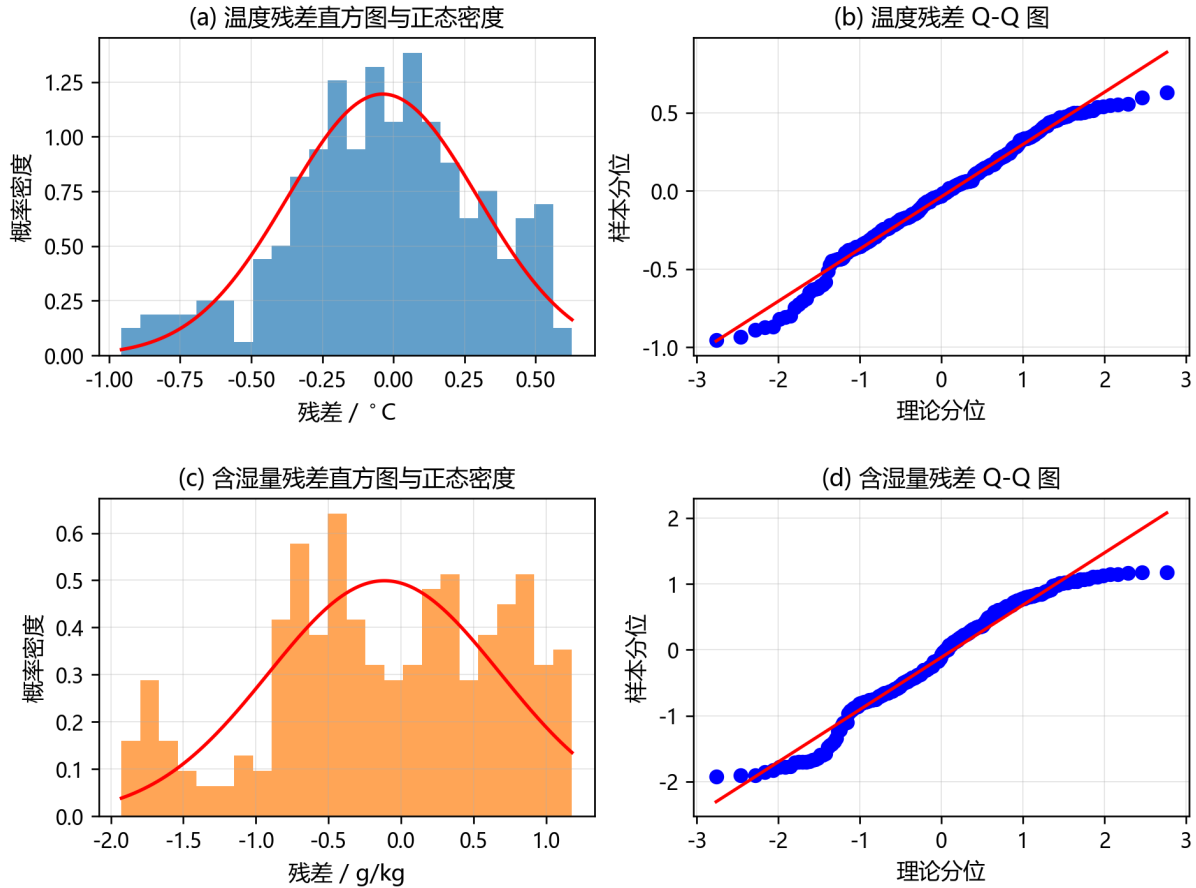


图 23 附件 1 拟合残差的直方图与 Q-Q 图（上：温度；下：含湿量）

11.4 收敛性分析

空间收敛（图 25(a)）： t_{dry} 随 N 的变化为 $205557.9 \rightarrow 205565.2 \rightarrow 205572.3 \rightarrow 205574.5 \rightarrow 205575.4 \rightarrow 205575.5 \text{ s}$ ($N = 24, 32, 48, 64, 96, 128$)， $N = 64$ 后变化已小于 1 s ， $N = 96 \rightarrow 128$ 仅变 0.1 s ，故取连续极限 205575.5 s 。本节的空间/时间扫描数据由脚本 `verify_data.py` 复跑产生并存入中间数据/verification.json，图 25 直接读该文件，绘图前会强制校验数据与解缓存一致（不一致即报错，不画图）。

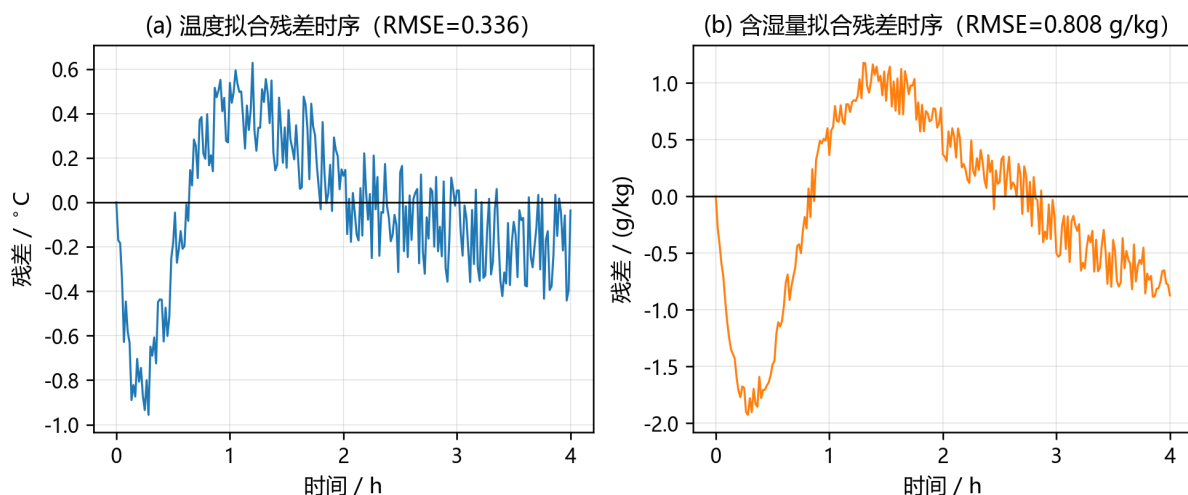


图 24 附件 1 拟合残差的时序（可见明显的系统失配区间）

报告值与交付文件的口径：正文报告的 205575.5 s 是 $N = 128$ 的连续极限值，而 result3.xlsx 按生产设置 $N = 64$ 生成，其终点为 205574.5 s，两者相差 1.0 s（相对差 5×10^{-6} ）。这一差异正是上表所列的空间离散误差，属同一模型的两种网格口径，不是两个不同结果。同理，§ 九、表 9 的 A 行取连续极限值，B、C 两行取 $N = 64$ 的生产值，B 行另给出 $N = 48$ 的复算值 464118.2 s 以说明该行差异确属网格误差；表 14 的灵敏度基准取 $N = 64$ 。

时间收敛（图 25(b)）：作为对照，用常见的节点中心有限体积 + θ 法（CN、前 4 步 Rannacher 启动、物性冻结于上一时间步）在 $N = 1600$ 下计算烘干时长并改变步长，结果如表 12。

表 12 定步长 θ 法的时间收敛性（问题 3， $N = 1600$ ）

$\Delta t / s$	40	20	10	5	2	1	0.5
t_{dry} / s	205508.5	205538.7	205553.8	205561.3	205565.9	205567.4	205568.1
相邻增量 / s	—	30.2	15.1	7.5	4.5	1.5	0.7

Δt 每减半、增量近似减半，是严格的 $\mathcal{O}(\Delta t)$ 一阶收敛：把相邻增量与步长的几何中点取双对数，拟合斜率为 1.01（一阶应取 1）。用最细三档步长对 $t(\Delta t) = L - c\Delta t$ 做最小二乘得 $c \approx 1.42 \text{ s/s}$ ，外推得该格式极限 205568.8 s，再加空间 Richardson 极限与时间修正后回到 205575.5 s，与本文谱解一致。一阶误差的来源是物性冻结于上一时间步，使实际推进只有 $\mathcal{O}(\Delta t)$ 相容。若在长时间段放宽步长（如取 $\Delta t = 10 \text{ s}$ 以节省计算量），仅此一项就会把烘干时长系统性低估约 15 s（相对偏差 0.008%）。

11.5 时间积分器与容差的稳定性

为避免单一积分器带来的偶然性，对问题 3 换积分器与容差重算：

表 13 烘干时长对时间积分器与容差的不敏感性（问题 3）

配置	BDF 10^{-9}	BDF 10^{-10}	BDF 10^{-12}	Radau 10^{-11}	LSODA 10^{-10}	1 s 密采样求交
t_{dry} / s	205574.525	205574.519	205574.518	205574.518	205574.519	205574.519

表 13 中六种积分器与容差配置（3 族积分器 $\times$ 4 个容差，另加 1 s 密采样求交）的极差仅 0.007 s（图 26）；「终端事件求交」与「1 s 密采样 + 线性求交」给出完全相同的

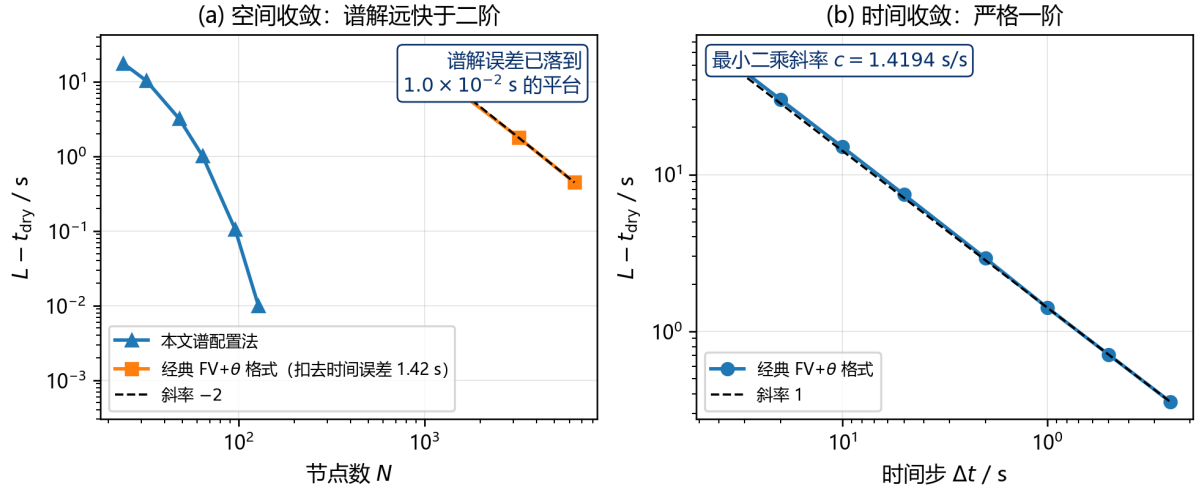


图 25 时间步收敛（严格一阶）与空间收敛（谱解远快于二阶）

值（差 0.000 s），说明事件定位本身没有引入误差。该时刻 $dC_0/dt = -2.94 \times 10^{-7} \text{ s}^{-1}$ ，即 1 s 的定位误差仅对应 $2.9 \times 10^{-7} \text{ kg/kg}$ 的浓度误差。

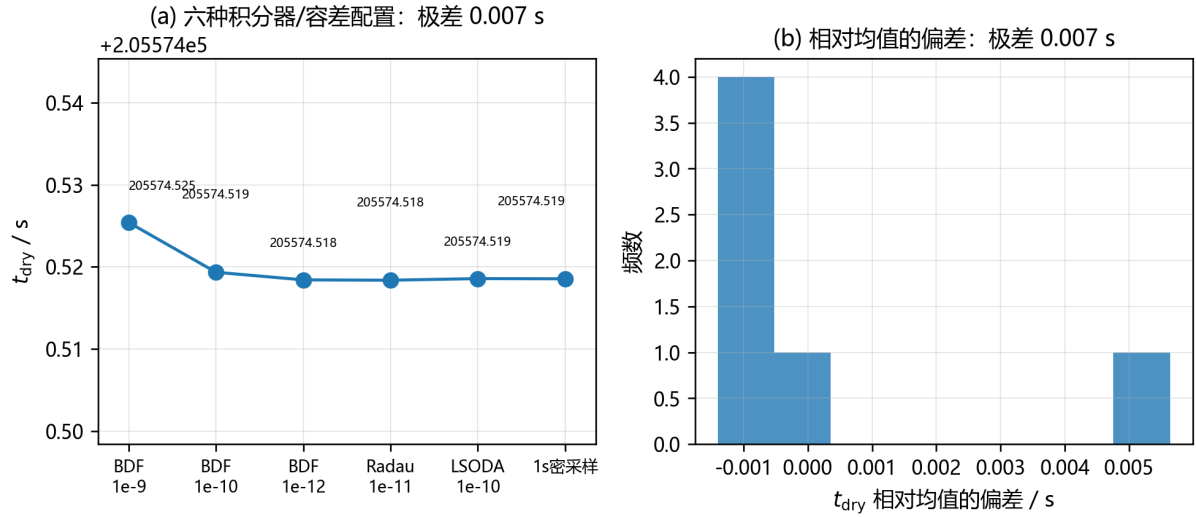


图 26 时间积分器与容差稳定性：全部配置的极差为 0.007 s

11.6 守恒性检验

固定半径时，单位长度药材的水分质量和截面面积平均含水率分别为

$$M_w(t) = \pi \rho_d R_0^2 \int_0^1 C(u, t) du, \quad \bar{C}(t) = \int_0^1 C(u, t) du.$$

由式 (2) 与 Robin 边界可得

$$\frac{dM_w}{dt} = -2\pi R_0 \rho_d h_m (C_s - C_{\text{air}}). \quad (23)$$

问题 4 中 $\rho_d R(t)^2 = \rho_{d0} R_0^2$ ，故同一校核仍成立，只需把边界周长改为 $2\pi R(t)$ 。数值上用高阶求积计算式 (23) 两侧，检验总量变化与表面流出量的一致性。原先的 $\int_0^1 2u C du$ 不是圆柱截面的面积平均量，现已改为上述正确表达。本文所有以「截面平均」报出的数字（问题一与问题三的结果分析，以及图 14(b) 的 $\bar{C}$ 曲线）一律按 $\bar{C} = \int_0^1 C du$ 计算；

实现上先由 Gauss–Legendre 求积把 Lagrange 基函数的积分算成一组固定权重，再用权重与节点值的内积逐时刻取值，故不依赖任何采样点数的选取。

十二、灵敏度与不确定度分析

本章的灵敏度分析均由脚本 `sensitivity.py` 重新求解，共 12 组参数扰动，指标为问题 3 的烘干时长 t_{dry} 。基准口径：基准解取 $N = 64$ （与 `result3.xlsx` 的生产网格一致）、附录 3 物性与一阶惯性拟合环境， $t_{\text{dry}} = 205574.5 \text{ s}$ ；表中所有相对变化都以该基准为分母。改用连续极限 205575.5 s 作分母只会把各相对变化改变 5×10^{-6} ，不影响排序与量级。

12.1 参数扰动结果

表 14 关键参数扰动对烘干时长的影响（基准： $N = 64$ ， $t_{\text{dry}} = 205574.5 \text{ s}$ ）

扰动	$t_{\text{dry}} / \text{s}$	相对变化	绝对变化 / s
$E + 2\%$ (3850 $\rightarrow$ 3927 K)	254624.0	+23.86%	+49050
$E - 2\%$ (3850 $\rightarrow$ 3773 K)	167355.9	-18.59%	-38219
$a + 2\%$ (0.45 $\rightarrow$ 0.459)	215664.2	+4.91%	+10090
$a - 2\%$ (0.45 $\rightarrow$ 0.441)	196057.7	-4.63%	-9517
$D_0 - 2\%$	209277.8	+1.80%	+3703
$D_0 + 2\%$	202019.9	-1.73%	-3555
$T_{\text{set}} - 0.2 \text{ K}$	206911.0	+0.65%	+1336
$T_{\text{set}} + 0.2 \text{ K}$	204250.0	-0.64%	-1325
$h_m - 2\%$	206055.5	+0.23%	+481
$h_m + 2\%$	205116.3	-0.22%	-458
$h - 2\%$	205580.3	+0.003%	+6
$h + 2\%$	205569.0	-0.003%	-6

表 14 的扰动口径如下，以免误读： E 、 a 、 D_0 三项只改物性式中的对应因子； h 、 h_m 只改传递系数； T_{set} 只是把烘房设定值平移 $\pm 0.2 \text{ K}$ ——因一阶惯性式的初值锚定在实测的 $T_{\text{air}}(0) = 28^\circ\text{C}$ 上，初始环境温度不受影响，被扰动的是设定值与整条升温轨迹。

主导不确定性的参数是活化温度 E ： $\pm 2\%$ 使 t_{dry} 变化 $+23.9\% / -18.6\%$ 。该响应明显不对称，升温方向的敏感性更强，这是 Arrhenius 因子 e^{-E/T_K} 的非线性所致（ E 增大时 D 的下降在低温段更快，而干燥前期温度较低）。活化浓度指数 a 次之（ $\pm 2\%$ 约 $\pm 4.8\%$ ）； D_0 、 T_{set} 、 h_m 的影响都在 2% 以内；而对流换热系数 h 的影响小于 0.01% ——因为温度在 2 h 内即达到准平衡， h 只影响升温的快慢，不影响后期的准等温干燥速率。

12.2 解析局部灵敏度对照

对 $D = D_0 e^{-a/C} e^{-E/T_K}$ 取对数，有 $\ln D = \ln D_0 - a/C - E/T_K$ ，故

$$\frac{\partial \ln D}{\partial T_K} = \frac{E}{T_K^2}, \quad \frac{\partial \ln D}{\partial a} = -\frac{1}{C}, \quad \frac{\partial \ln D}{\partial C} = \frac{a}{C^2}. \quad (24)$$

在 $T_K = 323.15 \text{ K}$ (50°C) 处， $\partial \ln D / \partial T_K = 3.69 \times 10^{-2} \text{ K}^{-1}$ ，表示其他变量局部不变时，温度偏差 1 K 使 D 约变化 3.7% 。这一局部导数不能直接替代全程烘干时长的扰动重算。在 $C = 0.15 \text{ kg/kg}$ 、 $a = 0.45$ 附近， $\partial \ln D / \partial a = -6.67$ ， a 增加 2% 即 $\Delta a = 0.009$ 时，线性近似给出 $\Delta \ln D \approx -0.060$ ，即局部 D 约下降 5.8% ；同时 $\partial \ln D / \partial C = 20.0 (\text{kg/kg})^{-1}$ 。该数量级与表中全模型重算所显示的后期敏感性相符。

12.3 环境口径的灵敏度

附件 1 的拟合残差存在系统性失配（图 24），故必须评估「如何把实测序列转成边界条件」这一建模选择的影响。两种口径的对比结果如表 15。**口径约定**：「原始序列插值」把温度与含湿量同时换回附件 1 的实测序列线性插值（而不是只换温度）； $t > 14400$ s 后两者都取各自的设定值。

表 15 环境口径对结果的影响（基准网格 $N = 64$ ）

环境口径	$T(0, 1800) / ^\circ\text{C}$	$T(R_0, 1800) / ^\circ\text{C}$	$C(R_0, 1800)$	$t_{\text{dry}} / \text{s}$
一阶惯性拟合（本文）	34.0425	37.1989	1.5109	205574.5
附件 1 原始序列插值	33.5753	36.7856	1.5102	205578.9
差值	-0.467	-0.413	-0.0007	+4.4

这是一个重要且反直觉的结论：环境口径对问题 1 的温度场影响高达 0.467°C （比所有数值误差之和大三个数量级），但对烘干时长的影响只有 $+4.4$ s（ 0.002% ）。原因是 t_{dry} 由长达 50 h 以上的恒温干燥段决定，而两种口径在该段的差异随 e^{-t/τ_T} 迅速衰减至零；反之前 30 min 的预热段正是环境口径差异最大的区间，故对问题 1 影响显著而对问题 3 几乎无影响。注意温度场的差异与含湿量口径无关：问题 1 中两场解耦，故切换 C_{air} 不改变 $T(0, 1800)$ 与 $T(R_0, 1800)$ 。

12.4 不确定度汇总

综合来看，本题结果的不确定度不是由数值方法决定的（数值误差 $< 0.01\%$ ），而是由以下三项决定，按贡献排序：

1. **物性经验式的标定精度**（主导）。 $E = 3850$ K 只要偏差 1% 就会使 t_{dry} 变化约 10%；而该参数由题目附录直接给定，无法由题面数据进一步校验。
2. **环境口径**（次要，且只影响早时段）。对问题 1 温度场影响 0.467°C ，对烘干时长影响 0.002% 。
3. **未建模的物理机制**（无法定量）。主要是表面蒸发潜热引起的降温，题目未提供汽化潜热与孔隙率，无法标定。

十三、模型的评价与推广

13.1 模型优点

方法与精度方面，变换 $u = (r/R)^2$ 将圆柱轴心正则化并使谱配置可用；问题 1 的常物性温度场在 $N = 8$ 时误差已达 10^{-10} 量级，非线性问题则依据网格加密选取 $N = 64$ 。准静态分解还给出 Bessel 闭式解，在本文验证时刻保留前 2 项即可达到机器精度。

验证与解释方面，闭式解、谱配置法和两族有限体积格式的互差不超过 2×10^{-6} ，且同时核验守恒性、收敛阶、积分器与容差稳定性；对事件时刻的一阶离散误差作了专门量化。实际扰动试验识别出 E 与 a 为主导参数，并以受控对照拆分物性变化与纯收缩效应，避免把跨问题净差异误判为单一几何因素。

13.2 模型缺点

模型的主要限制依次为：**物性标定不足**，主导参数 E 、 a 只能取自附录，宜以恒温薄层实验复核；**蒸发潜热缺失**，可能高估温度和扩散率；**热梯度驱动的水分迁移**未显式建模，该机制可产生额外水分通量 [13]；**各向同性仿射收缩**，不能描述非仿射或各向异性变形；**一维径向简化**，忽略端面对流与轴向梯度；**经验式外推风险**，题面未给出 $D(C, T)$ 的标定区间，故在 $C \rightarrow 0.15$ 附近仍需实验验证。

13.3 改进方向与推广

后续可用加热功率与排湿速率建立闭环烘房模型，以替代式 (4)；获得含湿焓、汽化潜热和界面相平衡数据后，可改用焓方程描述蒸发冷却。对球体仍可取 $u = \xi^2$ ，但须重新推导球坐标算子；平板无轴心奇异项，不必照搬该变换。谱配置框架可迁移至药物颗粒、食品和木材等体系，也可将阈值事件替换为有效成分保留率等目标；本文针对事件时刻的双格式误差核验方法同样可复用。就工艺本身而言，本文给出的「收缩使烘干时长缩短 60.6%」这一量级说明：按固定几何标定的工艺参数在中后期会系统性偏保守，分段温湿度等主动调控策略 [4] 值得进一步研究。

AI 工具使用声明。本参赛队在竞赛过程中使用了 AI 工具，主要用于候选模型比较、论文结构整理、语言润色、公式一致性检查与图表排版建议。模型方程、数值结果和结论均依据题目数据与可运行程序复核，详细使用情况见支撑材料《AI 工具使用详情.pdf》。

参考文献

- [1] Hussain M M, Dincer I. Two-dimensional heat and moisture transfer analysis of a cylindrical moist object subjected to drying: A finite-difference approach. *International Journal of Heat and Mass Transfer*, 2003, 46(21): 4033–4039.
- [2] Wang N, Brennan J G. A mathematical model of simultaneous heat and moisture transfer during drying of potato. *Journal of Food Engineering*, 1995, 24(1): 47–60.
- [3] Ao J, He W, Chen Y, Ai Z, Li T, Xiao H, Xiao Z. Modeling and analysis of heat and moisture transfer during *Fructus aurantii* drying based on CT scanning. *Drying Technology*, 2026, 44(8): 1183–1196.
- [4] Ju H-Y, Zhao S-H, Mujumdar A S, Zhao H-Y, Duan X, Zheng Z-A, Gao Z-J, Xiao H-W. Step-down relative humidity convective air drying strategy to enhance drying kinetics, efficiency, and quality of American ginseng root (*Panax quinquefolium*). *Drying Technology*, 2019, 38(7): 903–916.
- [5] Luikov A V. Systems of differential equations of heat and mass transfer in capillary-porous bodies (review). *International Journal of Heat and Mass Transfer*, 1975, 18(1): 1–14.
- [6] Whitaker S. Simultaneous Heat, Mass, and Momentum Transfer in Porous Media: A Theory of Drying. *Advances in Heat Transfer*, 1977, 13: 119–203.
- [7] 张卫鹏, 高振江, 肖红伟, 郑志安, 巨浩羽, 谢龙. 基于 Weibull 函数不同干燥方式下的茯苓干燥特性. *农业工程学报*, 2015, 31(5): 317–324.
- [8] Carslaw H S, Jaeger J C. *Conduction of Heat in Solids*. 2nd ed. Oxford: Clarendon Press, 1959.

- [9] Lewis W K. The Rate of Drying of Solid Materials. *Journal of Industrial & Engineering Chemistry*, 1921, 13(5): 427–432.
- [10] Trefethen L N. *Spectral Methods in MATLAB*. Philadelphia: Society for Industrial and Applied Mathematics (SIAM), 2000.
- [11] Berrut J-P, Trefethen L N. Barycentric Lagrange Interpolation. *SIAM Review*, 2004, 46(3): 501–517.
- [12] Richardson L F. IX. The approximate arithmetical solution by finite differences of physical problems involving differential equations, with an application to the stresses in a masonry dam. *Philosophical Transactions of the Royal Society of London. Series A*, 1911, 210: 307–357.
- [13] Philip J R, De Vries D A. Moisture movement in porous materials under temperature gradients. *Transactions, American Geophysical Union*, 1957, 38(2): 222–232.

A Bessel–Duhamel 展开系数推导

设 $\langle f, g \rangle = \int_0^R r f g \, dr$ 。函数 W 与特征函数 ϕ_n 满足同一齐次 Robin 条件，因而 Green 恒等式的边界项为零：

$$\int_0^R r [(\nabla^2 W) \phi_n - W(\nabla^2 \phi_n)] \, dr = R [W'(R) \phi_n(R) - W(R) \phi_n'(R)] = 0.$$

再利用 $\nabla^2 \phi_n = -(\beta_n^2/R^2) \phi_n$ 、 $\nabla^2 W = \kappa^2(A - W)$ ，左端被积函数化为 $\kappa^2 A r \phi_n - [(\beta_n^2/R^2) - \kappa^2] r W \phi_n$ ，于是

$$\kappa^2 A \langle 1, \phi_n \rangle - \left[\frac{\beta_n^2}{R^2} - \kappa^2 \right] \langle W, \phi_n \rangle = 0 \implies \langle W, \phi_n \rangle = -\frac{\kappa^2 A}{(\beta_n^2/R^2) - \kappa^2} \langle 1, \phi_n \rangle.$$

注意此处的负号： $\hat{w}$ 的初值是 $-W(r)$ ，展开系数 b_n 取的是 $-W$ 的带权投影，故该负号在 b_n 与 κ^2 前的符号相消。代入 $\langle 1, \phi_n \rangle = R^2 J_1(\beta_n)/\beta_n$ 与 $\|\phi_n\|^2 = \frac{R^2}{2} [J_0^2(\beta_n) + J_1^2(\beta_n)]$ ，得

$$b_n = \frac{\langle -W, \phi_n \rangle}{\|\phi_n\|^2} = \frac{A \kappa^2 N_n}{(\beta_n^2/R^2) - \kappa^2}, \quad N_n = \frac{2 J_1(\beta_n)}{\beta_n [J_0^2(\beta_n) + J_1^2(\beta_n)]}, \quad (25)$$

即正文式 (14) 中的 b_n 。本题所用参数与前四阶数值为

$$\begin{aligned} \alpha &= 1.688555 \times 10^{-7} \, \text{m}^2/\text{s}, \quad R_0^2/\alpha = 2368.89 \, \text{s}, \quad \text{Bi} = 1.388889, \quad \kappa R_0 = 1.123265, \\ \beta_{1..4} &= (1.4184725, 4.1664696, 7.2084785, 10.3082732), \\ b_{1..4} &= (47.2789, -0.65803, 0.096714, -0.027695). \end{aligned}$$

B 支撑材料清单

类别	文件	说明
源程序	common.py	参数、烘房环境标定、物性经验式、附件读取
	spectral.py	主求解器： $u = \xi^2$ 谱配置 + 自适应 BDF
	analytic_p1.py	问题 1 温度场 Bessel–Duhamel 闭式解
	fv_theta.py	对照格式：节点中心 FV + 定步长 θ 法
	fv_uniform.py	对照格式：均匀网格守恒型 FV + 后向 Euler
	run_all.py	一键复跑全部结果与验证
	make_results.py	生成 result1–4.xlsx
	sensitivity.py	灵敏度分析（12 组扰动 + 环境口径分叉）
	verify_data.py	复跑图 25/26 的收敛性与积分器数据，并做一致性校验
	make_figures.py、fig_core.py	生成全部论文插图、绘图样式与解缓存
结果数据	make_controlled_comparison.py	生成问题 4 受控对照图并复算固定半径基准
	console_utf8.py	控制台编码修复
	result1.xlsx	问题 1 温度/水分浓度， 1800×21
	result2.xlsx	问题 2 温度/水分浓度， 10800×21
	result3.xlsx	问题 3 水分浓度， 3427×21 （ $N = 64$ ，终点 205574.5 s）
中间数据	result4.xlsx	问题 4 水分浓度（含表面列）， 3047×22
	solutions.npz	四问的解缓存（含生成时的代码指纹，防止误用旧缓存）
	verification.json	图 25/26 的收敛性与积分器复跑数据
	sensitivity.txt/.npz	灵敏度分析原始输出
附件数据	附件 1.xlsx、附件 2.xlsx	赛题给出的烘房环境与半径实测序列；虽属赛题原始数据，仍随包保留，以便 common.py 按文件名自动定位附件、整包直接复跑
图表	fig01--fig26	论文插图 27 张 PNG（240 dpi 渲染，中文标签），其中 26 张被正文引用；未采用的 fig20_p4_vs_p3.png 不列入本清单（原因见下方说明）
AI 使用详情	AI 工具使用详情.pdf	AI 工具信息、使用环节与对输出的采纳情况

上表中 fig20_p4_vs_p3.png 为何未采用：该图直接把问题 3 与问题 4 的 t_{dry} 相减，而这两个问题同时改变了物性关系与几何，其差值 -11.1% 是二者共同作用的净结果，

把它当作”收缩效应”属于错误归因。本文改用同一物性下仅改半径的受控对照（表 9、图 19）来隔离收缩，纯收缩贡献为 -60.6% 。为避免被误引，该图不进入正文，也不列入本清单。

复跑方式（依赖 Python 3.9+ 与 numpy、scipy、openpyxl；其中 make_figures.py、make_controlled_comparison.py 重绘插图还需 matplotlib ≥ 3.5 ，其余脚本对 matplotlib 版本无要求）：

```
1 python run_all.py           # 全部结果 + 全部验证，约 4-5 分钟
2 python make_results.py      # 重新生成 result1-4.xlsx（写入 result/）
3 python sensitivity.py       # 重新运行灵敏度分析
4 python verify_data.py --recompute # 重建图 25/26 的验证数据（约 22 分钟）
5 python make_figures.py     # 重新生成全部插图（写入 figure/）
```

common.py 会在包内目录树中自动搜索附件路径，整个包可直接移动位置后运行。make_figures.py 会先校验 verification.json 与解缓存是否一致，不一致即中止并提示重建，避免画出与代码结果脱钩的图。

C 完整源程序代码

3.1 公共参数与环境标定：common.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # common.py —— 公共参数、烘房环境标定与物性经验式（四个问题共用）
4 # =====
5 # 做什么：集中定义四间共用的几何尺寸、传递系数与初值；把附件 1 的 241 组实测环境
6 # 数据标定成一阶惯性式  $T_{air}(t)/C_{air}(t)$ ；按题目附录 2/3/4 给出  $\rho$ 、 $c_p$ 、
7 #  $k$ 、 $D$  随含水率与温度变化的经验关系。主求解器与各验证脚本都从这里取物性，
8 # 故论文里写的物性与代码里算的是同一份。
9 # 输入：附件 1.xlsx（“时间/温度/水分浓度”实测序列，241 行）、附件 2.xlsx（“时间/半径”
10 # 序列，半径列单位 cm，仅问题 4 使用）；附录 2/3/4 的物性参数直接写在下面。
11 # 附件由 _find_attach() 在包内候选目录中按文件名搜索，故整个包可整体搬移。
12 # 输出：不写文件。对外提供  $R_0/L_c/h/hm/T_0/CO$  等常量，以及  $T_{air}$ 、 $C_{air}$ 、props、
13 #  $R_{of}$ 、bessel_roots 五个函数。
14 # 关键变量：
15 #  $R_0$  药材初始半径 2.000e-2 [m]
16 #  $L_c$  药材长度 0.25 [m]
17 #  $h$  对流换热系数 25.0 [W/(m2 K)]
18 #  $hm$  对流传质系数 8.0e-7 [m/s]
19 #  $T_0$  药材初温 28.0 [degC]
20 #  $CO$  药材初含水率（干基） 2.55 [kg/kg]
21 #  $T_{set}$  烘房温度惯性式稳态值 50.212 [degC]
22 #  $\tau_{air}$  烘房温度惯性时间常数 1877.5 [s]
23 #  $C_{set}$  烘房空气含湿量稳态值 0.050908 [kg/kg]
24 #  $\tau_{air}$  烘房空气含湿量时间常数 2776.3 [s]
25 #  $T_{END}$  惯性段/恒温段分段时刻 14400 s（附件 1 采样终点 4 h） [s]
26 # =====
27 import os
28 import numpy as np
29 import openpyxl
30 from scipy.optimize import curve_fit
31 from scipy.special import j0, j1
32 from scipy.optimize import brentq
33
34 # ----- 数据文件定位（可移植：相对本文件搜索） -----
35 # 为什么这么写：交付包会被解压到任意路径（且可能被整体移动），写死绝对路径必然失效。
36 # 候选顺序是“包根的题目/ → 包根 → 上一级 → 本目录”，覆盖压缩包原样解压与
37 # 只取 code/ 单跑两种用法。
38 _HERE = os.path.dirname(os.path.abspath(__file__))
39 _CANDIDATES = [
40     os.path.join(_HERE, "..", "题目"),
41     os.path.join(_HERE, ".."),
42     os.path.join(_HERE, "..", ".."),
43     _HERE,
44 ]
45
46 def _find_attach(name):
```

```

48 """在候选目录下查找 附件/<name>, 返回绝对路径; 全部落空则报错并列出搜索过的目录。
49
50 宁可直接抛 FileNotFoundError 也不要返回 None: 附件缺失时后续拟合会静默读到
51 错误数据, 报错比出错数好查。
52 """
53 for base in _CANDIDATES:
54     p = os.path.abspath(os.path.join(base, "附件", name))
55     if os.path.isfile(p):
56         return p
57     raise FileNotFoundError(
58         "找不到 附件/%s; 已搜索: %s" % (name, [os.path.abspath(b) for b in _CANDIDATES]))
59
60
61 # ----- 几何与传递系数 (附录 2) -----
62 # 圆柱侧面对流、两端绝热的假设下, 只有半径 RO 进入方程, 长度 Lc 仅用于报告体积。
63 RO = 0.02          # m 药材半径
64 Lc = 0.25          # m 药材长度
65 h = 25.0           # W/(m^2*K) 对流换热系数
66 hm = 8.0e-7        # m/s 对流传质系数
67
68 # ----- 初值 -----
69 T0 = 28.0          # degC
70 CO = 2.55          # kg/kg (干基)
71
72 # ----- 附件1: 烘房环境, 一阶惯性拟合 -----
73 def _load_env():
74     """读附件 1 的前三列, 返回 (时间 [s], 烘房温度 [degC], 空气含湿量 [kg/kg])。"""
75     ws = openpyxl.load_workbook(_find_attach("附件1.xlsx"), data_only=True).active
76     rows = list(ws.iter_rows(min_row=2, values_only=True))
77     t = np.array([float(r[0]) for r in rows])
78     Ta = np.array([float(r[1]) for r in rows])
79     Ca = np.array([float(r[2]) for r in rows])
80     return t, Ta, Ca
81
82
83 _t_env, _Ta_env, _Ca_env = _load_env()
84 T_air0 = _Ta_env[0]          # 28.0
85 C_air0 = _Ca_env[0]          # 0.01963
86 T_END = float(_t_env[-1])    # 14400 s
87
88
89 # 烘房升温/加湿的物理过程是一阶惯性 (送风量有限、围护结构有热容), 故用
90 #  $\phi(t) = \phi_{set} - (\phi_{set} - \phi(0)) \exp(-t/\tau)$  拟合, 只标定三个量:
91 # 稳态值、时间常数与初值。初值取实测首点, 不作为自由参数。
92 def _fT(t, Tset, tau):
93     """温度惯性式 (供 curve_fit 调用): t [s], 返回烘房温度 [degC]。"""
94     return Tset - (Tset - T_air0) * np.exp(-t / tau)
95
96
97 def _fC(t, Cset, tau):
98     """含湿量惯性式 (供 curve_fit 调用): t [s], 返回空气含湿量 [kg/kg]。"""
99     return Cset - (Cset - C_air0) * np.exp(-t / tau)
100
101
102 (Tset_fit, tauT_fit), _ = curve_fit(_fT, _t_env, _Ta_env, p0=[50.2, 1800.0])
103 (Cset_fit, tauC_fit), _ = curve_fit(_fC, _t_env, _Ca_env, p0=[0.0509, 2800.0])
104
105 # 标定参数的报告精度: 把拟合值按其有效位数保留 (Tset 到 0.001 degC、
106 # tau 到 0.1 s)。该取舍在 T(1800s) 上仅造成约 6e-5 degC 的变化,
107 # 远小于拟合自身的标准误差 (Tset: 0.030 degC、tauT: 12.7 s)。
108 Tset = 50.212
109 tauT = 1877.5
110 Cset = 0.050908
111 tauC = 2776.3
112 VARIANTS = {
113     "reported_precision": dict(Tset=Tset, tauT=tauT, Cset=Cset, tauC=tauC),
114     "full_precision_fit": dict(Tset=Tset_fit, tauT=tauT_fit,
115                                Cset=Cset_fit, tauC=tauC_fit),
116 }
117
118 # 拟合残差有两种报告口径, 看数时别混: 论文表 1 与 run_all.py 报的是 RMSE 用
119 #  $\div n$  的 0.336 degC / 8.1e-4 kg/kg (把标定值当作对整段序列的整体描述);
120 # curve_fit 的 pcov 走的是  $\div(n-2)$  的自由度口径, 只在需要参数标准误时使用。
121
122
123 # 工艺分段: 附件 1 在 t 约 4 h 达到设定值并稳定, 故 t>14400 s 直接取设定值。
124 # 注意该分段在 t=14400 s 处不连续: 一阶惯性式在该点的值与设定值尚差
125 # T_air: 1.037e-2 degC, C_air: 1.748e-4 kg/kg
126 # 这是"实测已达稳态, 故按稳态取值"这一建模选择的结果, 论文 S3.5 已注明量级;
127 # 若需要严格连续, 应把分段点后移或对惯性式做尾部截断, 两者都会改动 t_dry 的第 4 位有效数字。
128 def T_air(t):

```

```

129 """烘房温度 [degC]: t<=14400 s 取一阶惯性式, 其后取稳态设定值。
130
131 入参 t 可为标量或数组; 返回同形状数组 (后续用 numpy 广播, 标量入参也返回 0 维数组)。
132 """
133 t = np.asarray(t, dtype=float)
134 ramp = Tset - (Tset - T_air0) * np.exp(-t / tauT)
135 return np.where(t <= T_END, ramp, Tset)
136
137
138 def C_air(t):
139     """烘房空气含湿量 [kg/kg]: 分段口径与 T_air 完全相同 (同一时刻同一个 T_END)。"""
140     t = np.asarray(t, dtype=float)
141     ramp = Cset - (Cset - C_air0) * np.exp(-t / tauC)
142     return np.where(t <= T_END, ramp, Cset)
143
144
145 # ----- 物性经验式 -----
146 # 浓度下限: D 含 exp(-a/C), C=0 时该因子发散。物理上表面含水率也确实不会降到 0
147 # (烘干判据是 0.15 kg/kg), 故把求值用的 C 截断到 1e-3, 避免迭代中出现 inf/NaN。
148 C_FLOOR = 1.0e-3
149
150
151 def props(prob, C, T):
152     """返回 (rho, cp, k, D), 四个量与入参 C、T 同形状。
153
154     prob 决定用哪套物性 (对应题目三个附录):
155     1 -> 附录 2: 常数物性 (问题 1 用, 温度场与水分分解耦)
156     2、3 -> 附录 3: 含 C、T 的非线性关系 (问题 2、3 用)
157     4 -> 附录 4: 另一套系数 (问题 4 用, 配合实测收缩)
158
159     量纲: C 为干基含水率 [kg/kg]; T 为温度 [degC], 仅 Arrhenius 项内部换算成 K。
160     返回: rho [kg/m3]、cp [J/(kg K)]、k [W/(m K)]、D [m2/s]。
161     """
162     C = np.asarray(C, dtype=float)
163     T = np.asarray(T, dtype=float)
164     Cf = np.maximum(C, C_FLOOR)
165     TK = T + 273.15
166     if prob == 1:
167         one = np.ones_like(C)
168         rho = 820.0 * one
169         cp = 2600.0 * one
170         k = 0.36 * one
171         D = 7.0e-9 * np.exp(-0.89 / Cf)
172     elif prob in (2, 3):
173         rho = 650.0 + 128.0 * C
174         cp = 1450.0 + 2736.0 * C / (C + 1.0)
175         k = 0.21 + 0.38 * C / (C + 1.0)
176         D = 2.4e-3 * np.exp(-0.45 / Cf) * np.exp(-3850.0 / TK)
177     elif prob == 4:
178         rho = 760.0 + 90.0 * C
179         cp = 1850.0 + 2150.0 * C / (C + 1.0)
180         k = 0.12 + 0.20 * C / (C + 1.0)
181         D = 4.2e-4 * np.exp(-0.30 / Cf) * np.exp(-3850.0 / TK)
182     else:
183         raise ValueError("prob must be 1,2,3,4")
184     return rho, cp, k, D
185
186
187 # ----- 附件2: 半径历史 -----
188 def _load_radius():
189     """读附件 2, 返回 (时间 [s], 半径 [m]); 附件以 cm 记录, 此处统一乘 1e-2 换成 m。"""
190     ws = openpyxl.load_workbook(_find_attach("附件2.xlsx"), data_only=True).active
191     rows = list(ws.iter_rows(min_row=2, values_only=True))
192     t = np.array([float(r[0]) for r in rows])
193     R = np.array([float(r[1]) for r in rows]) * 1e-2 # cm -> m
194     return t, R
195
196
197 _t_R, _R_hist = _load_radius()
198
199
200 def R_of(t):
201     """问题4: 半径分段线性插值 (t 超出范围则取端点值)。"""
202     return np.interp(t, _t_R, _R_hist)
203
204
205 # ----- Bessel 特征值 -----
206 def bessel_roots(Bi, n):
207     """求 beta*J1(beta) = Bi*J0(beta) 的前 n 个正根 (升序)。
208
209     Bi = h*R0/k 为传热 Biot 数; 递推关系 J0'=-J1 保证该式正是 Robin 边界条件

```

```

210 k*dT/dr = -h*T 分离变量后的特征方程。求根用"先用 J1 的零点把实轴分段，
211 再在每段内用 brentq 收缩"的办法——不依赖初值猜测，且不会漏根或串根。
212 """
213 from scipy.special import jnp_zeros
214 z = jnp_zeros(1, n + 2) # J1 的正零点
215 roots = []
216 f = lambda b: b * j1(b) - Bi * j0(b)
217 # 第一个根在 (0, z0)
218 lo = 1e-12
219 hi = z[0] - 1e-12
220 if f(lo) * f(hi) < 0:
221     roots.append(brentq(f, lo, hi, xtol=1e-15, rtol=8.9e-16))
222 for i in range(len(z) - 1):
223     lo, hi = z[i] + 1e-12, z[i + 1] - 1e-12
224     if f(lo) * f(hi) < 0:
225         roots.append(brentq(f, lo, hi, xtol=1e-15, rtol=8.9e-16))
226     if len(roots) >= n:
227         break
228 return np.array(roots[:n])

```

3.2 主求解器: spectral.py

```

1 # -*- coding: utf-8 -*-
2 # =====
3 # spectral.py —— 主求解器:  $u=(r/R)^2$  坐标下的 Chebyshev 谱配置 + 自适应隐式时间积分
4 # =====
5 # 做什么: 在"径向坐标平方"  $u=(r/R)^2$  上同时求解水分浓度  $C$  与温度  $T$  两个耦合场, 四问共用。
6 # 换元后圆柱 Laplace 算子写成  $(4/R^2) d/du (u D d/du)$ , 在轴心  $u=0$  处完全正则,
7 # 故轴心与表面都不需要特殊控制体处理, 这是本方法区别于常规  $r$  坐标离散之处;
8 # 解在  $u$  上解析, 谱收敛成立。时间方向交给 SciPy 的自适应变阶隐式 BDF: 物性在
9 # 每个右端项求值处按当前  $(C, T)$  重算, 非线性被隐式格式直接吸收, 无需外层迭代。
10 # 输入: 不读文件。几何、环境与物性全部取自 common.py: R0、h、hm、T_air(t)、C_air(t),
11 # 其中环境式由附件 1 的 241 组实测数据标定, props() 按 prob 选题目附录 2/3/4 的
12 # 物性系数; 问题 4 的收缩半径  $R(t)$  取 common.R_of (附件 2 实测值, cm 已换算为 m)。
13 # 输出: 不写文件。对外提供 cheb_matrices() 与 HerbSolver 类, 被各结果、绘图与校验脚本
14 # (make_results.py、make_figures.py、run_all.py、verify_data.py 等)调用取解。
15 # 关键变量:
16 # N 谱节点数 (节点总数 N+1), 调用方常取 64; 空间误差随 N 指数下降
17 # u_j CGL 节点映射后的坐标  $u=(r/R)^2$ , 自 0 (轴心) 升到 1 (表面), 无量纲
18 # prob 1/2/3/4, 决定物性经验式; shrink=True 时启用 R(t) (仅问题 4)
19 # rtol 自适应时间积分的相对容差, 默认 1e-10, 调用方多用 1e-11 ~ 1e-12
20 # atol 绝对容差, 默认 1e-13; C [kg/kg] 与 T [degC] 量纲不同, 调用方按状态分段传数组
21 # =====
22 import numpy as np
23 from scipy.integrate import solve_ivp
24 import common as cm
25
26
27 def cheb_matrices(N):
28     """构造 CGL 节点  $x_j = \cos(j\pi/N)$  (自 1 降到 -1) 上的 Chebyshev 微分矩阵。
29
30     N 为多项式阶数 (节点共 N+1 个); 返回 (D, D@D, x): D 为一阶微分矩阵,
31     二阶直接取 D@D——在 CGL 节点上用一阶矩阵平方仍给出精确的谱微分矩阵,
32     不必另推二阶系数 (二阶矩阵随结果一并给出, 当前只用一阶)。
33     """
34     x = np.cos(np.pi * np.arange(N + 1) / N)
35     c = np.hstack([2.0, np.ones(N - 1), 2.0]) * ((-1.0) ** np.arange(N + 1))
36     X = np.tile(x, (N + 1, 1)).T
37     dX = X - X.T
38     D = np.outer(c, 1.0 / c) / (dX + np.eye(N + 1))
39     # 上面的公式在  $i=j$  处是 0/0, 对角元改用"本行其余元素之和取负":
40     # 这样 D 作用在常向量上严格得到 0 (常数的导数为零), 不吃浮点残差。
41     D -= np.diag(D.sum(axis=1))
42     return D, D @ D, x
43
44
45 class HerbSolver:
46     """四个问题统一的水分—温度耦合求解器。
47
48     N 谱节点数 (节点总数 N+1), 空间误差随 N 指数下降
49     prob 1/2/3/4, 决定 cm.props 用哪套物性经验式与环境口径
50     shrink 是否启用附件 2 的实测半径 R(t), 仅问题 4; 其余问题 R 恒为 R0
51     状态向量  $y = [C_0 \dots C_N, T_0 \dots T_N]$ , 前段单位 [kg/kg] (干基含水率),
52     后段单位 [degC]; 两段共用同一套  $u$  节点。
53     """
54

```

```

55 def __init__(self, N, prob, shrink=False):
56     self.N = N
57     self.Np = N + 1
58     self.prob = prob
59     self.shrink = shrink
60     # CGL 节点  $x_j = \cos(j\pi/N)$  自 1 降到 -1, 与  $u$  的换元关系是  $x = 1 - 2u$ ,
61     # 故  $u$  自  $u_0 = 0$  (轴心) 升到  $u_N = 1$  (表面), 且  $d/du = -2 d/dx$ 。
62     Dx, D2x, x = cheb_matrices(N)
63     self.u = (1.0 - x) / 2.0
64     self.Du = -2.0 * Dx
65     self.Np = N + 1
66
67 # ----- 几何与环境 -----
68 # 半径  $R(t)$  [m] 与烘房空气状态  $T_{air}(t)$  [degC]、 $C_{air}(t)$  [kg/kg];
69 # solve_ivp 需要右端项返回标量系数, 故这里统一 float() 化。
70 def R_of(self, t):
71     """当前半径  $R(t)$  [m]: shrink=True 时取附件 2 实测半径插值, 否则恒为 cm.R0."""
72     return float(cm.R_of(t)) if self.shrink else cm.R0
73
74 def T_air(self, t):
75     """烘房空气温度  $T_{air}(t)$  [degC]:  $t \leq 14400$  s 用一阶惯性式, 其后恒为 Tset."""
76     return float(cm.T_air(t))
77
78 def C_air(self, t):
79     """烘房空气含湿量  $C_{air}(t)$  [kg/kg]: 分段口径与  $T_{air}$  相同 (同一 T_END)。"""
80     return float(cm.C_air(t))
81
82 # ----- 右端项 -----
83 # 状态向量  $y$  的前  $N+1$  个分量是水分  $C$ , 后  $N+1$  个是温度  $T$ , 两者在同一组  $u$  节点上。
84 def rhs(self, t, y):
85     """右端项  $dy/dt$ , 供 solve_ivp 调用:  $t$  [s],  $y$  为状态向量 (含义见类说明)。
86
87     返回与  $y$  同形状的导数:  $dC/dt$  [kg/(kg s)] 与  $dT/dt$  [degC/s]。
88     """
89     N, Np = self.N, self.Np
90     C = y[:Np]
91     T = y[Np:]
92     R = self.R_of(t)
93     rho, cp, k, D = cm.props(self.prob, C, T)
94
95     # 水分: 通量形式  $g = u \cdot D \cdot dC/du$ , 方程即  $dC/dt = (4/R^2) dg/du$ 
96     gC = self.u * D * (self.Du @ C)
97     # 表面节点不取谱微分算出的通量, 而由 Robin 条件  $-D dC/dr = hm(C_s - C_{air})$ 
98     # 除以  $dr/du = R/(2 \sqrt{u})$  反解:  $g_N = (R/2) \cdot hm \cdot (C_{air} - C_N)$ ,
99     # 于是边界条件以物理通量的形式精确进入, 不引入谱微分的边界误差。
100     gC[N] = 0.5 * cm.hm * R * (self.C_air(t) - C[N]) # Robin 物理通量
101     dC = (4.0 / R ** 2) * (self.Du @ gC)
102
103     # 温度: 同上, 只是扩散系数换成  $k$ , 且被体积热容  $\rho \cdot cp$  除
104     gT = self.u * k * (self.Du @ T)
105     gT[N] = 0.5 * cm.h * R * (self.T_air(t) - T[N])
106     dT = (4.0 / (R ** 2 * rho * cp)) * (self.Du @ gT)
107
108     out = np.empty_like(y)
109     out[:Np] = dC
110     out[Np:] = dT
111     return out
112
113 # ----- 初值 -----
114 # 状态向量按 [C 段, T 段] 顺序拼接, 两场初始都是均匀场。
115 def y0(self):
116     """初始状态 [CO ... CO, TO ... TO]: CO = 2.55 [kg/kg]、TO = 28.0 [degC]。"""
117     return np.concatenate([np.full(self.Np, cm.CO), np.full(self.Np, cm.TO)])
118
119 # ----- 求解 -----
120 # 这里只做参数透传: method 可选 BDF (默认, 变阶自适应)、Radau、LSODA,
121 # 换积分器重算同一问题的目的是交叉验证时间离散误差。
122 def solve(self, t_end, t_eval=None, rtol=1e-10, atol=1e-13, method="BDF",
123          events=None, max_step=np.inf, first_step=None, dense=False):
124     """积分到  $t_{end}$  [s], 返回 scipy 的 OdeResult (sol.t [s]、sol.y = 状态矩阵)。
125
126      $t_{eval}$  指定输出时刻 [s]: rtol/atol 控制时间积分误差; events 用于定位烘干
127     时刻 (判据  $C_{max}$  下穿 0.15 kg/kg); method 换积分器做上述交叉验证。
128     """
129     return solve_ivp(self.rhs, (0.0, t_end), self.y0(), method=method,
130                      rtol=rtol, atol=atol, t_eval=t_eval, events=events,
131                      max_step=max_step, first_step=first_step, dense_output=dense)
132
133 # ----- 采样 -----
134 # 谱解活在  $u$  坐标上, 要还原成物理半径剖面或截面平均值都在这一节完成。
135 def r_nodes(self, t):

```

```

136     """当前时刻的物理半径节点  $r_j = R(t) * \sqrt{u_j}$  [m], 用于把谱解画到  $r$  坐标上."""
137     return np.sqrt(self.u) * self.R_of(t)
138
139 def _bary_weights(self):
140     """CGL 节点的重心插值权重  $w_j$  (两端点折半), 供 _bary 与面积权重共用."""
141     N = self.N
142     w = (-1.0) ** np.arange(N + 1)
143     w[0] *= 0.5
144     w[N] *= 0.5
145     return w
146
147 def area_weights(self):
148     """截面面积平均  $\bar{C} = \int_0^1 C(u, t) du$  在 CGL 节点上的权重 (缓存)。
149
150     论文 S11.6 给出的截面平均即此定义 ( $u=(r/R)^2$  下  $du$  已含  $2r dr$  的测度)。
151      $C$  在  $u$  上是  $N$  次多项式, 故用 Gauss--Legendre 求积对 Lagrange 基函数精确
152     积分一次即可得到权重向量; 此后任意时刻的平均值都是  $weights @ C\_nodes$ ,
153     与逐时刻插值求积完全等价但快得多。
154
155     注意: CGL 节点的**等权重和** (或若干物理半径采样点的算术平均) 都不是
156     面积平均——端点附近节点最密, 等权重会把近表面薄层放大, 结果偏低。
157     """
158     if getattr(self, "_area_w", None) is None:
159         nq = 2 * self.N + 2 # 被积函数为  $N$  次多项式, 留足余量
160         xg, wg = np.polynomial.legendre.leggauss(nq)
161         uq = 0.5 * (xg + 1.0) # 映射到  $[0, 1]$ , 节点严格在内部
162         wq = 0.5 * wg
163         w = self._bary_weights()
164         diff = uq[:, None] - self.u[None, :]
165         basis = (w[None, :] / diff) / np.sum(w[None, :] / diff, axis=1, keepdims=True)
166         self._area_w = wq @ basis
167     return self._area_w
168
169 def interp_u(self, y, u_t):
170     """把状态向量按  $u$  变量重心插值到  $u_t$  (谱精度), 返回 (C, T) 两个数组。
171
172      $y$  为某一时刻的状态向量;  $u_t$  为目标  $u$  坐标, 与物理半径的换算见  $r\_nodes$ 。
173     """
174     C = y[:self.Np]
175     T = y[self.Np:]
176     return (self._bary(C, u_t), self._bary(T, u_t))
177
178 def _bary(self, f, u_t):
179     """单场重心插值:  $f$  为 CGL 节点上的场值,  $u_t$  为目标  $u$  坐标, 返回同形状数组。"""
180     u_t = np.atleast_1d(np.asarray(u_t, dtype=float))
181     w = self._bary_weights()
182     u = self.u
183     out = np.empty(u_t.shape, dtype=float)
184     for k, ut in enumerate(u_t):
185         diff = ut - u
186         # 命中判据用相对容差: 绝对容差在  $u_t$  极接近某节点却不严格相等时会漏判,
187         # 该分支的  $1/diff$  会把插值结果污染成无意义的巨值。
188         hit = np.where(np.abs(diff) <= 1e-14 * max(1.0, abs(ut)))[0]
189         if hit.size:
190             out[k] = f[hit[0]]
191             continue
192         num = np.sum(w * f / diff)
193         den = np.sum(w / diff)
194         if den == 0.0:
195             # 数学上  $den = 1/prod(u_t - u_j)$  恒不为零, 但  $N$  较大且  $u_t$  落在
196             #  $[0, 1]$  之外时, 交替符号的项会在浮点下抵消到恰好 0, 此时  $num/den$ 
197             # 变成  $inf$ 。兜底取最近节点的值, 保证输出有限。
198             out[k] = f[int(np.argmin(np.abs(diff)))]
199         else:
200             out[k] = num / den
201     return out

```

3.3 问题 1 温度场闭式解: analytic_p1.py

```

1  # -*- coding: utf-8 -*-
2  # =====
3  # analytic_p1.py —— 问题 1 温度场的闭式解 (Bessel-Duhamel 特征函数展开)
4  # =====
5  # 做什么: 常物性、轴对称一维径向导热问题的解析解, 用来独立校核谱配置法 (spectral.py)
6  # 在问题 1 下的温度场并给出空间收敛阶的参照。解由准静态项与齐次瞬态级数两段
7  # 组成; 烘房温度按一阶惯性式  $T_{air}(t) = T_{set} - A * \exp(-t/\tau_T)$  变化。

```

```

8 # 输入：不读文件。常数物性取题目附录 2（直接写在下面）；R0、h、Tset、tauT、T0 取自
9 # common.py，其中的环境式由附件 1 的 241 组实测数据标定。
10 # 输出：不写文件。T_exact(r, t) 供 run_all.py、make_figures.py 做误差对照；
11 # 直接运行本模块会打印自检值（特征值残差、t=0 截断残差、nterm 敏感性）。
12 # 关键变量：
13 # RHO,CP,K 常数物性 820.0 [kg/m3]、2600.0 [J/(kg K)]、0.36 [W/(m K)]
14 # ALPHA 热扩散率 k/(rho*cp) = 1.6886e-7 [m2/s]
15 # BI 传热 Biot 数 h*R0/k = 1.3889（无量纲）
16 # A, KAPPA 惯性式幅值 Tset - T0 = 22.212 [degC]；准静态项波数 56.163 [1/m]
17 # BET,GAM 特征值 beta_n 与前 200 个衰减率 gamma_n = alpha*beta_n^2/R0^2 [1/s]
18 # BN 瞬态级数的 200 个系数 b_n（由初值展开定出）[degC]
19 # 方程与解法（0 < r < R0 的常物性径向导热）：
20 # rho*cp*dT/dt = (1/r) d/dr ( r*k*dT/dr ), dT/dr(0,t) = 0, T(r,0) = 28 degC.
21 # -k*dT/dr(R0,t) = h*(T(R0,t) - T_air(t)).
22 # 令 w = T - T_air(t) 使边界齐次，再作准静态分解 w = W(r)*exp(-t/tauT) + w_hat:
23 # W(r) = A + B*J0(kappa*r), B = h*A/(k*kappa*J1(kappa*R0) - h*J0(kappa*R0))
24 # w_hat = sum_n b_n*J0(beta_n*r/R0)*exp(-gamma_n*t), beta_n*J1(beta_n) = Bi*J0(beta_n)
25 # =====
26 import numpy as np
27 from scipy.special import j0, j1
28 import common as cm
29
30 # ----- 常数物性与派生量（题目附录 2） -----
31 # 问题 1 的温度场与水分场解耦，故 rho、cp、k 取常数，不随 C、T 更新。
32 RHO, CP, K = 820.0, 2600.0, 0.36 # [kg/m3] [J/(kg K)] [W/(m K)]
33 R = cm.R0 # 药材半径 R0 [m]
34 ALPHA = K / (RHO * CP) # 热扩散率 alpha = k/(rho*cp) [m2/s]
35 BI = cm.h * R / K # 传热 Biot 数 h*R0/k（无量纲）
36 A = cm.Tset - cm.T0 # 惯性式幅值 A = Tset - T0 [degC]
37 # 准静态项的波数 kappa [1/m]：把 w = W(r)*exp(-t/tauT) 代回方程后，指数因子上
38 # 剩下的常数项给出 kappa^2 = 1/(alpha*tauT)，量纲上正是 1/热扩散长度。
39 KAPPA = np.sqrt(1.0 / (ALPHA * cm.tauT))
40
41 # ----- 特征函数展开的系数（取 NROOT 项） -----
42 # 特征值 beta_n 由 Robin 条件分离变量得到，即 beta_n*J1(beta_n) = Bi*J0(beta_n) 的正根。
43 NROOT = 200
44 BET = cm.bessel_roots(BI, NROOT)
45 GAM = ALPHA * BET ** 2 / R ** 2 # 模态衰减率 gamma_n = alpha*beta_n^2/R0^2 [1/s]
46 # _NN 是归一化内积 N_n = 2*J1/(beta_n*(J0^2+J1^2))（J0、J1 在 beta_n 处取值），
47 # BN 则是瞬态级数系数 b_n：把初值 w_hat(r,0) = -W(r) 按 J0(beta_n r/R0) 展开得到，
48 # 其分母 beta_n^2/R0^2 - kappa^2 是本征波数与准静态波数的平方差：kappa 恰等于某个
49 # beta_n/R0 时该项发散，本题二者相差较远，各 b_n 都是有限值。
50 _NN = 2 * j1(BET) / (BET * (j0(BET) ** 2 + j1(BET) ** 2))
51 BN = A * KAPPA ** 2 * _NN / (BET ** 2 / R ** 2 - KAPPA ** 2)
52
53
54 def T_exact(r, t, Tset=None, tauT=None, nterm=NROOT):
55     """闭式解 T(r,t) [degC], r [m]、t [s]；返回与 r 同形状的数组。
56
57     Tset、tauT 不给就取 common.py 的标定值，传别的值可做环境参数敏感性分析；
58     nterm 为瞬态级数的截断项数，默认 NROOT = 200（n >= 2 的模态衰减很快）。
59     级数形式：T = T_air(t) + (A + B*J0(kappa*r))*exp(-t/tauT)
60                 + sum_n b_n*J0(beta_n*r/R0)*exp(-gamma_n*t)
61
62     """
63     Ts = cm.Tset if Tset is None else Tset
64     tau = cm.tauT if tauT is None else tauT
65     a = Ts - cm.T0
66     kap = np.sqrt(1.0 / (ALPHA * tau))
67     B = cm.h * a / (K * kap * j1(kap * R) - cm.h * j0(kap * R))
68     bn = a * kap ** 2 * _NN / (BET ** 2 / R ** 2 - kap ** 2)
69     r = np.atleast_1d(np.asarray(r, dtype=float))
70     val = np.full(r.shape, Ts - (Ts - cm.T0) * np.exp(-t / tau))
71     val += (a + B * j0(kap * r)) * np.exp(-t / tau)
72     for n in range(nterm):
73         val += bn[n] * j0(BET[n] * r / R) * np.exp(-GAM[n] * t)
74     return val
75
76 def _selftest():
77     """返回自检报告文本：特征值方程残差、t=0 处的全场极差与 nterm 敏感性表。
78
79     t=0 的极差就是瞬态级数在初值处的截断残差（理论上为 0），nterm 加密后该值
80     与 T(0,1800)、T(R0,1800) 是否变化，说明截断项数取得不够。供 run_all.py 打印。
81     """
82     out = []
83     out.append("alpha = %.6e m^2/s ; Bi = %.6f ; kappa = %.6f 1/m ; kappa*R = %.6f"
84               % (ALPHA, BI, KAPPA, KAPPA * R))
85     out.append("beta_1..4 = %s" % np.array2string(BET[:4], precision=10))
86     out.append("特征方程残差 max|beta_n*J1-Bi*J0| = %.3e"
87               % np.max(np.abs(BET * j1(BET) - BI * j0(BET))))
88     out.append("b_1..4 = %s" % np.array2string(BN[:4], precision=6))

```

```

89 out.append("gamma_n*t(1800) n=1..4 = %s" % np.array2string(GAM[:4] * 1800, precision=3))
90 rchk = np.linspace(0, R, 41)
91 out.append("t=0 全场极差 (截断残差, 应趋于 0) = %.3e" % np.ptp(T_exact(rchk, 0.0)))
92 out.append(" 注: t>0 时 n>=2 项已被 exp(-gamma_n t) 压到 e-13 以下, ")
93 out.append(" 故 t=1800 s 的结果对 nterm 不敏感 (见下)。")
94 for nt in [1, 2, 3, 4, 8]:
95     out.append("nterm=%d T(0,1800)=%.10f T(R,1800)=%.10f"
96               % (nt, T_exact(0.0, 1800.0, nterm=nt)[0],
97                 T_exact(R, 1800.0, nterm=nt)[0]))
98 return "\n".join(out)
99
100 if __name__ == "__main__":
101     print(_selftest())
102

```

3.4 对照格式一：节点中心有限体积 + θ 法：fv_theta.py

```

1  # -*- coding: utf-8 -*-
2  # =====
3  # fv_theta.py —— 对照格式：节点中心有限体积 + 定步长 theta 法 (CN 加 Rannacher 启动)
4  # =====
5  # 做什么：与 spectral.py 属于完全不同的离散族 (局部守恒、三对角、二阶空间精度、
6  #         定步长时间推进)，用来交叉验证主求解器，并考察"系数冻结的定步长格式"的
7  #         时间收敛阶。水分与温度共用同一套算子，只换 coef 与 kappa 两个系数。
8  # 输入：不读文件。几何、环境与物性取自 common.py (R0、hm、h、T_air、C_air、按题目
9  #       附录 2/3/4 取系数的 props、以及附件 2 的 R(t))；网格 N 与步长计划 dt_plan
10 #       由调用方 run_all.py、verify_data.py 给出。
11 # 输出：不写文件。simulate() 返回 xi 与各报告时刻的 C/T 快照 [kg/kg]、[degC]，
12 #       find_tdry() 返回烘干时刻 [s]。
13 # 关键变量：
14 #   N       径向控制体数 (节点共 N+1 个)，verify_data.py 取 1600
15 #   dt_plan 步长计划 [(t_lim, dt), ...]: t < t_lim 时用该 dt [s]
16 #   theta   时间格式权重, 0.5 即 Crank-Nicolson (二阶)，无量纲
17 #   n_ram   Rannacher 启动步数, 前 n_ram 步强制 theta = 1 (后向 Euler)
18 #   hi      烘干时刻搜索上限 3.0e5 [s]; crit 判据 C_max < 0.15 [kg/kg] (干基)
19 # 格式 (节点 xi_i = i/N; 体积测度 vol_0 = h2/8、vol_i = xi_i*h、vol_N = h/2 - h2/8) :
20 #   dphi_i/dt = (1/(R2*kappa_i*vol_i)) * (g_{i+1/2} - g_{i-1/2})
21 #   g_{i+1/2} = xi_{i+1/2}*coef_{i+1/2}*(phi_{i+1}-phi_i)/h (面系数取算术平均)
22 #   g_{i-1/2} = 0 (轴心对称)；g_{N+1/2} = -R*beta*(phi_N - phi_amb) (表面 Robin 通量)
23 #   水分: phi=C、coef=D、kappa=1、beta=hm、phi_amb=C_air; 温度换成 coef=k、
24 #       kappa=rho*cp、beta=h、phi_amb=T_air。物性冻结在上一时间步的值。
25 # =====
26 import numpy as np
27 from scipy.linalg import solve_banded
28 import common as cm
29
30
31 def simulate(N, prob, t_end, dt_plan, shrink=False, theta=0.5, n_ram=4,
32             report_idx=None):
33     """在给定步长计划下用 theta 法积分, 返回 (xi, out_t, out_C, out_T)。
34
35     N 为控制体数, t_end [s] 为终了时刻, prob、shrink 决定物性与是否启用 R(t);
36     theta、n_ram、dt_plan 见文件头。report_idx 是"必须记录的"列表 [s], 升序;
37     每步结束后把已越过的时刻快照进 out_*, 快照值取自步末。
38
39     与 fv_uniform.solve_fv 的口径差异 (两者都是 O(dt) 精度, 此处只作声明, 不试图
40     统一): 本函数把 R(t)、环境 T_air/C_air 取在**步首**, 而 solve_fv 取在**步末**
41     (t_new = n*dt)。差异量级为 O(dt), 故两者比较时必须各自做时间步外推, 不能直接
42     对比单步结果。
43     """
44     h = 1.0 / N
45     xi = np.arange(N + 1) * h
46     xif = (np.arange(N) + 0.5) * h
47     # 节点中心格式的体积测度 (已约去 2*pi): 节点 i 代表环形控制体
48     # int_{xi_{i-1/2}}^{xi_{i+1/2}} xi dxi, 故紧贴轴心与表面的半控制体各为 h2/8。
49     vol = np.empty(N + 1)
50     vol[0] = h * h / 8.0
51     vol[1:N] = xi[1:N] * h
52     vol[N] = h / 2.0 - h * h / 8.0
53
54     C = np.full(N + 1, cm.CO)
55     T = np.full(N + 1, cm.TO)
56     t = 0.0
57     step = 0
58     out_t, out_C, out_T = [], [], []
59     ri = 0

```

```

60
61 # 步长查表: 取第一个满足  $t < t\_lim$  的  $dt$ , 全不满足时用最后一项。调用方据此
62 # 实现"先用大步长跑过预热段、再切小步长"的时间推进。
63 def cur_dt(tt):
64     for (t_lim, d) in dt_plan:
65         if tt < t_lim:
66             return d
67     return dt_plan[-1][1]
68
69 while t < t_end - 1e-9:
70     dt = cur_dt(t)
71     if t + dt > t_end:
72         dt = t_end - t
73     R = float(cm.R_of(t)) if shrink else cm.R0
74     Cair = float(cm.C_air(t))
75     Tair = float(cm.T_air(t))
76     # Rannacher 启动: CN 对初值跳变敏感、会激发非物理的高频振荡, 故前  $n\_ram$  步
77     # 改用后向 Euler ( $\theta=1$ ) 先把快模态阻尼掉, 再切回  $\theta=0.5$  取二阶精度。
78     th = 1.0 if step < n_ram else theta
79
80     # 物性冻结在步首状态上、整步不再更新: 每步于是只解两个常数三对角系统,
81     # 代价是时间精度降为一阶 (这也是本格式被称为"系数冻结"的原因)。
82     rho, cp, k, D = cm.props(prob, C, T)
83
84     # 同一个 _operator 组装两个场: 水分取 coef=D、容量 kappa=1、表面系数 hm、
85     # 环境 C_air; 温度取 coef=k、容量 kappa=rho*cp、表面系数 h、环境 T_air。
86     Lc, cc = _operator(N, h, xi, xif, vol, D, np.ones(N + 1), R,
87                       cm.hm, Cair)
88     Lt, ct = _operator(N, h, xi, xif, vol, k, rho * cp, R,
89                       cm.h, Tair)
90
91     C = _theta_step(Lc, cc, C, dt, th)
92     T = _theta_step(Lt, ct, T, dt, th)
93     t += dt
94     step += 1
95
96     # 报告时刻可能落在步内部, 故只要本步末时刻越过它就记录, 快照取步末状态。
97     while ri < len(report_idx) and t >= report_idx[ri] - 1e-9:
98         out_t.append(t); out_C.append(C.copy()); out_T.append(T.copy())
99         ri += 1
100 return xi, out_t, out_C, out_T
101
102
103 def _operator(N, h, xi, xif, vol, coef, kappa, R, beta, amb):
104     """组装三对角算子 (a, up, lo) 与边界常数向量 c, 满足
105      $d\phi_i/dt = a_i\phi_i + up_i\phi_{i+1} + lo_i\phi_{i-1} + c_i$ 。
106
107     coef 为扩散/导热系数 (面上取算术平均), kappa 为容量 (水分取 1、温度取 rho*cp),
108     beta 为表面传递系数 (水分 hm、温度 h), amb 为环境值 (C_air 或 T_air), 单位同 phi。
109     i=0 侧由对称性  $g_{-1/2} = 0$  自然封闭, 故第 0 行只有一个面; i=N 侧把 Robin 通量
110      $-R\beta(\phi_N - amb)$  展开, 未知量留在对角、环境值进常数项 c。
111     """
112     a = np.zeros(N + 1) # 主对角
113     up = np.zeros(N + 1) # 上对角 (i -> i+1)
114     lo = np.zeros(N + 1) # 下对角 (i -> i-1)
115     c = np.zeros(N + 1)
116
117     face = 0.5 * (coef[:-1] + coef[1:]) # 面系数 (算术平均)
118     gf = xif * face / h # 内部面系数  $xi_{i+1/2} * coef/h$ 
119
120     inv = 1.0 / (R ** 2 * kappa * vol) # 每行都要除的体积热容因子
121     # i = 0: 轴心节点, 只有外侧一个面 ( $g_{-1/2} = 0$ ), 故只有一个非对角元
122     a[0] = -inv[0] * gf[0]
123     up[0] = inv[0] * gf[0]
124     # i = 1..N-1: 内部节点, 左右各一个面, 对角是两者的负和
125     i = np.arange(1, N)
126     a[i] = -inv[i] * (gf[i] + gf[i - 1])
127     up[i] = inv[i] * gf[i]
128     lo[i] = inv[i] * gf[i - 1]
129     # i = N: 表面节点, 左侧一个面再加上 Robin 通量项  $R\beta$ , 环境值进常数项 c
130     a[N] = -inv[N] * (gf[N - 1] + R * beta)
131     lo[N] = inv[N] * gf[N - 1]
132     c[N] = inv[N] * R * beta * amb
133     return (a, up, lo), c
134
135
136 def _theta_step(op, c, phi, dt, th):
137     """推进一个时间步, 解  $(I - dt*\theta*L) \phi_{new} = \phi + dt*((1-\theta)*L \phi + c)$ 。
138
139     op 为 _operator 返回的 (a, up, lo), th 为 theta (1 为后向 Euler、0.5 为 CN)。
140     显式部分只做乘法, 故右端不需要额外三对角求解; 矩阵按 solve_banded 要求的

```

```

141 (1, 1) 带宽装箱，不必构造稠密矩阵。
142 """
143 a, up, lo = op
144 n = len(phi)
145 lhs_a = 1.0 - dt * th * a
146 lhs_up = -dt * th * up[:-1]
147 lhs_lo = -dt * th * lo[1:]
148 rhs = phi + dt * (1.0 - th) * (a * phi + up * np.r_[phi[1:], 0.0]
149                               + lo * np.r_[0.0, phi[:-1]]) + c) + dt * th * c
150 ab = np.zeros((3, n))
151 ab[0, 1:] = lhs_up
152 ab[1, :] = lhs_a
153 ab[2, :-1] = lhs_lo
154 return solve_banded((1, 1), ab, rhs)
155
156
157 def find_tdry(N, prob, dt_plan, shrink=False, hi=300000.0, crit=0.15):
158     """从 t=0 单次前向推进，返回全场最大含水率首次下穿判据 crit 的时刻 [s]。
159
160     N、prob、dt_plan、shrink 同 simulate；hi 为搜索上限 3.0e5 [s]，到上限仍未
161     下穿则返回 nan；crit 为烘干判据 0.15 [kg/kg]（干基含水率）。判据取 max_r C
162     而不是平均值：轴心处水分最后才干，最大值达标即整个截面达标。
163     """
164     h = 1.0 / N
165     xi = np.arange(N + 1) * h
166     xif = (np.arange(N) + 0.5) * h
167     # 体积测度同 simulate：节点中心格式下由 int_{xi-1/2}^{xi+1/2} xi dxi 得到。
168     vol = np.empty(N + 1)
169     vol[0] = h * h / 8.0
170     vol[1:N] = xi[1:N] * h
171     vol[N] = h / 2.0 - h * h / 8.0
172
173     C = np.full(N + 1, cm.CO)
174     T = np.full(N + 1, cm.TO)
175     t = 0.0
176     step = 0
177     Cprev, tprev = None, None
178     while t < hi - 1e-9:
179         dt = dt_plan[-1][1]
180         for (t_lim, d) in dt_plan:
181             if t < t_lim:
182                 dt = d
183                 break
184         # 时间离散口径与 simulate 不同：半径取步末（与新解同点），环境取步中点值。
185         R = float(cm.R_of(t + dt)) if shrink else cm.R0 # 取步末半径
186         Cair = float(cm.C_air(t + 0.5 * dt)) # 环境取步中点值
187         Tair = float(cm.T_air(t + 0.5 * dt))
188         # 前 4 步 Rannacher 启动（与 simulate 的默认 n_ram=4 同口径）
189         th = 1.0 if step < 4 else 0.5
190         rho, cp, k, D = cm.props(prob, C, T)
191         Lc, cc = _operator(N, h, xi, xif, vol, D, np.ones(N + 1), R, cm.hm, Cair)
192         Lt, ct = _operator(N, h, xi, xif, vol, k, rho * cp, R, cm.h, Tair)
193         Cnew = _theta_step(Lc, cc, C, dt, th)
194         T = _theta_step(Lt, ct, T, dt, th)
195         # 记下本步起点 (tprev, Cprev)：判据下穿时靠这两点做插值求交。
196         cprev = float(np.max(C))
197         C = Cnew
198         tprev, Cprev = t, cprev
199         t += dt
200         step += 1
201         cnew = float(np.max(C))
202         if cnew < crit:
203             # Cmax 在 [tprev, t] 上线性插值求交
204             if Cprev is None or Cprev == cnew:
205                 return t
206             return tprev + (crit - Cprev) / (cnew - Cprev) * (t - tprev)
207     return float("nan")

```

3.5 对照格式二：均匀网格守恒型有限体积：fv_uniform.py

```

1 # -*- coding: utf-8 -*-
2 # =====
3 # fv_uniform.py —— 独立参照解：均匀 r 网格 + 守恒型有限体积 + 后向 Euler + Picard
4 # =====
5 # 做什么：用一套与主求解器完全不同的离散再解同一个初边值问题，两法一致即说明解的是
6 # 同一个 PDE，且各自的离散误差都已在目标精度以下。物性在每个时间步内用不动点

```

```

7 # (Picard) 迭代刷新, 故后向 Euler 的非线性不需要牛顿法。
8 # 输入: 不读文件。几何、环境与物性取自 common.py (R0、hm、h、T_air、C_air、按题目
9 # 附录 2/3/4 取系数的 props、附件 2 的 R(t)); 网格数 N、步长 dt [s] 与报告时刻
10 # 由调用方 run_all.py、make_figures.py 给出。
11 # 输出: 不写文件。solve_fv() 返回 (xi, C_tab, T_tab), sample_xi() 做线性插值采样。
12 # 关键变量:
13 # N 径向控制体数 (节点共 N+1 个), 均匀网格 xi_i = i/N
14 # dt 时间步长 [s]; 后向 Euler 等步长推进, 时间方向一阶
15 # n_picard 每步的物性迭代次数, 默认 3 (系数冻结在上一轮迭代值上)
16 # R(t) 不收缩时恒为 R0 = 2.000e-2 [m]; hm 8.0e-7 [m/s]; h 25.0 [W/(m2 K)]
17 # 体积测度 vol_0 = h^2/8、vol_i = xi_i*h、vol_N = h/2 - h^2/8, 与 fv_theta.py 同
18 # 与主求解器的三处刻意差别: 坐标用均匀 r 而非 u = xi^2; 离散用守恒型有限体积 (面通量
19 # 加体积测度) 而非谱微分; 时间用等步长后向 Euler (一阶) 而非自适应变阶 BDF。
20 # =====
21 import numpy as np
22 import common as cm
23
24
25 def solve_fv(N, dt, t_end, prob, t_report, shrink=False, n_picard=3):
26     """积分到 t_end [s] 并采样, 返回 (xi, C_tab, T_tab)。
27
28     N 为网格数, dt [s] 为等步长, prob、shrink 决定物性与是否启用 R(t) (见文件头),
29     n_picard 为每步的物性迭代次数; t_report 为报告时刻数组 [s]。
30     xi —— 无量纲节点坐标 i/N, 长度 N+1;
31     C_tab —— 各报告时刻的含水率 [kg/kg], 形状 (len(t_report), N+1);
32     T_tab —— 各报告时刻的温度 [degC], 形状同上; 未落在整步上的时刻保持 nan。
33     """
34     h = 1.0 / N
35     xi = np.arange(N + 1) * h
36     xif = (np.arange(N) + 0.5) * h # 内部面 xi_{i+1/2}, i=0..N-1
37
38     # 节点中心格式的体积测度 (已约去 2*pi): 节点 i 代表环形控制体
39     # int_{xi_{i-1/2}}^{xi_{i+1/2}} xi dxi, 故紧贴轴心与表面的半控制体各为 h^2/8。
40     vol = np.empty(N + 1)
41     vol[0] = h * h / 8.0
42     vol[1:N] = xi[1:N] * h
43     vol[N] = h / 2.0 - h * h / 8.0
44
45     C = np.full(N + 1, cm.CO)
46     T = np.full(N + 1, cm.TO)
47
48     nstep = int(round(t_end / dt))
49     trep = np.asarray(t_report, dtype=float)
50     # 报告时刻按"步号 = round(t/dt)"定位, 落在半步上的时刻取不到值, 其行保持 nan。
51     idx = {int(round(t / dt)): k for k, t in enumerate(trep)}
52     Ctab = np.full((len(trep), N + 1), np.nan)
53     Ttab = np.full((len(trep), N + 1), np.nan)
54
55     def R_of(t):
56         return float(cm.R_of(t)) if shrink else cm.R0
57
58     # 半径与环境一律取步末 t_new: 后向 Euler 的解就落在 t_new, 系数与状态取同一时刻。
59     # fv_theta.simulate 取的是步首, 两者相差 O(dt), 故对比时必须各自做时间步外推。
60     for n in range(1, nstep + 1):
61         t_new = n * dt
62         t_old = (n - 1) * dt
63         R = R_of(t_new)
64         Cair = float(cm.C_air(t_new))
65         Tair = float(cm.T_air(t_new))
66
67         # Picard 迭代: 物性 D、k、rho、cp 依赖未知的 C、T, 于是先把它们冻结在上一轮
68         # 迭代值上 (首轮用上一时间步的解) 解出新的 C、T, 再刷新物性, 如此重复 n_picard 次。
69         Cnew, Tnew = C.copy(), T.copy()
70         for _ in range(n_picard):
71             rho, cp, k, D = cm.props(prob, Cnew, Tnew)
72             # ---- 水分三对角: 隐式项整理成 (I - dt*L) C_new = b ----
73             # 内点的两个面各占一个非对角元; 轴心侧因 g_{-1/2} = 0 只剩一个面;
74             # 表面节点的 Robin 通量把 hm 折进对角, 把 hm*C_air 移到右端 b。
75             A = np.zeros((3, N + 1))
76             b = C.copy()
77             for i in range(N + 1):
78                 diag = 1.0
79                 if i == 0:
80                     gf = xif[0] * 0.5 * (D[0] + D[1]) / h
81                     coef = dt * gf / (R ** 2 * vol[0])
82                     diag += coef
83                     A[0, i] = -coef
84                 elif i == N:
85                     gf = xif[N - 1] * 0.5 * (D[N - 1] + D[N]) / h
86                     coef_i = dt * gf / (R ** 2 * vol[N])
87                     coef_b = dt * (R * cm.hm) / (R ** 2 * vol[N])

```

```

88         diag += coef_i + coef_b
89         A[2, i] = -coef_i          # 系数 C_{N-1} 在次对角
90         b[i] += coef_b * Cair
91     else:
92         gf_p = xif[i] * 0.5 * (D[i] + D[i + 1]) / h
93         gf_m = xif[i - 1] * 0.5 * (D[i - 1] + D[i]) / h
94         cp_ = dt * gf_p / (R ** 2 * vol[i])
95         cm_ = dt * gf_m / (R ** 2 * vol[i])
96         diag += cp_ + cm_
97         A[0, i] = -cp_
98         A[2, i] = -cm_
99         A[1, i] = diag
100     Cnew = _thomas(A, b)
101     # ---- 温度三对角 ----
102     # 结构与水分完全相同，只把 D 换成 k，并在体积测度上再除以节点热容 rho*cp。
103     A = np.zeros((3, N + 1))
104     b = T.copy()
105     for i in range(N + 1):
106         diag = 1.0
107         if i == 0:
108             gf = xif[0] * 0.5 * (k[0] + k[1]) / h
109             coef = dt * gf / (R ** 2 * rho[0] * cp[0] * vol[0])
110             diag += coef
111             A[0, i] = -coef
112         elif i == N:
113             gf = xif[N - 1] * 0.5 * (k[N - 1] + k[N]) / h
114             coef_i = dt * gf / (R ** 2 * rho[N] * cp[N] * vol[N])
115             coef_b = dt * (R * cm.h) / (R ** 2 * rho[N] * cp[N] * vol[N])
116             diag += coef_i + coef_b
117             A[2, i] = -coef_i          # 系数 T_{N-1} 在次对角
118             b[i] += coef_b * Tair
119         else:
120             gf_p = xif[i] * 0.5 * (k[i] + k[i + 1]) / h
121             gf_m = xif[i - 1] * 0.5 * (k[i - 1] + k[i]) / h
122             cp_ = dt * gf_p / (R ** 2 * rho[i] * cp[i] * vol[i])
123             cm_ = dt * gf_m / (R ** 2 * rho[i] * cp[i] * vol[i])
124             diag += cp_ + cm_
125             A[0, i] = -cp_
126             A[2, i] = -cm_
127         A[1, i] = diag
128     Tnew = _thomas(A, b)
129
130     C, T = Cnew, Tnew
131     # 报告时刻按步号命中，快照直接存该步末的整条剖面。
132     if n in idx:
133         Ctab[idx[n]] = C
134         Ttab[idx[n]] = T
135     return xi, Ctab, Ttab
136
137
138 def _thomas(A, b):
139     """追赶法 (Thomas 算法) 解三对角系统  $Ax = b$ ，返回解向量  $x$ 。
140
141     A 为 (3, n) 的紧凑带: A[0] = 上对角 (偏移 +1)、A[1] = 主对角、A[2] = 下对角
142     (偏移 -1)。不选主元: 本模块的矩阵由隐式扩散加正的对角项构成，主对角严格大于
143     两个非对角元之和 (严格对角占优)，消元过程中不会出现零主元。
144     """
145     n = len(b)
146     cp = np.zeros(n)
147     dp = np.zeros(n)
148     cp[0] = A[0, 0] / A[1, 0]
149     dp[0] = b[0] / A[1, 0]
150     for i in range(1, n):
151         m = A[1, i] - A[2, i] * cp[i - 1]
152         cp[i] = A[0, i] / m
153         dp[i] = (b[i] - A[2, i] * dp[i - 1]) / m
154     x = np.zeros(n)
155     x[-1] = dp[-1]
156     for i in range(n - 2, -1, -1):
157         x[i] = dp[i] - cp[i] * x[i + 1]
158     return x
159
160
161 def sample_xi(xi, vals, x_target):
162     """把节点值 vals 线性插值到目标位置 x_target (无量纲坐标 i/N)。
163
164     x_target 可为标量或数组，返回同形状结果; 超出 xi 范围时取端点值 (np.interp 默认)。
165     """
166     return np.interp(np.asarray(x_target, float), xi, vals)

```

3.6 一键复跑全部结果与验证: run_all.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # run_all.py —— 一键复跑: 问题 1-4 的全部结果与全部验证
4 # =====
5 # 做什么: 把四个问题的求解、三族离散格式互证、网格与时间收敛性、烘干时间 t_dry
6 #         与关键答案汇总串成一趟跑完, 过程与结果写成一份文本报告; 只算与打印,
7 #
8 #         不生成 xlsx、不画图。
9 # 输入: common.py (物性、烘房环境、初值; 附件 1/2 由它定位)、analytic_p1.py
10 #       (问题 1 闭式解)、spectral.py (主求解器)、fv_theta.py 与 fv_uniform.py
11 #       (两个对照离散族)、verify_data.py (只读验证数据, 做一致性校验)。
12 # 输出: 复跑报告.txt, 写到当前工作目录; xlsx 与插图由别的脚本生成。
13 # 运行方式与依赖顺序:
14
15 #     1) python verify_data.py --recompute   重建验证数据 (约 22 min, 可跳过)
16 #     2) python run_all.py                 全部结果与验证 (实测约 6.5 min)
17 #     python run_all.py --fast             再跳过时间步/积分器敏感性扫描
18 #     3) python make_results.py 生成 result/*.xlsx, make_figures.py 出图
19 #     出错时异常直接抛出、停在出错那一节; 把上面的模块逐个单独跑即可定位。
20 # 关键变量:
21
22 # CRIT      烘干判据的干基含水率阈值 0.15                      [kg/kg]
23 # RCOLS     报告表用的 5 个物理半径 0.0/0.5/1.0/1.5/2.0        [cm]
24 # N         谱节点数, 扫描 24/32/48/64/96/128, 生产值取 64
25 # atol      时间积分绝对容差按状态排列 (C_0..C_N, T_0..T_N) 分两段给: 水分
26 #           1e-13、温度 1e-11; rtol 取 1e-9~1e-12
27 # FAST      命令行带 --fast 时为 True, 跳过最耗时的敏感性扫描
28
29 # =====
30
31 # ----- 控制台编码 (必须在任何 print 之前) -----
32 # 中文进度信息只有先把 stdout 切到 UTF-8 才能正确输出, 故 fix_console() 必须排在
33 # 本文件所有 print 之前; 它只改编码, 不参与任何计算。
34 from console_utf8 import fix_console
35 fix_console()
36
37 # ----- 误 import 的防护 -----
38 # 本脚本没有 main(), 复跑逻辑直接写在模块顶层, import 即整段执行并改写 复跑报告.txt,
39 # 故拦一道: 误 import 立刻报错, 而不是悄悄跑十几分钟、顺手覆盖掉已有报告。
40 if __name__ != "__main__":
41     raise RuntimeError(
42         "run_all 的复跑逻辑在模块顶层, import 会立刻重跑全流程并覆盖 复跑报告.txt。"
43         "请用 `python run_all.py` 运行。")
44
45 import sys
46 import time
47 import numpy as np
48 import common as cm
49 import analytic_p1 as an
50 from spectral import HerbSolver
51 from fv_theta import simulate as fv_theta_run, find_tdry as fv_theta_tdry
52 from fv_uniform import solve_fv, sample_xi
53
54 FAST = "--fast" in sys.argv
55 OUT = [] # 报告全文先攒在内存, 末尾一次性落盘, 中途报错不会留半份文件
56 TO = time.time() # 计时起点, 末尾打印总耗时
57
58
59 def w(s=""):
60     """写一行报告: 同时进内存缓冲与屏幕 (flush=True 便于重定向时时看到进度)。"""
61     OUT.append(str(s))
62     print(str(s), flush=True)
63
64
65 def hdr(t):
66     """打一节标题: 空行 + 上下两条 78 字符分隔线, 其余与 w 相同。"""
67     w("")
68     w("=" * 78)
69     w(t)
70     w("=" * 78)
71
72
73 # 报告表统一用这 5 个物理半径 [m]: 0.0、0.5、1.0、1.5、2.0 cm, 即论文表 1~表 6 的 5 列。
74 RCOLS = [0.0, 0.005, 0.010, 0.015, 0.020]
75
76 # ----- 第 0 节: 烘房环境标定 (附件 1) -----
77 # 这里用 common.py 里那套拟合参数把 RMSE、R2、标准误本地重算一遍做独立复核:
```

```

78 # 参数取自 curve_fit 的拟合结果, 误差统计自己写 (见 _stats), 不引用 curve_fit 的 pcov.
79 hdr("0. 烘房环境标定 (附件1, 一阶惯性拟合, 初值固定为实测值)")
80 _te, _Ta, _Ca = cm._t_env, cm._Ta_env, cm._Ca_env
81
82
83 def _mT(t, Ts, tau):
84     """温度惯性式 (与 common._fT 同形, 供 _stats 复算残差): t [s], 返回 [degC]."""
85     return Ts - (Ts - cm.T_air0) * np.exp(-t / tau)
86
87
88 def _mC(t, Cs, tau):
89     """含湿量惯性式 (与 common._fC 同形): t [s], 返回空气含湿量 [kg/kg]."""
90     return Cs - (Cs - cm.C_air0) * np.exp(-t / tau)
91
92
93 def _stats(f, p, t, y):
94     """算一次拟合的 RMSE、决定系数 R2 与两个参数的近似标准误, 返回 (rmse, r2, se).
95
96     f 是模型函数, p 是参数向量, t 是自变量 [s], y 是实测值.
97     RMSE 用  $\sqrt{\frac{1}{n} \sum (y_i - f(t_i, p))^2}$  口径: 把标定值当作对整段序列的整体描述, 与 common.py 报告的口径一致.
98     标准误另用  $\sqrt{\frac{1}{n-2} \sum (y_i - f(t_i, p))^2}$  的自由度口径算, 因为要的是参数自身的统计不确定度.
99     Jacobian 用前向差分求, 步长取  $|p_i| \times 1e-7 + 1e-12$  (防止参数为 0 时步长归零); 标准误取  $(J^T J)^{-1}$  对角元乘 s2 后开方, 即线性化后的参数标准差.
100
101     """
102     r = y - f(t, *p)
103     rmse = float(np.sqrt(np.mean(r ** 2)))
104     r2 = 1.0 - float(np.sum(r ** 2) / np.sum((y - y.mean()) ** 2))
105     J = np.column_stack([(f(t, *p + np.eye(len(p))[i] * (abs(p[i]) * 1e-7 + 1e-12))
106                          - f(t, *p)) / (abs(p[i]) * 1e-7 + 1e-12)
107                          for i in range(len(p))])
108     s2 = float(np.sum(r ** 2) / (len(t) - len(p)))
109     se = np.sqrt(np.diag(s2 * np.linalg.inv(J.T @ J)))
110     return rmse, r2, se
111
112
113 _pT = np.array([cm.Tset_fit, cm.tauT_fit])
114 _pC = np.array([cm.Cset_fit, cm.tauC_fit])
115 rmseT, r2T, seT = _stats(_mT, _pT, _te, _Ta)
116 rmseC, r2C, seC = _stats(_mC, _pC, _te, _Ca)
117 w("T_set = %.6f degC (标准误 %.4f)   tau_T = %.4f s (标准误 %.2f)"
118   % (cm.Tset_fit, seT[0], cm.tauT_fit, seT[1]))
119 w("C_set = %.6f kg/kg (标准误 %.2e)   tau_C = %.4f s (标准误 %.2f)"
120   % (cm.Cset_fit, seC[0], cm.tauC_fit, seC[1]))
121 w("温度拟合: RMSE = %.4f degC, R2 = %.6f" % (rmseT, r2T))
122 w("湿度拟合: RMSE = %.3e kg/kg, R2 = %.6f" % (rmseC, r2C))
123 w("初值固定为附件1 实测首点: T=%.2f degC, C=%.6f kg/kg" % (cm.T_air0, cm.C_air0))
124 w("生产取值 (按有效位数报告): Tset=%.3f tauT=%.1f Cset=%.6f tauC=%.1f"
125   % (cm.Tset, cm.tauT, cm.Cset, cm.tauC))
126 w("两阶段工艺: 0<t<=14400 s 按上式, t>14400 s 恒定为 (Tset, Cset)。")
127
128 # ----- 第 1 节: 闭式解自检 -----
129 # an._selftest() 返回一段文本: 特征方程残差 max|beta*J1-Bi*J0|、t=0 的全场极差
130 # (即瞬态级数在初值处的截断残差) 与 nterm 敏感性表。它不做自动断言, 需人眼判读:
131 # 残差应在机器精度量级 (1e-12 上下), t=0 极差应远小于 1e-6, nterm 加到 2 以上后
132 # T(0,1800) 不应再变。
133 hdr("1. 问题1 —— 闭式解自检 (Bessel-Duhamel)")
134 w(an._selftest())
135
136 # ----- 第 2 节: 问题 1 谱解的空间收敛性 -----
137 # 只加密空间 (N=8-64): 问题 1 是常物性线性问题, 谱方法误差随 N 指数下降, 故从很粗的
138 # N=8 起扫也能看出谱收敛; 时间容差固定在最紧的 rtol=1e-12 (atol 水分 1e-14、温度
139 # 1e-12), 使残差完全由空间离散主导。对照量是 analytic_p1.T_exact 的闭式解。
140 hdr("2. 问题1 —— 谱配置法 vs 闭式解 (只加密空间, rtol=1e-12)")
141 w("N 全场max误差/K   T(0,1800)误差/K   T(R,1800)误差/K")
142 for N in [8, 12, 16, 24, 32, 64]:
143     s = HerbSolver(N, prob=1)
144     at = np.concatenate([np.full(s.Np, 1e-14), np.full(s.Np, 1e-12)])
145     sol = s.solve(1800.0, t_eval=[1800.0], rtol=1e-12, atol=at)
146     rc = np.linspace(0, cm.R0, 201)
147     Ti = s._bary(sol.y[s.Np:, -1], (rc / cm.R0) ** 2)
148     Te = an.T_exact(rc, 1800.0)
149     w("N=5d %13.3e %16.3e %17.3e"
150       % (N, np.max(np.abs(Ti - Te)), Ti[0] - Te[0], Ti[-1] - Te[-1]))
151
152 # ----- 第 3、4 节: 问题 1 的表 1 与表 2 -----
153 # 报告 7 个时刻 (100-1800 s, 每 300 s 一个点) × 5 个半径, 即论文表 1、表 2 的排版;
154 # 生产网格 N=64, atol 按状态排列 (C_0..C_N, T_0..T_N) 分两段给: 水分 1e-13、温度 1e-11。
155 hdr("3. 问题1 —— 表1 (温度 / degC)")
156 s1 = HerbSolver(64, prob=1)
157 at1 = np.concatenate([np.full(s1.Np, 1e-13), np.full(s1.Np, 1e-11)])
158 tt1 = np.array([100., 300., 600., 900., 1200., 1500., 1800.])

```

```

159 sol1 = s1.solve(1800.0, t_eval=list(tt1), rtol=1e-11, atol=at1)
160 U1 = (np.array(RCOLS) / cm.R0) ** 2
161 w("t/s" + ".join("%10s" % ("r=%.1fcm" % (x * 100)) for x in RCOLS))
162 tabT = np.zeros((7, 5))
163 tabC = np.zeros((7, 5))
164 for i in range(7):
165     Cn, Tn = s1.interp_u(sol1.y[:, i], U1)
166     tabT[i] = Tn
167     tabC[i] = Cn
168     w("%-8.0f" % tt1[i] + ".join("%10.4f" % v for v in Tn))
169
170 hdr("4. 问题1 —— 表2 (水分浓度 / (kg/kg))")
171 w("t/s" + ".join("%10s" % ("r=%.1fcm" % (x * 100)) for x in RCOLS))
172 for i in range(7):
173     w("%-8.0f" % tt1[i] + ".join("%10.4f" % v for v in tabC[i]))
174
175 # ----- 第 5 节: 水分场的网格收敛性 -----
176 # 与第 2 节同一套 N (24-128), 但看的是水分场: 扩散系数含 exp(-a/C), 含水率一降低该
177 # 指数项就变得很陡, 水分场比温度场难解, 故 N 从 24 起而不是从 8 起。同时报 t=1200 s
178 # 的表面值, 用来核对"不同时刻的收敛行为是否一致"。
179 hdr("5. 问题1 —— 谱解网格收敛性 (水分场)")
180 w("N" + "C(0,1800) C(R,1800) C(R,1200)")
181 for N in [24, 32, 48, 64, 96, 128]:
182     s = HerbSolver(N, prob=1)
183     at = np.concatenate([np.full(s.Np, 1e-14), np.full(s.Np, 1e-12)])
184     sol = s.solve(1800.0, t_eval=[1200.0, 1800.0], rtol=1e-12, atol=at)
185     a = s.interp_u(sol.y[:, 0], (0.02 / cm.R0) ** 2)[0][0]
186     b = s.interp_u(sol.y[:, 1], U1)[0]
187     w("N=%-5d %-9f %-9f %-9f" % (N, b[0], b[-1], a))
188
189 # ----- 第 6 节: 第三族离散族的互证 (有限体积 + 后向 Euler) -----
190 # 4 组 (N, dt) = (200, 1.0)、(200, 0.5)、(400, 0.5)、(400, 0.25): 同一空间网格配两档时间步,
191 # 后向 Euler 是一阶格式, 步长减半时误差也应减半, 这是判"离散误差已压到目标精度以下"
192 # 的依据; 三种离散族给出同一个数, 才说明它们解的是同一个 PDE。
193 hdr("6. 问题1 —— 第三种离散族互证 (均匀 r 网格守恒型有限体积 + 后向 Euler)")
194 w("N dt C(R,1800) C(R,1200) T(R,1800)")
195 for N, dt in [(200, 1.0), (200, 0.5), (400, 0.5), (400, 0.25)]:
196     xi, Ctab, Ttab = solve_fv(N, dt, 1800.0, prob=1, t_report=[1200.0, 1800.0])
197     c12 = float(np.atleast_1d(sample_xi(xi, Ctab[0], 0.02 / cm.R0))[0])
198     c18 = float(np.atleast_1d(sample_xi(xi, Ctab[1], 0.02 / cm.R0))[0])
199     t18 = float(np.atleast_1d(sample_xi(xi, Ttab[1], 0.02 / cm.R0))[0])
200     w("N=%-4d %-6.2f %-12.8f %-12.8f %-12.8f" % (N, dt, c18, c12, t18))
201 w("谱配置法(N=128): 1.51091810 1.65914239 37.198922")
202
203 # ----- 第 7 节: 问题 2 的表 3 与表 4 -----
204 # 报告 6 个时刻 (0.5-3 h, 每 0.5 h 一个点) × 5 个半径, 即论文表 3、表 4 的排版:
205 # 问题 2 的物性随 C、T 变化, atol 取水分 1e-12 / 温度 1e-10 (比第 3 节松一档)。
206 hdr("7. 问题2 —— 表3 (温度 / degC) 与表4 (水分浓度 / (kg/kg))")
207 tt2 = np.array([1800., 3600., 5400., 7200., 9000., 10800.])
208 s2 = HerbSolver(64, prob=2)
209 at2 = np.concatenate([np.full(s2.Np, 1e-12), np.full(s2.Np, 1e-10)])
210 sol2 = s2.solve(10800.0, t_eval=list(tt2), rtol=1e-11, atol=at2)
211 w("表3 温度")
212 w("t/h" + ".join("%10s" % ("r=%.1fcm" % (x * 100)) for x in RCOLS))
213 _p2T = np.zeros((6, 5))
214 _p2C = np.zeros((6, 5))
215 for i in range(6):
216     Cn, Tn = s2.interp_u(sol2.y[:, i], U1)
217     _p2T[i] = Tn
218     _p2C[i] = Cn
219     w("%-8.1f" % (tt2[i] / 3600) + ".join("%10.4f" % v for v in Tn))
220 w("")
221 w("表4 水分浓度")
222 w("t/h" + ".join("%10s" % ("r=%.1fcm" % (x * 100)) for x in RCOLS))
223 for i in range(6):
224     w("%-8.1f" % (tt2[i] / 3600) + ".join("%10.4f" % v for v in _p2C[i]))
225
226 hdr("8. 问题2 —— 两种离散族在 (0.5h, r=0) 的互证")
227 w("谱配置法 N=64/96/128 :")
228 for N in [64, 96, 128]:
229     s = HerbSolver(N, prob=2)
230     at = np.concatenate([np.full(s.Np, 1e-13), np.full(s.Np, 1e-12)])
231     sol = s.solve(1800.0, t_eval=[1800.0], rtol=1e-12, atol=at)
232     w("N=%-4d %-9f" % (N, s.interp_u(sol.y[:, -1], np.array([0.0]))[1][0]))
233 w("经典 FV+theta 格式 N=1600, 时间步一阶外推:")
234 # FV+theta 的时间方向只有一阶精度, 单看某个 dt 的值无法与谱解相比; 故取 4 档步长
235 # (2/1/0.5/0.25 s), 用最细两档估一阶误差斜率 c [s/s], 再外推到 dt -> 0 的极限
236 # (t(dt=0) = t(dt_min) - c*dt_min), 之后才与谱解对照。
237 lad = []
238 for dt in [2.0, 1.0, 0.5, 0.25]:
239     xi, tt, Cs, Ts = fv_theta_run(1600, 2, 1800.0, [(1e18, dt)],

```

```

240         report_idx=[1800.0])
241     lad.append((dt, Ts[0][0]))
242     w("    dt=%-6.3f %-9f" % (dt, Ts[0][0]))
243     c = (lad[-2][1] - lad[-1][1]) / (lad[-2][0] - lad[-1][0])
244     w("    一阶外推 dt->0 : %-9f" % (lad[-1][1] - c * lad[-1][0]))
245
246     # ----- 问题 3、4 共用的烘干判据与终止事件 -----
247     # CRIT 是题目给的烘干判据：干基含水率处处不高于 0.15 kg/kg。含水率沿半径由中心向外
248     # 递减，最大值在轴心 u=0（表面先干、中心最后达标），故事件函数取水分段的最大值减 CRIT：
249     # 它穿过 0 的时刻就是"处处达标"的首达时刻；配 direction=-1 表示只在下降方向触发。
250     CRIT = 0.15
251
252
253 def dry_time(N, prob, shrink=False, method="BDF", rtol=1e-11, ac=1e-13, at_=1e-11):
254     """求烘干时长 t_dry，返回（求解器实例，解对象，t_dry [s]）。
255
256     N        谱节点数（节点总数为 N+1）；
257     prob     问题号，3 或 4，决定用哪套物性经验式；
258     shrink   是否用附件 2 的实测半径 R(t)（问题 4 为 True）；
259     method   时间积分器名（BDF/Radau/LSODA），rtol 是它的相对容差；
260     ac, at_  分别是水分段与温度段的绝对容差（状态按 (C_0..C_N, T_0..T_N) 排列）。
261
262     积分上限 300000 s 只是"肯定够长"的松上界：事件是终止事件，达标即停，不会白算。
263     """
264     s = HerbSolver(N, prob=prob, shrink=shrink)
265     at = np.concatenate([np.full(s.Np, ac), np.full(s.Np, at_)])
266
267     def ev(t, y):
268         return np.max(y[:s.Np]) - CRIT
269     ev.terminal = True
270     ev.direction = -1
271     sol = s.solve(300000.0, rtol=rtol, atol=at, events=ev, method=method)
272     return s, sol, float(sol.t_events[0][0])
273
274
275 # ----- 第 9 节：问题 3 的烘干时间 -----
276 # 扫描 N=24-128 看 t_dry 是否随网格收敛；生产值取 N=64（与 result3.xlsx 的网格一致），
277 # 下面各节以及其它脚本报的 t_dry 都以它为准（N=128 的连续极限见 verify_data.py）。
278 hdr("9. 问题3 —— 烘干时间 t_dry")
279 w("谱配置法网格收敛性：")
280 for N in [24, 32, 48, 64, 96, 128]:
281     s, sol, td = dry_time(N, 3, rtol=1e-10)
282     w("N=%-5d tdry = %-11.3f s = %-8.4f h" % (N, td, td / 3600))
283     s3, solE3, td3 = dry_time(64, 3)
284     w("生产值 N=64: tdry = %-3f s = %-4f h = %-4f d" % (td3, td3 / 3600, td3 / 86400))
285     w("终点 C(0)=%.8f, C(表面)=%.8f, 最大值位于 u=%.4g"
286       % (solE3.y[0, -1], solE3.y[s3.N, -1],
287         s3.u[int(np.argmax(solE3.y[:s3.Np, -1]))]))
288
289 # fig22/fig23 的验证数据由 verify_data.py 单独复跑（约 22 min，不适合塞进一键复跑），
290 # 这里只做一致性校验：数据文件里的 N=64 项必须与上面刚算出的生产值相符。
291 # 校验不通过也不中断本节，而是把原因打印进报告（见下面的 except 分支），
292 # 让一次复跑就能看出"图的数据"与"这次算的解"到底哪边过期了。
293 w("")
294 w("fig22/fig23 验证数据一致性校验（中间数据/verification.json）：")
295 try:
296     import verify_data as VD
297     VD.check_consistency(VD.load(verbose=False), tdry_cached=td3, verbose=False)
298     w("    通过：数据文件与本次复跑一致（N=64 项 %-4f s）" % td3)
299 except Exception as exc:
300     # 数据缺失或不一致都要显式报出来
301     w("    ！！未通过：%s" % exc)
302
303 if not FAST:
304     w("")
305     w("时间积分器/容差敏感性（验证 tdry 不是数值噪声）：")
306     # 4 个配置跨 3 个积分器族（BDF 两档容差、Radau、LSODA）：若它们的 t_dry 只差
307     # 百分之一秒，说明这个时刻由方程本身决定，不是积分器或容差挑出来的。
308     for meth, tol in [("BDF", 1e-9), ("BDF", 1e-12), ("Radau", 1e-11), ("LSODA", 1e-10)]:
309         _, _, td = dry_time(64, 3, method=meth, rtol=tol)
310         w("    %-6s rtol=%-1.0e tdry = %-3f s" % (meth, tol, td))
311
312 w("")
313 w("表5 药材烘干过程的水分浓度（每 6 h）")
314 # 论文表 5 的行：6-54 h 每 6 h 一行，未行再补 t_dry 本身（刚好达标的那一行）。
315 hours = [6, 12, 18, 24, 30, 36, 42, 48, 54]
316 th = np.array([h * 3600.0 for h in hours] + [td3])
317 at3 = np.concatenate([np.full(s3.Np, 1e-13), np.full(s3.Np, 1e-11)])
318 sol3 = s3.solve(td3, t_eval=list(th), rtol=1e-11, atol=at3)
319 w("t/h      " + " ".join("%10s" % ("r=%.1fcm" % (x * 100)) for x in RCOLS))
320 for i, h in enumerate(hours):
321     Cn, _ = s3.interp_u(sol3.y[:, i], U1)

```

```

321     w("%-8d " % h + "".join("%10.4f" % v for v in Cn))
322     Cn, _ = s3.interp_u(sol3.y[:, -1], U1)
323     w("结束 " + "".join("%10.4f" % v for v in Cn))
324
325 # ----- 第 10 节: 问题 4 的烘干时间 (含实测收缩) -----
326 # 扫描 N=24~96 (比问题 3 少最细的 N=128 一档); 半径历史来自附件 2, 由 R_of 分段线性
327 # 插值取值, 故这里的 t_dry 同时含物性差异与几何收缩, 不能与问题 3 直接相减当收缩效应。
328 hdr("10. 问题4 —— 含收缩的烘干时间 t_dry")
329 w("R(0)=%.4f cm, R(tdry)=%.4f cm, R(259200)=%.4f cm"
330    % (cm.R_of(0) * 100, cm.R_of(td3) * 100, cm.R_of(259200) * 100))
331 w("请配置法网格收敛性: ")
332 for N in [24, 32, 48, 64, 96]:
333     s, sol, td = dry_time(N, 4, shrink=True, rtol=1e-10)
334     w("N=%-5d tdry = %11.3f s = %8.4f h = %4.4f d" % (N, td, td / 3600, td / 86400))
335     s4, solE4, td4 = dry_time(64, 4, shrink=True)
336     w("生产值 N=64: tdry = %3.1f s = %4.4f h = %4.4f d" % (td4, td4 / 3600, td4 / 86400))
337
338 w("")
339 w("表6 药材烘干过程的水分浓度 (列为当前物理距离, r_j>R(t) 留空)")
340 hours4 = [h for h in hours if h * 3600.0 < td4] # 问题4 的 tdry < 54 h
341 th4 = np.array([h * 3600.0 for h in hours4] + [td4])
342 at4 = np.concatenate([np.full(s4.Np, 1e-13), np.full(s4.Np, 1e-11)])
343 sol4 = s4.solve(td4, t_eval=list(th4), rtol=1e-11, atol=at4)
344 w("t/h      R/cm      r=0.0      r=0.5      r=1.0      r=1.5      表面")
345 for i, h in enumerate(hours4):
346     Rt = s4.R_of(th4[i])
347     row = "%-8d %.3f " % (h, Rt * 100)
348     for r in RCOLS[4:]:
349         if r <= Rt + 1e-12:
350             Cn, _ = s4.interp_u(sol4.y[:, i], (r / Rt) ** 2)
351             row += "%10.4f" % Cn[0]
352         else:
353             row += "%10s" % "—"
354     row += "%10.4f" % sol4.y[s4.N, i]
355     w(row)
356
357 # ----- 第 11 节: 对照格式的时间步敏感性 -----
358 # 步长计划 [(7200 s, min(1, Δt)), (1e18 s, Δt)]: 前 7200 s 恒实用不超过 1 s 的小步长,
359 # 之后才用 Δt, 这样扫到的是恒温干燥段的步长误差, 不与升温段混在一起。
360 # Δt 从 10 s 减到 0.5 s; 若 t_dry 随步长近似线性变化, 该格式就是 O(Δt) 一阶,
361 # 任何在固定粗步长序列上做的外推都不代表连续极限。
362 hdr("11. 烘干时长对时间离散的敏感性 (对照格式为何不可直接引用)")
363 w("经典 FV+theta 格式, N=1600, 长时段步长放宽到 Δt: ")
364 for d in [10.0, 5.0, 2.0, 1.0, 0.5]:
365     plan = [(7200.0, min(1.0, d)), (1e18, d)]
366     td = fv_theta_tdry(1600, 3, plan)
367     w(" Δt=%-5.2f tdry = %10.2f s (相对谱解 %+.2f s)" % (d, td, td - td3))
368 w("")
369 w("结论: 该格式的 tdry 随 Δt 近似线性变化 (每减半步长增量减半), 属一阶时间误差; ")
370 w("因此任何在固定粗步长序列上做网格外推得到的值都不是连续极限。")
371
372 # ----- 第 12 节: 关键答案汇总 -----
373 # 把前面各节算出的、论文要引用的数字集中重打一遍, 便于与正文表格逐个对位。
374 hdr("12. 关键答案汇总")
375 w("问题1 中心温度 T(0,1800) = %.6f -> %.4f" % (tabT[6][0], tabT[6][0]))
376 w("问题1 表面温度 T(R,1800) = %.6f -> %.4f" % (tabT[6][4], tabT[6][4]))
377 w("问题1 表面含水 C(R,1800) = %.6f -> %.4f" % (tabC[6][4], tabC[6][4]))
378 w("问题2 中心温度 T(0,3h) = %.6f -> %.4f" % (_p2T[5][0], _p2T[5][0]))
379 w("问题2 中心含水 C(0,3h) = %.6f -> %.4f" % (_p2C[5][0], _p2C[5][0]))
380 w("问题2 表面含水 C(R,3h) = %.6f -> %.4f" % (_p2C[5][4], _p2C[5][4]))
381 w("问题3 t_dry = %.1f s = %4.4f h" % (td3, td3 / 3600))
382 w("问题4 t_dry = %.1f s = %4.4f h" % (td4, td4 / 3600))
383 w("")
384 w("总耗时 %.1f s" % (time.time() - T0))
385
386 open("复跑报告.txt", "w", encoding="utf-8").write("\n".join(OUT))
387 print("witten 复跑报告.txt")

```

3.7 结果文件生成: make_results.py

```

1 # -*- coding: utf-8 -*-
2 # =====
3 # make_results.py —— 生成四个问题的结果文件 result1-4.xlsx
4 # =====
5 # 做什么: 把谱配置法的解按题目模板采样写盘: 第 1 行是到药材中心的距离, A 列是
6 # 时间, 表体是各时刻各半径的温度或干基含水率, 数值一律保留 4 位小数。

```

```

7 # 求解与写盘在同一趟里完成，不读已有的结果文件。
8 # 输入: spectral.HerbSolver 现场求解 (物性、烘房环境与 R(t) 取 common.py, 附件由它定位)。
9 # 生产网格 N=64, 与 run_all.py 报的 t_dry、论文各表同一口径。
10 # 输出: <包根>/result/result1.xlsx ~ result4.xlsx。result1、result2 各含"温度""水分浓度"
11 # 两张表, result3、result4 各一张表; result4 的列是当时的物理距离, r_j > R(t_i) 的
12 # 位置已不存在故留空, 未列"药材表面"给出该时刻的表面值 C(R(t), t)。
13 # 运行方式与依赖顺序:
14 # python make_results.py (可在任意目录运行: 输出路径按本文件位置解析)
15 # 它不依赖 run_all.py 或 verify_data.py, 但会重新求解四个问题 (需要数分钟),
16 # 且重跑直接覆盖 result/*.xlsx, 故没有必要时不要重跑。
17 # 关键变量:
18 # N 谱节点数 64 (论文的生产网格)
19 # RCOLS 结果表的 21 个距离列 0.0-2.0, 步长 0.1 [cm]
20 # RC 同一组距离换算成米, 用于插值 [m]
21 # CRIT 烘干判断阈值 0.15 (决定 result3/4 的时间终点) [kg/kg]
22 # =====
23
24 # ----- 误 import 的防护 -----
25 # 本脚本没有 main(), 生成逻辑直接写在模块顶层, import 即整段执行并覆盖 result/*.xlsx,
26 # 故拦一道: 误 import 立刻报错, 而不是悄悄覆盖掉已经交付的结果文件。
27 if __name__ != "__main__":
28     raise RuntimeError(
29         "make_results 的生成逻辑在模块顶层, import 会直接覆盖 result/*.xlsx。"
30         "请用 'python make_results.py' 运行; 确实需要 import 时, 请先把生成逻辑移入 main()。")
31
32 import os
33 from console_utf8 import fix_console
34 fix_console()
35 import numpy as np
36 import openpyxl
37 import common as cm
38 from spectral import HerbSolver
39
40 OUTDIR = os.path.abspath(os.path.join(os.path.dirname(os.path.abspath(__file__)),
41                                     "..", "result"))
42 os.makedirs(OUTDIR, exist_ok=True)
43
44 RCOLS = np.round(np.arange(0.0, 2.0001, 0.1), 10) # 0.0 .. 2.0 cm
45 RC = RCOLS * 1e-2 # m
46 CRIT = 0.15
47 N = 64
48
49
50 def write_sheet(ws, times, values, header_last=None):
51     """把一个 (时刻 × 半径) 的数值矩阵写成一张工作表。
52
53     第 1 行是表头: A1 为模板规定的交叉标题, B1 起依次是 21 个距离 [cm];
54     A 列是各报告时刻 [s]; 表体是 values[i, j], 四舍五入到 4 位小数。
55     values 为 None 或非有限值时该单元留空 (问题 4 里已收缩掉的半径即如此处理);
56     header_last 不为 None 时, 另用它在最后一列加一列表头与数值, 如 result4 的"药材表面"。
57     """
58     ws.cell(row=1, column=1, value="时间\\到药材中心的距离")
59     for j, rc in enumerate(RCOLS):
60         ws.cell(row=1, column=2 + j, value=float(rc))
61     if header_last is not None:
62         ws.cell(row=1, column=2 + len(RCOLS), value=header_last)
63     for i in range(len(times)):
64         ws.cell(row=2 + i, column=1, value=float(times[i]))
65         for j in range(len(RCOLS)):
66             v = values[i, j]
67             if v is not None and np.isfinite(v):
68                 ws.cell(row=2 + i, column=2 + j, value=round(float(v), 4))
69     if header_last is not None:
70         for i in range(len(times)):
71             v = values[i, len(RCOLS)]
72             if v is not None and np.isfinite(v):
73                 ws.cell(row=2 + i, column=2 + len(RCOLS), value=round(float(v), 4))
74
75
76 def sample(s, sol, u_targets, which):
77     """把解采样成矩阵, 返回形状 (报告时刻数, 采样点数)。
78
79     s 求解器实例 (用它的 interp_u 做插值);
80     sol solve_ivp 的解对象, sol.y 的各列依次对应各报告时刻;
81     u_targets 目标归一化半径 u = (r/R)^2 (无量纲, 故与半径用什么单位无关);
82     which "T" 取温度 [degC], 取其它值则取干基含水率 [kg/kg]。
83     """
84     out = np.empty((sol.y.shape[1], len(u_targets)))
85     for i in range(sol.y.shape[1]):
86         C, T = s.interp_u(sol.y[:, i], u_targets)
87         out[i] = T if which == "T" else C

```

```

88     return out
89
90
91 # ----- result1: 问题 1 (0-1800 s, 步长 1 s) -----
92 # 温度与水分浓度各写一张表。A 列从 t=1 s 起 (不含 t=0 的初始条件行), 共 1800 行;
93 # 插值坐标取  $u=(r/R)^2$  (请配置法的自变量), 故距离要先除以 R0 再平方。
94 print("result1 ...")
95 s = HerbSolver(N, prob=1)
96 tt = np.arange(1.0, 1801.0, 1.0)
97 at = np.concatenate([np.full(s.Np, 1e-13), np.full(s.Np, 1e-11)])
98 sol = s.solve(1800.0, t_eval=tt, rtol=1e-11, atol=at)
99 U = (RC / cm.R0) ** 2
100 wb = openpyxl.Workbook()
101 ws = wb.active; ws.title = "温度"
102 write_sheet(ws, tt, sample(s, sol, U, "T"))
103 ws2 = wb.create_sheet("水分浓度")
104 write_sheet(ws2, tt, sample(s, sol, U, "C"))
105 wb.save(os.path.join(OUTDIR, "result1.xlsx"))
106
107 # ----- result2: 问题 2 (0-10800 s, 步长 1 s) -----
108 # 结构与 result1 相同, 只是物性换成随 C、T 变化的附录 3 关系, 故需要解到 3 h。
109 print("result2 ...")
110 s2 = HerbSolver(N, prob=2)
111 tt2 = np.arange(1.0, 10801.0, 1.0)
112 at2 = np.concatenate([np.full(s2.Np, 1e-12), np.full(s2.Np, 1e-10)])
113 sol2 = s2.solve(10800.0, t_eval=tt2, rtol=1e-11, atol=at2)
114 wb = openpyxl.Workbook()
115 ws = wb.active; ws.title = "温度"
116 write_sheet(ws, tt2, sample(s2, sol2, U, "T"))
117 ws2 = wb.create_sheet("水分浓度")
118 write_sheet(ws2, tt2, sample(s2, sol2, U, "C"))
119 wb.save(os.path.join(OUTDIR, "result2.xlsx"))
120
121
122 def dry(s, shrink=False, rtol=1e-11):
123     """求烘干时长 t_dry [s], 事件口径与 run_all.dry_time 相同。
124
125     事件函数取状态中水分那段的最大值减 CRIT: 含水率沿半径由中心向外递减, 最大值在轴心
126     u=0 (表面先干、中心最后达标), 故它穿过 0 的时刻就是"处处不高于 0.15 kg/kg"的首达
127     时刻; terminal 配 direction=-1 表示只在下降方向触发。rtol 是积分的相对容差。
128     shrink 在本函数体内并不使用: 是否含收缩由构造求解器 s 时决定, 保留它只是让调用处
129     一眼看出这一路是问题 4。
130     """
131     at = np.concatenate([np.full(s.Np, 1e-13), np.full(s.Np, 1e-11)])
132
133     def ev(t, y):
134         return np.max(y[:s.Np]) - CRIT
135     ev.terminal = True
136     ev.direction = -1
137     e = s.solve(300000.0, rtol=rtol, atol=at, events=ev)
138     return float(e.t_events[0][0])
139
140
141 # ----- result3: 问题 3 (到 t_dry, 步长 60 s) -----
142 # 先由事件法解出终止时刻 t_dry, 再把 60 s 整数倍的时刻连同 t_dry 本身作为输出行——
143 # 末行不是 60 s 的整数倍, 它正是烘干刚好达标的那一刻。
144 print("result3 ...")
145 s3 = HerbSolver(N, prob=3)
146 td3 = dry(s3)
147 tt3 = np.arange(60.0, np.floor(td3 / 60.0) * 60.0 + 1.0, 60.0)
148 tt3 = np.append(tt3, td3)
149 at3 = np.concatenate([np.full(s3.Np, 1e-13), np.full(s3.Np, 1e-11)])
150 sol3 = s3.solve(td3, t_eval=list(tt3), rtol=1e-11, atol=at3)
151 U3 = (RC / cm.R0) ** 2
152 wb = openpyxl.Workbook(); ws = wb.active; ws.title = "Sheet1"
153 write_sheet(ws, tt3, sample(s3, sol3, U3, "C"))
154 wb.save(os.path.join(OUTDIR, "result3.xlsx"))
155
156 # ----- result4: 问题 4 (含收缩, 到 t_dry, 步长 60 s) -----
157 # 列的含义与 result3 相同, 但距离列是"当时的物理距离": 每行按该时刻的 R(t) 判断哪些
158 # 距离仍然存在 (r <= R(t) 才有值), 不存在的留空, 末列另给当前表面值。
159 print("result4 ...")
160 s4 = HerbSolver(N, prob=4, shrink=True)
161 td4 = dry(s4, shrink=True)
162 tt4 = np.arange(60.0, np.floor(td4 / 60.0) * 60.0 + 1.0, 60.0)
163 tt4 = np.append(tt4, td4)
164 at4 = np.concatenate([np.full(s4.Np, 1e-13), np.full(s4.Np, 1e-11)])
165 sol4 = s4.solve(td4, t_eval=list(tt4), rtol=1e-11, atol=at4)
166 vals = np.full((len(tt4), len(RCOLS) + 1), np.nan)
167 for i, t in enumerate(tt4):
168     Rt = s4.R_of(t)

```

```

169     for j, r in enumerate(RC):
170         if r <= Rt + 1e-12:          # 该物理位置仍然存在
171             Cn, _ = s4.interp_u(sol4.y[:, i], (r / Rt) ** 2)
172             vals[i, j] = Cn[0]
173             vals[i, -1] = sol4.y[s4.N, i]      # 当前表面值
174 wb = openpyxl.Workbook(); ws = wb.active; ws.title = "Sheet1"
175 write_sheet(ws, tt4, vals, header_last="药材表面")
176 wb.save(os.path.join(OUTDIR, "result4.xlsx"))
177
178 print("result1: %d 行 x %d 列" % (len(tt), len(RCOLS) + 1))
179 print("result2: %d 行 x %d 列" % (len(tt2), len(RCOLS) + 1))
180 print("result3: %d 行, tdry=%.3f s" % (len(tt3), td3))
181 print("result4: %d 行, tdry=%.3f s" % (len(tt4), td4))
182 print("已写入:", OUTDIR)

```

3.8 灵敏度分析: sensitivity.py

```

1  # -*- coding: utf-8 -*-
2  # =====
3  # sensitivity.py —— 关键参数扰动的灵敏度分析（真实重算，不是口头推断）
4  # =====
5  # 做什么：逐个扰动物性经验式与传递系数，把问题 3 的整个烘干过程重新解一遍，看烘干时长
6  #          t_dry 与两个时刻的中心含水率随之变化多少，据此排定“哪个参数最要紧”。
7  # 输入：common.py（物性、一阶惯性环境、初值）、spectral.HerbSolver（主求解器）、
8  #        fig_core.DATADIR（输出目录）；基准解与 result3.xlsx 用同一网格 BASE_N。
9  # 输出：<交付根>/中间数据/sensitivity.txt（可读表格）与 sensitivity.npz（供绘图与核对）。
10 # 运行方式与依赖顺序：python sensitivity.py, 可单独运行，与 run_all.py 互不依赖；
11 #     每个扰动都要从头重解一次（共 14 次），耗时主要在求解上。
12 # 关键变量：
13 #   BASE_N   基准谱节点数 64（与 result3.xlsx 的生产网格一致）
14 #   CRIT     烘干判据阈值 0.15 [kg/kg]
15 #   C_FLOOR  物性式求值用的含水率下限 1e-3（防 exp(-a/C) 发散） [kg/kg]
16 #   扰动幅度 物性与传递系数取 ±2%：题目未给经验式的标定区间，2% 足以看清各参数的相对
17 #             重要性，而网格与容差带来的数值误差只有 1e-5 相对量级；Tset 取 ±0.2 K。
18 # 扰动对象与考察指标：
19 #   物性侧   活化温度 E=3850 K、活化浓度指数 a、扩散系数前因子 D0；
20 #   传递侧   对流换热系数 h、对流传质系数 hm；
21 #   环境侧   烘房设定温度 T_set，以及环境口径（一阶惯性拟合式 vs 附件 1 原始序列插值）。
22 #   指标     问题 3 的烘干时长 t_dry，以及 t=24 h、48 h 的中心含水率；所有相对变化都以
23 #             本网格（BASE_N）的基准 t_dry 为分母（连续极限 205575.5 s 见论文第 11.4 节）。
24 # =====
25 import os
26 from console_utf8 import fix_console
27 fix_console()
28 import numpy as np
29 import common as cm
30 from spectral import HerbSolver
31 import fig_core as FC
32
33 CRIT = 0.15
34 C_FLOOR = 1e-3
35 BASE_N = 64      # 灵敏度基准网格，与 result3.xlsx 的生产网格一致
36 OUT = os.path.join(FC.DATADIR, "sensitivity.txt")
37 NPZ = os.path.join(FC.DATADIR, "sensitivity.npz")
38 LOG = []
39
40
41 def log(s):
42     """写一行灵敏度报告：同时进内存缓冲 LOG 与屏幕，末尾一起落盘成 txt。"""
43     print(s, flush=True)
44     LOG.append(str(s))
45
46
47 def props_pert(prob, C, T, pert):
48     """按扰动因子重算物性，返回 (rho, cp, k, D)，与 common.props 的结构一致。
49
50     prob 决定用哪套经验式（1、2/3、4 三支）；C 为干基含水率 [kg/kg]，T 为温度 [degC]；
51     pert 是扰动字典，键 E / a / D0 给出对应因子的乘数（缺省 1.0，即不扰动）；
52     h 与 hm 只出现在边界条件里，不参与物性式，故这里不处理。
53     量纲同 common.props: rho [kg/m3]、cp [J/(kg K)]、k [W/(m K)]、D [m2/s]。
54     """
55     C = np.asarray(C, dtype=float)
56     T = np.asarray(T, dtype=float)
57     Cf = np.maximum(C, C_FLOOR)
58     TK = T + 273.15
59     E = 3850.0 * pert.get("E", 1.0)

```

```

60 fd = pert.get("D0", 1.0)
61 fa = pert.get("a", 1.0)
62 if prob == 1:
63     rho = np.full_like(C, 820.0)
64     cp = np.full_like(C, 2600.0)
65     k = np.full_like(C, 0.36)
66     D = 7e-9 * fd * np.exp(-0.89 * fa / Cf)
67 elif prob in (2, 3):
68     rho = 650.0 + 128.0 * C
69     cp = 1450.0 + 2736.0 * C / (C + 1.0)
70     k = 0.21 + 0.38 * C / (C + 1.0)
71     D = 2.4e-3 * fd * np.exp(-0.45 * fa / Cf) * np.exp(-E / TK)
72 else:
73     rho = 760.0 + 90.0 * C
74     cp = 1850.0 + 2150.0 * C / (C + 1.0)
75     k = 0.12 + 0.20 * C / (C + 1.0)
76     D = 4.2e-4 * fd * np.exp(-0.30 * fa / Cf) * np.exp(-E / TK)
77 # 返回 (rho, cp, k, D): h / hm 不参与物性式 (它们只在边界条件里出现),
78 # 由调用方按 perturb 直接改 cm.h / cm.hm, 故不在此返回。
79 return rho, cp, k, D
80
81
82 def run_case(tag, pert=None, ambient="fit", N=BASE_N, prob=3, probe1=False):
83     """在给定扰动下重解一遍, 返回 (t_dry [s], CO(24h), CO(48h), probe)。
84
85     扰动只在本函数内部生效: 临时替换 cm.props / cm.h / cm.hm / cm.Tset / cm.T_air /
86     cm.C_air, 算完在 finally 里全部还原, 故各组扰动之间互不影响。扰动的口径如下:
87     * E / a / D0 —— 只改物性式中的对应因子, 物性关系其余部分不变;
88     * h / hm —— 只改这两个传递系数;
89     * Tset —— 只把烘房设定值平移 (温差, ±0.2 K)。一阶惯性的初值锚定在实测的
90         T_air(0)=28 degC 上, 故初值不受影响, 被扰动的是设定值与整条升温
91         轨迹;
92     * ambient —— "fit" 用拟合曲线; "raw" 把温度与含湿度同时换回附件 1 实测序列的
93         线性插值 (4 h 之后仍取各自设定值)。
94     probe 是三元组 (T(0,1800), T(R,1800), C(R,1800)), 仅 probe1=True 时给出, 供论文
95     "环境口径对结果的影响"一表直接引用; 两个中心含水率取不到时返回 nan。
96     tag 是调用处给这一组扰动起的名字 (由调用方打印进报告), 函数体内并不使用它。
97     """
98     import spectral
99     pert = pert or {}
100     base_props = cm.props
101     base_T, base_C = cm.T_air, cm.C_air
102     base_h, base_hm = cm.h, cm.hm
103     base_Tset = cm.Tset
104
105     def _props(p, C, T):
106         rho, cp, k, D = props_pert(p, C, T, pert)
107         return rho, cp, k, D
108
109     cm.props = _props
110     cm.h = base_h * pert.get("h", 1.0)
111     cm.hm = base_hm * pert.get("hm", 1.0)
112     if "Tset" in pert:
113         # 直接平移设定值: cm.T_air 的表达式 Tset-(Tset-T_air0)exp(-t/tauT)
114         # 自动把 t=0 锚在 T_air0 上, 因此不会像 base_T(t)+dT 那样连初值一起改。
115         cm.Tset = base_Tset + pert["Tset"]
116     if ambient == "raw":
117         cm.T_air = lambda t: np.where(np.asarray(t, dtype=float) <= cm.T_END,
118                                     np.interp(np.asarray(t, dtype=float),
119                                                  cm._t_env, cm._Ta_env), cm.Tset)
120         cm.C_air = lambda t: np.where(np.asarray(t, dtype=float) <= cm.T_END,
121                                     np.interp(np.asarray(t, dtype=float),
122                                                  cm._t_env, cm._Ca_env), cm.Cset)
123
124     try:
125         # 状态按 (C_0..C_N, T_0..T_N) 排列, atol 分两段给: 水分 1e-13、温度 1e-11。
126         s = spectral.HerbSolver(N, prob=prob)
127         at = np.concatenate([np.full(s.Np, 1e-13), np.full(s.Np, 1e-11)])
128
129         def ev(t, y):
130             return np.max(y[:s.Np]) - CRIT
131         ev.terminal = True
132         ev.direction = -1
133         e = s.solve(320000.0, rtol=1e-10, atol=at, events=ev)
134         td = float(e.t_events[0][0])
135         # 中心含水率取 24 h 与 48 h 两个固定时刻: 事件求解不保留这两个时刻的值,
136         # 故另起一次带 t_eval 的求解: 只取落在 [0, min(t_dry, 48 h)] 内的时刻,
137         # 取不到的就保持 nan。
138         targets = [24 * 3600.0, 48 * 3600.0]
139         tmax = min(td, 48 * 3600.0)
140         tv = [x for x in targets if x <= tmax + 1e-9]

```

```

141     c24 = c48 = np.nan
142     if tv:
143         so = s.solve(tmax, t_eval=tv, rtol=1e-10, atol=at)
144         for k, x in enumerate(tv):
145             v = float(s.interp_u(so.y[:, k], np.array([0.0]))[0][0])
146             if abs(x - targets[0]) < 1.0:
147                 c24 = v
148             if abs(x - targets[1]) < 1.0:
149                 c48 = v
150
151     # 问题 1 的三个报数: 供论文"环境口径对结果的影响"表直接引用,
152     # 避免表中数字与脚本输出脱钩。
153     probe = None
154     if probe1:
155         s1 = spectral.HerbSolver(64, prob=1)
156         at1 = np.concatenate([np.full(s1.Np, 1e-13), np.full(s1.Np, 1e-11)])
157         so1 = s1.solve(1800.0, t_eval=[1800.0], rtol=1e-11, atol=at1)
158         c_c, t_c = s1.interp_u(so1.y[:, -1], np.array([0.0]))
159         c_s, t_s = s1.interp_u(so1.y[:, -1], np.array([1.0]))
160         probe = (float(t_c[0]), float(t_s[0]), float(c_s[0]))
161     finally:
162         cm.props = base_props
163         cm.T_air, cm.C_air = base_T, base_C
164         cm.h, cm.hm = base_h, base_hm
165         cm.Tset = base_Tset
166     return td, c24, c48, probe
167
168 def main():
169     log("==" * 74)
170     log("灵敏度分析 (问题 3, 烘干时长 t_dry: 基准 t_dry 由 BDF 求解给出)")
171     log("基准网格 N=%d, 基准口径 (问题 3 物性 + 一阶惯性拟合环境)" % BASE_N)
172     log("注意: 基准值随网格而变 (N=48 得 205572.3 s, N=64 得 205574.5 s, ")
173     log("连续极限 205575.5 s)。本表所有相对变化均以本基准为分母。")
174     log("==" * 74)
175
176     base = run_case("base", probe1=True)
177     log("基准 (无扰动): t_dry = %.1f s = %.4f h ; CO(24h)=%.4f ; CO(48h)=%.4f"
178         % (base[0], base[0] / 3600, base[1], base[2]))
179     t0 = base[0]
180
181     # 12 组扰动 = 6 个参数各两档: 物性与传递系数取 ±2%, 烘房设定温度取 ±0.2 K。
182     # 行标签同时写清了扰动前后的数值, 论文表 8 与这里的口径必须一致。
183     cases = [
184         ("E +2% (3850 -> 3927 K)", {"E": 1.02}),
185         ("E -2% (3850 -> 3773 K)", {"E": 0.98}),
186         ("a +2% (0.45 -> 0.459)", {"a": 1.02}),
187         ("a -2% (0.45 -> 0.441)", {"a": 0.98}),
188         ("DO +2%", {"DO": 1.02}),
189         ("DO -2%", {"DO": 0.98}),
190         ("h +2% (25 -> 25.5 W/m2K)", {"h": 1.02}),
191         ("h -2%", {"h": 0.98}),
192         ("hm +2% (8e-7 -> 8.16e-7)", {"hm": 1.02}),
193         ("hm -2%", {"hm": 0.98}),
194         ("Tset +0.2 K (50.212 -> 50.412)", {"Tset": 0.2}),
195         ("Tset -0.2 K", {"Tset": -0.2}),
196     ]
197
198     rows = []
199     log("")
200     log("%-34s %12s %10s %10s" % ("扰动", "t_dry / s", "相对变化", "绝对变化/s"))
201     log("==" * 74)
202     for name, pert in cases:
203         td, c24, c48, _ = run_case(name, pert)
204         rel = (td - t0) / t0 * 100
205         rows.append((name, td, rel, td - t0, c24, c48))
206         log("%-34s %12.1f %9.3f% %10.1f" % (name, td, rel, td - t0))
207         print(" CO(24h)=%.4f CO(48h)=%.4f" % (c24, c48))
208
209     # ---- 环境口径分叉: 温度与含湿量同时换回附件 1 实测序列插值 ----
210     # 这是建模选择而不是参数扰动: 附件 1 的拟合残差存在系统性失配 (论文图 3),
211     # 故要专门看一眼"换用原始序列插值"会不会改变结论。
212     log("")
213     log("环境口径分叉 (附件 1 原始序列插值 vs 一阶惯性拟合): ")
214     log("两种口径的 T_air 与 C_air 都同时切换; 4 h 之后的平台段取自设定值。")
215     td_raw, c24r, c48r, probe_raw = run_case("raw", ambient="raw", probe1=True)
216     log("拟合口径 t_dry = %.1f s ; T(0,1800)=%.4f ; T(R,1800)=%.4f ; C(R,1800)=%.4f"
217         % (t0, base[3][0], base[3][1], base[3][2]))
218     log("原始插值 t_dry = %.1f s ; T(0,1800)=%.4f ; T(R,1800)=%.4f ; C(R,1800)=%.4f"
219         % (td_raw, probe_raw[0], probe_raw[1], probe_raw[2]))
220     log("差: t_dry %+.1f s (%+.4f%); T(0) %+.4f K ; T(R) %+.4f K ; C(R) %+.4f"
221         % (td_raw - t0, (td_raw - t0) / t0 * 100,

```

```

222     probe_raw[0] - base[3][0], probe_raw[1] - base[3][1],
223     probe_raw[2] - base[3][2]))
224     log(" C0(24h): %.4f -> %.4f ; C0(48h): %.4f -> %.4f" % (base[1], c24r, base[2], c48r))
225
226     # ---- 排序: 按 |相对变化| 从大到小排, 主导参数一眼可见 ----
227     log("")
228     log("按 |相对变化| 排序 (主导不确定性的参数): ")
229     for nm, td, rel, ab, _, _ in sorted(rows, key=lambda r: -abs(r[2])):
230         log(" % -34s % +8.3f%%" % (nm, rel))
231
232     # ---- 解析局部灵敏度对照: 由  $\ln D = \ln D_0 - a/C - E/T_K$  直接求偏导, 与上面的全模型
233     # 重算比量级。注意它是某个温度、某个含水率下的局部导数, 不能代替全程重算的  $t_{dry}$ 
234     # 变化——两者量级相符只说明扰动响应方向合理, 最终结论仍以全模型重算为准。
235     log("")
236     log("解析局部灵敏度对照 ( $D = D_0 \exp(-a/C) \exp(-E/T)$ ): ")
237     for T_C in [28.0, 40.0, 50.0]:
238         TK = T_C + 273.15
239         log(" T= %.0f degC:  $d\ln D/dT = E/T^2 = %.5f / K$  (即 1 K 偏差 -> D 变 %.2f%%)"
240             % (T_C, 3850.0 / TK ** 2, 3850.0 / TK ** 2 * 100))
241     for Cv in [2.55, 1.0, 0.15]:
242         log(" C= %.2f kg/kg:  $d\ln D/da = -1/C = %.4f$  (a 相对变 2%% ->  $\ln D$  变 %.3f%%)"
243             % (Cv, -1.0 / Cv, -1.0 / Cv * 0.45 * 0.02 * 100))
244
245     np.savez(os.path.join(FC.DATADIR, "sensitivity.npz"),
246             rows=np.array([r[1], r[2], r[3]] for r in rows),
247             names=np.array([r[0] for r in rows]),
248             base=np.array([t0, base[1], base[2]]),
249             # 基准网格一并存档, 便于事后核对表中数字的口径
250             base_N=np.array([BASE_N]),
251             probe_fit=np.array(base[3]),
252             raw=np.array([td_raw, c24r, c48r]),
253             probe_raw=np.array(probe_raw))
254     open(OUT, "w", encoding="utf-8").write("\n".join(LOG))
255     log("")
256     log("已写出: " + OUT)
257
258
259 if __name__ == "__main__":
260     main()

```

3.9 绘图基础模块: fig_core.py

```

1  # -*- coding: utf-8 -*-
2  # =====
3  # fig_core.py —— 论文插图的统一绘图设置、出图入口与四问解缓存
4  # =====
5  # 做什么: 集中设定 matplotlib 的中文字体、字号、网格与出图分辨率, 并对外只留一个
6  # 出图入口 save(), 使全部插图的大小字号风格一致; 同时缓存四问的解, 让
7  # make_figures.py 等脚本不必每次重新求解。
8  # 输入: common.py、spectral.py 与本文件的源码 (用于算缓存指纹);
9  # 中间数据/solutions.npz (存在且版本/指纹相符时直接复用, 否则重算)。
10 # 输出: figure/*.png 由 save() 逐张写出, 240 dpi
11 # 中间数据/solutions.npz 由 build_cache() 写出四问解 + 版本号 + 源码指纹
12 # 关键变量:
13 # FIGDIR / DATADIR 出图目录 figure/ 与中间数据/ (相对交付根, 不存在则自动创建)
14 # DPI 出图分辨率 240 dpi: 插图排版时会缩到 6.30 in 版心内, 缩放系数约 0.76,
15 # 按 200 dpi 渲染印后只剩约 260 ppi, 240 dpi 才够 310 ppi 以上
16 # CACHE_VERSION 缓存版本号字符串; 改动物理模型、参数、离散或容差必须同步递增
17 # N 四问解统一使用的谱节点数 64 ( $N_p = 65$  个 CGL 节点, 两场共 130 个状态量)
18 # CRIT 烘干判据阈值 0.15 kg/kg (干基含水率), 仅用于问题 3、4 的事件求交
19 # =====
20 import os
21 import numpy as np
22 import matplotlib
23 matplotlib.use("Agg")
24 import matplotlib.pyplot as plt
25 from matplotlib import font_manager as fm
26
27 HERE = os.path.dirname(os.path.abspath(__file__))
28 ROOT = os.path.abspath(os.path.join(HERE, ".."))
29 FIGDIR = os.path.join(ROOT, "figure")
30 DATADIR = os.path.join(ROOT, "中间数据")
31 os.makedirs(FIGDIR, exist_ok=True)
32 os.makedirs(DATADIR, exist_ok=True)
33
34 # ----- 中文字体与字号 -----

```

```

35 # 不把字体文件注册进 fontManager, 中文标签就渲染成一片方框: matplotlib 自带的
36 # 字体表里没有 CJK 字体。axes.unicode_minus=False 同理——CJK 字体缺 U+2212 字形,
37 # 负号不退回 ASCII 连字符也会显示成方框。
38 for _fp in [r"C:\Windows\Fonts\msyh.ttc", r"C:\Windows\Fonts\simhei.ttf"]:
39     if os.path.exists(_fp):
40         fm.fontManager.addfont(_fp)
41 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
42 plt.rcParams["axes.unicode_minus"] = False
43 plt.rcParams["font.size"] = 10.5
44 plt.rcParams["axes.titlesize"] = 11
45 plt.rcParams["axes.labelsize"] = 10.5
46 plt.rcParams["legend.fontsize"] = 9
47 plt.rcParams["xtick.labelsize"] = 9.5
48 plt.rcParams["ytick.labelsize"] = 9.5
49 plt.rcParams["figure.dpi"] = 200
50 plt.rcParams["savefig.dpi"] = 240
51 plt.rcParams["savefig.bbox"] = "tight"
52 plt.rcParams["axes.grid"] = True
53 plt.rcParams["grid.alpha"] = 0.3
54 plt.rcParams["grid.linewidth"] = 0.6
55
56 # 出图分辨率: 插图按 \textwidth 排版后会被缩到 6.30 in 版心内, 缩放系数约 0.76,
57 # 按 200 dpi 渲染到纸面上的有效像素密度只剩约 260 ppi, 240 dpi 才有 310 ppi 以上。
58 # 注意 dpi 只决定像素密度, 不改变图内字号的磅值(磅值由 figsize 与版心宽度之比
59 # 决定), 故提高 dpi 不会动到任何一张图的版面与版式。
60 DPI = 240
61
62
63 def save(fig, name):
64     """把画布 fig 存成 figure/<name>, 关闭画布并打印一行日志; 返回写出的绝对路径。
65
66     所有插图都收敛到这个入口, 是为了把分辨率和紧裁剪钉死在一处——逐图手写
67     savefig 很容易漏掉 dpi。注意本函数会 close(fig), 调用方不得再复用该画布。
68     """
69     p = os.path.join(FIGDIR, name)
70     fig.savefig(p, dpi=DPI)
71     plt.close(fig)
72     print(" [fig] %s" % name)
73     return p
74
75
76 # ----- 四问解缓存 -----
77 # 缓存版本号: 任何改变解的内容的改动(物理模型、参数、离散、容差)都必须同时
78 # 递增此号——否则旧的 solutions.npz 会被静默沿用, 图上画的是旧解。
79 CACHE_VERSION = "2026-09-13.N64.rtol1e-11"
80
81 import hashlib
82
83
84 def _code_fingerprint():
85     """对求解相关源文件(common.py、spectral.py、fig_core.py)整体取 SHA-256,
86     返回前 16 位十六进制字符串。
87
88     把它随解一起落盘, 下次启动时对比即可发现"改了代码但缓存没重算"; 只靠版本号
89     字符串要靠人记得手工递增, 漏改一次就会画出旧解。
90     """
91     h = hashlib.sha256()
92     for name in ("common.py", "spectral.py", "fig_core.py"):
93         p = os.path.join(HERE, name)
94         if os.path.exists(p):
95             with open(p, "rb") as fh:
96                 h.update(fh.read())
97     return h.hexdigest()[:16]
98
99
100 CACHE_FINGERPRINT = _code_fingerprint()
101
102
103 def build_cache(force=False):
104     """求解问题 1-4 并写入 中间数据/solutions.npz, 供出图脚本复用, 避免重复求解。
105
106     缓存中的键(时间一律 [s]: y 的前 Np 行是水分浓度 C [kg/kg]、后 Np 行是温度
107     T [degC], 按 CGL 节点 u = 0..1 排列):
108     p1_t, p1_y          问题 1: 0-1800 s, 报告点间隔 10 s
109     p2_t, p2_y          问题 2: 0-10800 s, 报告点间隔 60 s
110     p3_t, p3_y, p3_tdry 问题 3: 0-t_dry, 均匀 1400 点; t_dry [s]
111     p4_t, p4_y, p4_tdry 问题 4: 含收缩, 同上
112     version, fingerprint 版本号与源码指纹, 用于判定缓存是否对应当前代码
113
114     force=True (或环境变量 A_FORCE_CACHE=1) 时无条件重算; 版本号或源码指纹与当前
115     代码不符时同样重算, 故缓存不会悄悄沿用旧解。

```

```

116 """
117 import common as cm
118 from spectral import HerbSolver
119
120 cache = os.path.join(DATADIR, "solutions.npz")
121 if os.environ.get("A_FORCE_CACHE") == "1":
122     force = True
123 if os.path.exists(cache) and not force:
124     try:
125         z = np.load(cache)
126         ver = str(z["version"]) if "version" in z else "<无版本号>"
127         fp = str(z["fingerprint"]) if "fingerprint" in z else "<无指纹>"
128     except Exception as exc:
129         # 缓存损坏: 重算而不是崩溃
130         print("[cache] 缓存无法读取 (%s), 改为重算" % exc)
131     else:
132         if ver == CACHE_VERSION and fp == CACHE_FINGERPRINT:
133             print("[cache] 命中: %s (版本 %s)" % (cache, ver))
134             return
135         print("[cache] 已存在的缓存与当前代码不符, 重算: ")
136         print("    缓存版本 %s / 当前版本 %s" % (ver, CACHE_VERSION))
137         print("    缓存指纹 %s / 当前指纹 %s" % (fp, CACHE_FINGERPRINT))
138         print("    若确认要沿用旧解, 只能先把 solutions.npz 备份回来, "
139             "再跳过 build_cache() 直接调用 load_cache(); ")
140         print("    A_FORCE_CACHE=0 并不代表沿用旧解 (版本/指纹不符时仍会重算)。")
141 print("[cache] 开始计算四问解 ...")
142 N = 64
143 # 谱节点数: Np = 65 个 CGL 节点, 两场共 130 个状态量
144 d = {}
145
146 # ---- 问题1: 0-1800 s, 报告点间隔 10 s ----
147 s1 = HerbSolver(N, prob=1)
148 t1 = np.arange(0.0, 1800.1, 10.0)
149 # atol 按状态块分开始: 前 Np 个是水分浓度 C [kg/kg] (量级 1), 后 Np 个是温度
150 # T [degC] (量级 3e2), 两块量级差两个数量级, 同一个绝对容差对两者并不等价。
151 at1 = np.concatenate([np.full(s1.Np, 1e-13), np.full(s1.Np, 1e-11)])
152 sol1 = s1.solve(1800.0, t_eval=t1, rtol=1e-11, atol=at1)
153 d["p1_t"], d["p1_y"] = sol1.t, sol1.y
154
155 # ---- 问题2: 0-10800 s, 报告点间隔 60 s ----
156 s2 = HerbSolver(N, prob=2)
157 t2 = np.arange(0.0, 10800.1, 60.0)
158 at2 = np.concatenate([np.full(s2.Np, 1e-12), np.full(s2.Np, 1e-10)])
159 sol2 = s2.solve(10800.0, t_eval=t2, rtol=1e-11, atol=at2)
160 d["p2_t"], d["p2_y"] = sol2.t, sol2.y
161
162 # ---- 问题3: 先只求解到烘干时刻, 再按均匀时刻表回放, 供画光滑曲线 ----
163 CRIT = 0.15
164 # 烘干判据: 中心含水率降到 0.15 kg/kg (干基)
165 s3 = HerbSolver(N, prob=3)
166 at3 = np.concatenate([np.full(s3.Np, 1e-13), np.full(s3.Np, 1e-11)])
167
168 def ev(t, y):
169     # y[:Np] 是水分浓度场, 其最大值出现在干燥最慢的中心点。terminal 让积分在事件处
170     # 停下, 不必白算到 t=300000 s; direction=-1 只认由大变小穿越 (初值 2.55 kg/kg
171     # 远高于阈值, 不会误触发)。
172     return np.max(y[:s3.Np]) - CRIT
173
174 ev.terminal = True
175 ev.direction = -1
176 e3 = s3.solve(300000.0, rtol=1e-11, atol=at3, events=ev)
177 td3 = float(e3.t_events[0][0])
178 t3 = np.linspace(0.0, td3, 1400)
179 # 1400 个点足够把 60 h 量级的曲线画光滑
180 sol3 = s3.solve(td3, t_eval=list(t3), rtol=1e-11, atol=at3)
181 d["p3_t"], d["p3_y"], d["p3_tdry"] = sol3.t, sol3.y, np.array([td3])
182
183 # ---- 问题4: 物性换成附录 4, 并开启附件 2 的实测收缩 R(t) ----
184 s4 = HerbSolver(N, prob=4, shrink=True)
185 at4 = np.concatenate([np.full(s4.Np, 1e-13), np.full(s4.Np, 1e-11)])
186
187 def ev4(t, y):
188     # 事件定义同问题 3 的 ev; 问题 4 的判据仍取当前半径下的中心含水率
189     return np.max(y[:s4.Np]) - CRIT
190
191 ev4.terminal = True
192 ev4.direction = -1
193 e4 = s4.solve(300000.0, rtol=1e-11, atol=at4, events=ev4)
194 td4 = float(e4.t_events[0][0])
195 t4 = np.linspace(0.0, td4, 1400)
196 sol4 = s4.solve(td4, t_eval=list(t4), rtol=1e-11, atol=at4)
197 d["p4_t"], d["p4_y"], d["p4_tdry"] = sol4.t, sol4.y, np.array([td4])
198
199 # 版本与指纹随解一起落盘, 供下次启动时判定缓存是否仍然对应当前代码
200 d["version"] = np.array(CACHE_VERSION)
201 d["fingerprint"] = np.array(CACHE_FINGERPRINT)
202 np.savez_compressed(cache, **d)

```

```

197 print("[cache] 完成: ", cache)
198 print("[cache] 版本 %s, 指纹 %s" % (CACHE_VERSION, CACHE_FINGERPRINT))
199
200
201 def load_cache():
202     """读出 中间数据/solutions.npz, 返回 (数据字典 z, HerbSolver 类, common 模块)。
203
204     连带返回后两者, 是为了让出图脚本一次拿到画图所需的全部入口; 本函数只做读取,
205     版本号与源码指纹的校验在 build_cache() 里, 故调用前必须先跑一次 build_cache()。
206     """
207     import common as cm
208     from spectral import HerbSolver
209     z = np.load(os.path.join(DATADIR, "solutions.npz"))
210     return z, HerbSolver, cm

```

3.10 论文插图生成: make_figures.py

```

1  # -*- coding: utf-8 -*-
2  # =====
3  # make_figures.py —— 生成论文插图 (本脚本产出 25 张 PNG, 其中 24 张被正文引用)
4  # =====
5  # 做什么: 按论文出现顺序逐张画出 25 幅插图: 前 4 幅是附件 1/2 的数据处理与环境标定
6  # 诊断, 中段是问题 1-4 的径向剖面、时空云图与烘干终点, 末段是三层验证、
7
8  # 收敛性与误差分析, 最后 1 幅是技术路线流程图。每幅图画的是哪个物理量、横纵轴
9  # 是什么、数据取自哪个文件的哪个键、各子图 (a)(b)(c) 分别是什么, 都写在该段
10 # 代码上方的注释里, 按注释对照图即可。
11 # 输入: 中间数据/solutions.npz (四问解; 缺失或版本/指纹不符时由 fig_core.build_cache()
12 # 自动重算); 中间数据/verification.json (收敛性与容差数据, fig22、fig23 使用,
13 # 由 verify_data.py --recompute 或 run_all.py 生成); 附件 1/2 原始序列经
14
15 # common.py 载入。
16 # 输出: figure/fig01_env_fit.png ... figure/fig25_flow.png 共 25 张, 240 dpi。另有受控对照图
17 # fig20_p4_controlled.png 由 make_controlled_comparison.py 单独生成, 全合计 26 张;
18 # 正文共引用 25 张, 本脚本这 25 张里只有备用图 fig20_p4_vs_p3.png 未被引用——它把物性
19 # 与几何同时改变后的净差异画在一起, 差值不能当作纯收缩效应。
20 # 用法: python make_figures.py (先跑过 run_all.py 或 make_results.py, 再直接出图)
21
22 # 关键变量:
23 # RCOLS 剖面采样的 5 个物理半径 [m]: 0、0.005、0.010、0.015、0.020, 即 0-2.0 cm
24 # CRIT 烘干判断阈值 0.15 kg/kg (干基含水率)
25 # FLUX_T_END 表面传质通量曲线的终了时刻 72 h = 259200 s
26 # S1-S4 缓存解与插值接口打包成的轻量对象, 交给 _samples() 复用
27 # =====
28 import os
29 from console_utf8 import fix_console
30 fix_console()
31 import numpy as np
32 from scipy.special import j0, j1
33 import matplotlib.pyplot as plt
34 import matplotlib.path as mpath # 等值线数值的白描边
35 from matplotlib.patches import FancyBboxPatch # 技术路线与示意图的圆角方框
36 from fig_core import (save, load_cache, build_cache, FIGDIR, DATADIR, DPI)
37 import fig_core as FC
38 import verify_data as VD
39
40 RCOLS = np.array([0.0, 0.005, 0.010, 0.015, 0.020]) # m, 即 0、0.5、1.0、1.5、2.0 cm
41 CRIT = 0.15 # kg/kg (干基) 烘干判断阈值
42
43 # ----- 全篇语义色表 -----
44 # 25 张图共用一套颜色语义: 同一种颜色在任何一页上永远指同一种东西。这是"翻到哪页都
45 # 显得专业"的底层原因, 也是读者不会看错图的保证。本表只作用于 make_figures.py:
46 # make_controlled_comparison.py 独立运行、不 import 本模块, 那里另有一份同值的副本,
47 # 改动时必须同步——否则同一含义在两个脚本出的图里会是两种颜色。
48 COL_TEMP = "#c0392b" # 温度: 暖红
49 COL_MOIST = "#1f6fb2" # 水分浓度: 冷蓝
50 COL_CRIT = "#2c3e50" # 判断 / 阈值线: 深靛 (一律点线)
51 COL_PHASE = "#b9770e" # 阶段分界、环境设定值: 赭黄 (一律虚线)
52 COL_MEAS = "#8a8f98" # 实测散点: 中灰 (数据是陪衬, 主结论才上色)
53 COL_SIDE = "#9aa0a6" # 陪衬曲线: 浅灰
54
55
56 def _glow(ax, x, y, color, lw=2.0, glow=3.6, alpha=0.20, zorder=3, **kw):
57     """画一条"发光主线": 先铺一层同色、宽而淡的底线, 再压一条实线, 返回实线的 Line2D。
58
59     目的是让主结论曲线在一堆陪衬线里第一眼就跳出来, 而不必把线加粗到失真。

```

```

60 底层用 alpha 而不是预混浅色, 是为了在深浅两种底色上都保持同一个色相。
61 注意两层线都进图例时会出现重复条目, 故主线的 label 只挂在实践上。
62 """
63 ax.plot(x, y, color=color, lw=glow, alpha=alpha, solid_capstyle="round",
64         zorder=zorder - 1)
65 return ax.plot(x, y, color=color, lw=lw, zorder=zorder, **kw)[0]
66
67
68 def _cross_time(t, v, crit):
69     """求 v 由大变小、首次跌破 crit 的时刻 [s], 返回 float; 始终大于 crit 则返回 nan。
70
71     用相邻两点线性求交, 而不是直接取第一个低于阈值的采样点: 采样间隔在问题 3 里是
72     百秒量级, 直接取点会把达标时刻系统地推迟若干步, 图上的标注与正文就对不上了。
73     """
74     idx = np.where(np.asarray(v) <= crit)[0]
75     if idx.size == 0:
76         return np.nan
77     k = int(idx[0])
78     if k == 0:
79         return float(t[0])
80     return float(t[k - 1] + (v[k - 1] - crit) / (v[k - 1] - v[k]) * (t[k] - t[k - 1]))
81
82
83 def _samples(s, sol, t_idx=None, rcols=RCOLS):
84     """把解按  $u=(r/R)^2$  插值到 rcols 给出的物理半径上, 返回 (C, T) 两个 (nt, nr) 数组。
85
86     C 为干基含水率 [kg/kg]、T 为温度 [degC]; nt 为取样时刻数、nr = len(rcols)。
87     t_idx 给出要取的时刻下标 (默认全部); s 需提供 R_of、interp_u 与 Np 属性,
88     形状取自传入的 sol (sol.y 的第 1 维是时刻、第 2 维是状态)。
89
90     问题 4 的半径随收缩变小, 固定的物理距离 r_j 可能在某个时刻已落到药材之外
91     ( $r_j > R(t_i)$ )。该位置此时不存在——按论文 S7.4 与 make_results.py 的约定填
92     NaN, 而不能把  $u_t=(r_j/R(t))^2 > 1$  送去插值: 那是对求解域之外的做外推, 浮点下
93     还会因重心插值分母抵消到 0 而返回 inf。
94     """
95     idx = range(sol.y.shape[1]) if t_idx is None else t_idx
96     idx = list(idx)
97     C = np.full((len(idx), len(rcols)), np.nan)
98     T = np.full((len(idx), len(rcols)), np.nan)
99     for k, i in enumerate(idx):
100         Rt = s.R_of(sol.t[i])
101         inside = rcols <= Rt + 1e-12
102         if not np.any(inside):
103             continue
104         Cn, Tn = s.interp_u(sol.y[:, i], (rcols[inside] / Rt) ** 2)
105         C[k, inside] = Cn
106         T[k, inside] = Tn
107     return C, T
108
109
110 def main():
111     """按论文引用顺序生成 25 张插图, 返回 (t_dry3, t_dry4) 两个烘干时刻 [s]。
112
113     绘图语句全部写在本函数体内, 按下面的分节注释一段一图。开头先把四问解从缓存取
114     出来, 再用 mk() 打包成只需 interp_u/R_of 的轻量对象交给 _samples(), 这样采样
115     逻辑只写一份, 四个问题共用。
116     """
117     build_cache()
118     z, HerbSolver, cm = load_cache()
119     s1 = HerbSolver(64, prob=1)
120     s2 = HerbSolver(64, prob=2)
121     s3 = HerbSolver(64, prob=3)
122     s4 = HerbSolver(64, prob=4, shrink=True)
123     t1, y1 = z["p1_t"], z["p1_y"]
124     t2, y2 = z["p2_t"], z["p2_y"]
125     t3, y3 = z["p3_t"], z["p3_y"]
126     t4, y4 = z["p4_t"], z["p4_y"]
127     td3 = float(z["p3_tdry"][0])
128     td4 = float(z["p4_tdry"][0])
129     print("tdry3=%.3f s   tdry4=%.3f s" % (td3, td4))
130
131     class _S: # 轻量容器, 复用 _samples 接口
132         pass
133
134     def mk(s, t, y):
135         """把 (求解器 s、缓存时刻 t [s]、解 y) 打包成 _S, 属性名与 _samples 的调用一致。"""
136         o = _S(); o.y = y; o.t = t; o.R_of = s.R_of
137         o.interp_u = s.interp_u; o.Np = s.Np; o.N = s.N; o.u = s.u
138         return o
139
140     S1, S2, S3, S4 = mk(s1, t1, y1), mk(s2, t2, y2), mk(s3, t3, y3), mk(s4, t4, y4)

```

```

141
142 # ----- 第 1 组：附件数据的处理与环境标定诊断 (fig01-fig04) -----
143 # 这一组只用附件本身与 common.py 的标定结果，不涉及四问的解，先画完再进入问题 1。
144 print("[fig01] 环境序列与拟合")
145 # ----- fig01: 烘房环境这条边界条件本身与其一阶惯性拟合 -----
146 # 数据取自 common.py 载入的附件 1 实测序列: cm._t_env [s], cm._Ta_env [degC]、
147 # cm._Ca_env [kg/kg] (0~4 h, 每 60 s 一点)。横轴均为 时间/h。
148 # (a) 纵轴 烘房温度/degC (灰点实测、红实线一阶惯性拟合、黑点线设定值 50.212 degC、
149 # 绿虚线标出 4 h 的阶段分界); (b) 纵轴 含湿量/(kg/kg)，同一套画法，设定值
150 # 0.050908 kg/kg。可见 4 h 之后实测与拟合基本重合，故其后直接取设定值。
151
152 # 加花：实测散点压成中灰、一阶惯性拟合线发光——这张图承担两个论点（数据处理合格、
153 # 阶段分界取 4 h），主角是拟合线而不是散点。两幅各加一处箭头标注，把正文那句
154 # "t 约 4 h 达到设定值并稳定"直接画在图上，读者不必回正文找依据。
155 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
156 tt = np.linspace(0, 14400, 400)
157 Ta_fit = cm.Tset - (cm.Tset - cm.T_air0) * np.exp(-tt / cm.tauT)
158 Ca_fit = cm.Cset - (cm.Cset - cm.C_air0) * np.exp(-tt / cm.tauC)
159 for k, (meas, fit, setv, lab_set) in enumerate([
160     (cm._Ta_env, Ta_fit, cm.Tset, r"$T_{\rm set}=50.212^{\circ}\text{C}$"),
161     (cm._Ca_env, Ca_fit, cm.Cset, r"$C_{\rm set}=0.050908\text{ kg/kg}$")]):
162     ax[k].plot(cm._t_env / 3600, meas, "o", ms=2.6, alpha=.45,
163               color=COL_MEAS, label="附件1 实测")
164     _glow(ax[k], tt / 3600, fit, COL_TEMP, lw=1.8, label="一阶惯性拟合")
165     ax[k].axhline(setv, color="k", ls=":", lw=1, label=lab_set)
166     ax[k].axvline(4.0, color=COL_PHASE, ls="--", lw=1.3, label="阶段分界 4 h")
167     ax[k].set_xlabel("时间 / h")
168     ax[k].legend(fontsize=8, loc="center right")
169 ax[0].set_ylabel("烘房温度 / $^{\circ}\text{C}$")
170 ax[0].set_title("(a) 烘房温度：实测与拟合")
171 ax[1].set_ylabel("含湿量 / (kg/kg)")
172 ax[1].set_title("(b) 烘房含湿量：实测与拟合")
173 # 标注文字贴着 4 h 分界线放在轴底：不加引线，避免引线横穿拟合曲线（装饰不许压数据）
174 for k, setv in enumerate([cm.Tset, cm.Cset]):
175     lo, hi = ax[k].get_ylim()
176     ax[k].text(3.88, lo + (hi - lo) * 0.03, "4 h 起\n达设定值",
177               fontsize=9.5, color=COL_PHASE, ha="right", va="bottom")
178 save(fig, "fig01_env_fit.png")
179
180 print("[fig02] 拟合残差诊断")
181 # ----- fig02: 拟合残差的时间序列诊断 -----
182 # 残差定义为 实测 - 拟合式，取附件 1 全序列。横轴均为 时间/h。
183 # (a) 纵轴 温度残差/degC，序列 RMSE = 0.336 degC; (b) 纵轴 含湿量残差/(g/kg)
184 # （原值乘 1e3；含湿量残差只有 1e-4 kg/kg 量级，不放大在图上就是一条直线），
185 # RMSE = 0.808 g/kg。两幅都画了 y=0 基准线，用来看残差有没有系统性偏移。
186 rT = cm._Ta_env - (cm.Tset - (cm.Tset - cm.T_air0) * np.exp(-cm._t_env / cm.tauT))
187 rC = cm._Ca_env - (cm.Cset - (cm.Cset - cm.C_air0) * np.exp(-cm._t_env / cm.tauC))
188 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
189 ax[0].plot(cm._t_env / 3600, rT, lw=1)
190 ax[0].axhline(0, color="k", lw=.8)
191 ax[0].set_xlabel("时间 / h"); ax[0].set_ylabel("残差 / $^{\circ}\text{C}$")
192 ax[0].set_title("(a) 温度拟合残差时序 (RMSE=0.336)")
193 ax[1].plot(cm._t_env / 3600, rC * 1e3, lw=1, color="C1")
194 ax[1].axhline(0, color="k", lw=.8)
195 ax[1].set_xlabel("时间 / h"); ax[1].set_ylabel("残差 / (g/kg)")
196 ax[1].set_title("(b) 含湿量拟合残差时序 (RMSE=0.808 g/kg)")
197 save(fig, "fig02_env_resid_time.png")
198
199 print("[fig03] 残差统计 (直方图+Q-Q)")
200 # ----- fig03: 残差的正态性统计诊断 (2x2) -----
201 # 上行 (a)(b) 是温度残差，下行 (c)(d) 是含湿量残差 (放大 1e3 成 g/kg，同 fig02)。
202 # (a)(c) 横轴 残差/对应单位、纵轴 概率密度：柱状为直方图，红实线为同均值同标准差的
203 # 正态密度; (b)(d) 是 Q-Q 图，横轴 理论分位、纵轴 样本分位。两幅用来看标定的残差
204 # 能否按白噪声处理，若明显偏离正态，后面用 RMSE 描述拟合优度就要加一句说明。
205 from scipy import stats
206 # hspace 必须手动放宽：默认 0.2 时下排两幅子图的标题会与上排子图的
207 # x 轴刻度数字叠在一起（实测在 9.2 in 画布 + 11 pt 标题下已经压字）。
208 fig, ax = plt.subplots(2, 2, figsize=(9.2, 6.6))
209 fig.subplots_adjust(hspace=0.45)
210 for k, (r, nm, un) in enumerate([(rT, "温度", "$^{\circ}\text{C}$"), (rC * 1e3, "含湿量", "g/kg")]):
211     ax[k, 0].hist(r, bins=24, density=True, alpha=.7, color="C4" % k)
212     xs = np.linspace(r.min(), r.max(), 200)
213     ax[k, 0].plot(xs, stats.norm.pdf(xs, r.mean(), r.std()), "r-", lw=1.6)
214     ax[k, 0].set_xlabel("残差 / %s" % un); ax[k, 0].set_ylabel("概率密度")
215     ax[k, 0].set_title("(%s) %s残差直方图与正态密度" % ("ac"[k], nm))
216     stats.probplot(r, dist="norm", plot=ax[k, 1])
217     ax[k, 1].set_title("(%s) %s残差 Q-Q 图" % ("bd"[k], nm))
218     ax[k, 1].set_xlabel("理论分位"); ax[k, 1].set_ylabel("样本分位")
219 save(fig, "fig03_env_resid_stat.png")
220
221 print("[fig04] 附件2 半径历史")

```

```

222 # ----- fig04: 附件 2 的药材半径历史 (问题 4 的收缩输入) -----
223 # 数据取自 common.py 载入的附件 2: cm._t_R [s] 与 cm._R_hist [m], 横轴均为 时间/h。
224 # (a) 纵轴 半径 R/cm; (b) 纵轴 逐区间收缩速率  $|\Delta R/\Delta t|/(cm/h)$ , 跨 3 个数量级用半对数轴;
225 # (c) 纵轴 无量纲收缩比  $R/R_0$ , 红虚线标出终点 0.599。
226 #
227 # 为什么 (b) 用逐区间差分而不是 np.gradient: 附件 2 是每 30 min 一点的实测序列,
228 #
229 # 半径按 0.001~0.127 cm 的**步进**下降 (144 个区间里 92 个  $\Delta R$  恰为 0), 中心差分会把
230 # 一次步进摊到相邻两个区间上、凭空造出"半格"的假台阶; 逐区间  $\Delta R/\Delta t$  才是这一步实际
231 # 发生的速率。半径不变的区间速率为**严格 0***, 半对数轴无法表示, 故置 NaN 断开不画
232 # (早先用 +1e-12 兜底, 结果在图上画出一条贴着 1e-12 的长直线, 读起来像"速率降到
233 # 1e-12 cm/h", 其实那一段是"收缩停了")。
234 tR = cm._t_R; Rh = cm._R_hist
235 d_rate = np.abs(np.diff(Rh) * 100) / (np.diff(tR) / 3600.0) # cm/h, 逐区间速率
236 d_xm = (tR[:-1] + tR[1:]) / 2 / 3600.0 # 区间中点 [h]
237 d_rate = np.where(d_rate > 0, d_rate, np.nan)
238 # 画布与布局: 三个面板都带中文/公式纵轴标签, 默认间距会把 (b)、(c) 的纵轴标签压进
239 # 左邻面板的绘图区 (实测压入 0.157 in, 观感就是"文字叠在一起"), 故改用 constrained
240 # 布局自动给标签留位。画布宽 8.2 in 是按"存出 PNG 宽 ÷ 版心宽"反推的: 印到 6.30 in
241 # 版心时轴标签 7.96 pt、刻度 7.20 pt, 都过 7 pt 可读线 (10.4 in 时只有 7.56 / 6.84)。
242 fig = plt.figure(figsize=(8.2, 3.05), layout="constrained")
243 ax = fig.subplots(1, 3)
244 ax[0].plot(tR / 3600, Rh * 100, "o-", ms=2.2, lw=1.2)
245 ax[0].set_xlabel("时间 / h"); ax[0].set_ylabel("$R$ / cm")
246 ax[0].set_title("(a) 半径历史")
247 ax[1].semilogy(d_xm, d_rate, "-o", ms=2.4, lw=1.2, color="C1")
248 ax[1].set_ylim(5e-4, 5e-1) # 数据范围 1e-3~2.5e-1, 上下各留约半格余量
249 ax[1].set_xlabel("时间 / h"); ax[1].set_ylabel(r"$|\mathbf{d}R/\mathbf{d}t|/(cm/h)$")
250 ax[1].set_title("(b) 收缩速率 (半对数)")
251 ax[2].plot(tR / 3600, Rh / cm.R0, lw=1.5, color="C2")
252 ax[2].axhline(0.599, color="r", ls="--", lw=1.2, label=r"$R/R_0=0.599$")
253 ax[2].set_xlabel("时间 / h"); ax[2].set_ylabel("$R/R_0$")
254 ax[2].set_title("(c) 无量纲收缩比"); ax[2].legend(fontsize=8)
255 save(fig, "fig04_radius.png")
256
257 print("[fig05] 无量纲分析")
258 # ----- fig05: 无量纲数与特征时间尺度 (问题 1 的物性口径) -----
259 # 本图不画解, 只用问题 1 的常数物性算无量纲数: 热扩散率  $\alpha=k/(rho*cp)$ 、
260 # 质扩散率  $D_0=D(C_0)=7e-9*exp(-0.89/2.55)$ 。横轴/纵轴见下。
261 # (a) 横轴 时间/s、纵轴 Fourier 数, 画  $Fo_T=\alpha*t/R_0^2$  与  $Fo_D=D_0*t/R_0^2$  两条线,
262 # 说明热快质慢; (b) 横轴 特征时间/h (对数)、纵轴三个尺度  $R_0^2/\alpha$ 、 $R_0^2/D_0$ 、
263 #  $L^2/\alpha$ ; (c) 关掉坐标轴, 直接列出  $Bi$ 、 $Bi_m$ 、 $Le$  与两个  $Fo$  在  $t=1800$  s 的值。
264 alpha = 0.36 / (820 * 2600)
265 D0 = 7e-9 * np.exp(-0.89 / 2.55)
266 # 画布由 10.4 in 缩到 9.6 in (字号不动): 印后字号按"存出 PNG 宽 ÷ 6.30 in 版心"算,
267 # 10.4 in 时刻度只有 6.84 pt、低于 7 pt 的可读下限; 缩到 9.6 in 后刻度约 7.5 pt、
268 # 轴标签约 8.3 pt。宽度仍是 \textwidth, 图的印刷高度基本不变。
269 fig, ax = plt.subplots(1, 3, figsize=(9.6, 2.85))
270 tts = np.linspace(60, 1800, 200)
271 ax[0].plot(tts, alpha * tts / cm.R0 ** 2, lw=1.6, label=r"${\rm Fo}_T$")
272 ax[0].plot(tts, D0 * tts / cm.R0 ** 2, lw=1.6, label=r"${\rm Fo}_D$")
273 ax[0].set_xlabel("时间 / s"); ax[0].set_ylabel("Fourier 数")
274 ax[0].set_title("(a)  $\mathbf{Fo}_D$  与  $\mathbf{Fo}_T$ : 热快质慢"); ax[0].legend()
275 names = [" $R_0^2/\alpha$ ", " $R_0^2/D_0$ ", " $L^2/\alpha$ "]
276 vals = [cm.R0 ** 2 / alpha / 3600, cm.R0 ** 2 / D0 / 3600, 0.25 ** 2 / alpha / 3600]
277 ax[1].barh(names, vals, color=["C0", "C1", "C2"])
278 ax[1].set_xscale("log"); ax[1].set_xlabel("特征时间 / h")
279 ax[1].set_title("(b) 特征时间尺度")
280 # 横轴上限放宽到 600 h: 最长那条 102.8 h 的数值标签原来按  $v*1.15$  摆放, 右端会跑出
281 # 坐标框 0.34 in (印后 74 pt), 正是"文字压到图上"的典型。放宽上限后条长仍占 76%,
282 # 而标签改按固定 4 pt 摆, 不再随数量级变化。
283 ax[1].set_xlim(0.35, 600)
284 ax[1].tick_params(axis="y", pad=5.0) # 类别标签与色条左端之间留出约 5 pt
285 for i, v in enumerate(vals):
286     ax[1].annotate("%.1f h" % v, xy=(v, i), xytext=(4, 0), textcoords="offset points",
287                    va="center", ha="left", fontsize=8)
288 ax[2].axis("off")
289 # (c) 原先把 5 个量用等宽字体拼成一个整块, 字体与全篇不一致、(c) 与 (a)(b) 像是两篇
290 # 文章的图。改成两列排版: 左列符号右对齐、右列数值左对齐, "=" 自然成一条竖线。
291 rows = [(r"${\mathbf{Bi}}$", 25 * cm.R0 / 0.36),
292         (r"${\mathbf{Bi}}_m$", 8e-7 * cm.R0 / D0),
293         (r"${\mathbf{Le}}$", D0 / alpha),
294         (r"${\mathbf{Fo}}_T(1800)$", alpha * 1800 / cm.R0 ** 2),
295         (r"${\mathbf{Fo}}_D(1800)$", D0 * 1800 / cm.R0 ** 2)]
296 for k, (sym, val) in enumerate(rows):
297     yy = 0.78 - 0.16 * k
298     ax[2].text(0.47, yy, sym, ha="right", va="center", fontsize=10.5)
299     ax[2].text(0.53, yy, "= %.4f" % val, ha="left", va="center", fontsize=10.5)
300 ax[2].set_title("(c) 无量纲数与判据")
301 save(fig, "fig05_dimensionless.png")
302

```

```

303 # ----- 第 2 组: 坐标变换示意 (fig06) -----
304 # 下面一幅图不画数据, 只画离散节点本身, 用来说明为什么换到  $u=xi^2$  求解。
305 print("[fig06] 坐标变换示意")
306 # ----- fig06: 常规  $r$  坐标与正则化坐标  $u=xi^2$  的节点分布 -----
307 # 画的是离散节点位置, 不涉及物理解。节点由  $N=16$  的 Chebyshev-Gauss-Lobatto 点
308 #  $x_j$  映射而来:  $r_j=(1-x_j)*R_0/2$ ,  $u_j=(1-x_j)/2$ 。
309 # (a) 横轴  $r/cm$ 、纵轴隐去: 常规  $r$  坐标下节点在轴心处高度密集,  $r=0$  处算子奇异、需要
310 # 特殊处理; (b) 横轴  $u=xi^2$ : 同一批节点在  $u$  坐标下等距,  $u=0$  处算子已正则;
311 # (c) 横轴  $u$ 、纵轴  $r/cm$ : 画出映射曲线  $r=R*\sqrt{u}$  及节点落在曲线上的位置。
312
313 # 画布与布局: 默认间距下 (c) 的纵轴标签会压进 (b) 的绘图区 (实测 0.048 in), 故用
314 # constrained 布局自动给标签留位; 画布宽 8.2 in 是按"存出 PNG 宽 ÷ 版心宽"反推的,
315 # 印到 6.30 in 版心时轴标签约 8.1 pt、刻度约 7.3 pt。
316 fig = plt.figure(figsize=(8.2, 3.05), layout="constrained")
317 ax = fig.subplots(1, 3)
318 N = 16
319 xj = np.cos(np.pi * np.arange(N + 1) / N)
320 rj = (1 - xj) / 2 * cm.R0 * 100
321 aj = (1 - xj) / 2
322 ax[0].plot(rj, np.zeros_like(rj), "o", ms=6, color="C3")
323 ax[0].set_ylim(-1, 1); ax[0].set_yticks([])
324 ax[0].set_xlabel("$r$ / cm"); ax[0].set_title("(a) 常规 $r$ 坐标节点\n轴心处密集、需特殊处理")
325 ax[1].plot(aj, np.zeros_like(aj), "o", ms=6, color="C0")
326 ax[1].set_ylim(-1, 1); ax[1].set_yticks([])
327 ax[1].set_xlabel("$u=\xi^2$"); ax[1].set_title("(b) 正则化坐标 $u$ 节点\n$u=0$ 处算子正则")
328 uu = np.linspace(0, 1, 200)
329 ax[2].plot(uu, np.sqrt(uu) * cm.R0 * 100, lw=1.8)
330 ax[2].plot(aj, np.sqrt(aj) * cm.R0 * 100, "o", ms=6, color="C0")
331 ax[2].set_xlabel("$u$"); ax[2].set_ylabel("$r$ / cm")
332 ax[2].set_title("(c) 映射 $r=R\sqrt{u}$")
333 save(fig, "fig06_u_transform.png")
334
335 # ----- 第 3 组: 问题 1 的解与离散精度验证 (fig07~fig11) -----
336 # 问题 1 的物性取附录 2 的常数, 温度场与水分分解耦; fig07~fig09 画数值解本身,
337 # fig10~fig11 拿 Bessel 闭式解作基准校核离散精度。
338 C1, T1 = _samples(S1, S1)
339 print("[fig07-08] 问题1 剖面 / 云图")
340 # ----- fig07: 问题 1 的温度场与水分浓度场的径向剖面 -----
341 # 数据取自 中间数据/solutions.npz 的 p1_t[s] 与 p1_y (前 65 行是水分浓度 C、后 65 行
342 # 是温度 T), 经 _samples() 按  $u=(r/R_0)^2$  插值到 RCOLS 五个物理半径, 另存 C1、T1。
343 # 横轴均为 到中心距离  $r/cm$ 。(a) 纵轴 温度/degC; (b) 纵轴 水分浓度 C/(kg/kg)。
344 # 两幅都画  $t = 100、600、1200、1800$  s 四个时刻, 对比即可看出热与质传播的快慢差异。
345 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
346 for k, tt in enumerate([100, 600, 1200, 1800]):
347     i = int(np.argmax(np.abs(t1 - tt)))
348     ax[0].plot(RCOLS * 100, T1[i], "o-", ms=3.5, lw=1.4, label="t=%d s" % tt)
349     ax[1].plot(RCOLS * 100, C1[i], "s-", ms=3.5, lw=1.4, label="t=%d s" % tt)
350 ax[0].set_xlabel("到中心距离 $r$ / cm"); ax[0].set_ylabel("温度 / $^\circ\text{C}$")
351 ax[0].set_title("(a) 问题1 温度剖面演化"); ax[0].legend(fontsize=8)
352 ax[1].set_xlabel("到中心距离 $r$ / cm"); ax[1].set_ylabel("水分浓度 / (kg/kg)")
353 ax[1].set_title("(b) 问题1 水分浓度剖面演化"); ax[1].legend(fontsize=8)
354 save(fig, "fig07_p1_profiles.png")
355
356 # ----- fig08: 问题 1 的时空云图 (温度场与水分浓度场) -----
357 # 数据同样来自 p1_t、p1_y, 这里改用 220 个  $u$  节点密采样, 再按  $r=\sqrt{u}*R_0$  换算
358 # 成物理半径 rr [cm]。横轴均为  $r/cm$ 、纵轴均为 时间/min。
359 # (a) 温度时空云图, 色标 温度/degC; (b) 水分浓度时空云图, 色标 C/(kg/kg)。
360 # 与 fig05(c) 的 Fo_T、Fo_D 对照可见温度场已经先趋于均匀, 水分浓度场却基本未动。
361 # 加花: 4 张云图原先只有一块裸色块, 看不出"结构在哪"。这里叠等值线 + 线上标数值
362
363 # (深色描边白字, 压在任何底色上都读得清), 并在 (b) 右上角加一处内嵌放大, 把
364 #  $r>1.8$  cm 那条约 2 mm 厚的表面薄层摊开——问题 1 的物理内容全挤在这条薄带里。
365 # 内嵌面板的字号与主面板同级 (9.5 pt, 印后 7.2 pt): 不能按"内嵌就该小一号"缩,
366 # 缩了印到纸上就低于 7 pt 的可读下限。等值线是插值结果, 本图  $u$  方向 220 点、
367 # 时间方向 181 点, 足以支撑这几个水平, 不会造出假细节。
368 uu = np.linspace(0, 1, 220)
369 rr = np.sqrt(uu) * cm.R0 * 100
370 Tg = np.array([s1.interp_u(y1[:, i], uu)[1] for i in range(len(t1))])
371 Cg = np.array([s1.interp_u(y1[:, i], uu)[0] for i in range(len(t1))])
372 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
373 # colorbar 是从 ax[0] 里"偷"走宽度的, 不占用两幅子图之间的间隙;
374 # 默认 wspace 下 colorbar 的标签会顶到右图的 y 轴标题上, 故手动放宽。
375 fig.subplots_adjust(wspace=0.42)
376 tmin = t1 / 60
377 m0 = ax[0].pcolormesh(rr, tmin, Tg, shading="auto", cmap="inferno")
378 plt.colorbar(m0, ax=ax[0], label="温度 / $^\circ\text{C}$")
379 ax[0].set_xlabel("$r$ / cm"); ax[0].set_ylabel("时间 / min")
380 ax[0].set_title("(a) 问题1 温度时空云图")
381 m1 = ax[1].pcolormesh(rr, tmin, Cg, shading="auto", cmap="viridis")
382 plt.colorbar(m1, ax=ax[1], label="水分浓度 / (kg/kg)")
383 ax[1].set_xlabel("$r$ / cm"); ax[1].set_ylabel("时间 / min")

```

```

384 ax[1].set_title("(b) 问题1 水分浓度时空云图")
385 # 云图上关网格: 全局 axes.grid=True 会把虚线网格叠在色块上, 盖住数据纹理
386 ax[0].grid(False); ax[1].grid(False)
387 # 等值线取物理上有含义的整数, 数值直接标在线上 (深色字 + 白描边, 任何底色上都读得清)
388 lv_T = [30, 32, 34, 36]
389 lv_C = [2.0, 2.2, 2.4, 2.5]
390 for a, Z, lv in ((ax[0], Tg, lv_T), (ax[1], Cg, lv_C)):
391     cs = a.contour(rr, tmin, Z, levels=lv, colors="white", linewidths=0.9)
392     for t in a.clabel(cs, fmt="%g", fontsize=9, inline=True, inline_spacing=2):
393         t.set_color("#1a1a1a")
394         t.set_path_effects([pe.withStroke(linewidth=2.0, foreground="white")])
395 # 内嵌放大: 放在 (b) 左上角那一整块"纯色区"上——那里本来没有信息, 压掉不损失任何东西,
396 # 却能把 r>1.75 cm 那条约 2 mm 厚的表面薄层摊开。不放 indicate_inset_zoom 的连接线:
397 # 放大区在右边缘, 引线会横穿整个色块。
398 axin = ax[1].inset_axes([0.075, 0.47, 0.47, 0.41])
399 axin.pcolormesh(rr, tmin, Cg, shading="auto", cmap="viridis")
400 cs2 = axin.contour(rr, tmin, Cg, levels=lv_C, colors="white", linewidths=0.8)
401 for t in axin.clabel(cs2, fmt="%g", fontsize=9, inline=True, inline_spacing=2):
402     t.set_color("#1a1a1a")
403     t.set_path_effects([pe.withStroke(linewidth=2.0, foreground="white")])
404 axin.set_xlim(1.75, 2.0)
405 axin.tick_params(labelsize=9.5, labelleft=False)
406 axin.set_title("表面薄层 $r>1.75$ cm", fontsize=9.5, pad=3)
407 axin.grid(False)
408 for s in axin.spines.values():
409     s.set_color("white"); s.set_linewidth(1.4)
410 save(fig, "fig08_p1_spacetime.png")
411
412 print("[fig09] 问题1 表面薄层与 Robin 条件核对")
413 # ----- fig09: 问题 1 表面薄层的 Robin 边界条件核对 -----
414 # 取最后一条剖面 (t=1800 s)。(a) 横轴 r/cm——只放大到 1.6~2.0 cm 的表面薄层, 纵轴
415 # 水分浓度 C/(kg/kg), 黑点线标出表面 r=2.0 cm; (b) 纵轴 表面浓度梯度 |dC/dr|/(kg/kg/m):
416 # 左柱是剖面数值差分值, 右柱是 Robin 条件 -h_m*(C_s-C_air)/D 的预测值, 两者的相对差
417 # 写在子图标题里。这是边界条件实现得对不对的一道直接校验。
418 i18 = len(t1) - 1
419 Cn, _ = s1.interp_u(y1[:, i18], uu)
420 Ds = 7e-9 * np.exp(-0.89 / max(Cn[-1], 1e-3))
421 grad_num = (Cn[-1] - Cn[-2]) / ((rr[-1] - rr[-2]) / 100)
422 grad_robin = -8e-7 * (Cn[-1] - float(cm.C_air(1800.0))) / Ds
423 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
424 ax[0].plot(rr, Cn, lw=1.8)
425 ax[0].axvline(2.0, color="k", ls=":", lw=1)
426 ax[0].set_xlim(1.6, 2.0)
427 ax[0].set_xlabel("$r$ / cm"); ax[0].set_ylabel("C / (kg/kg)")
428 ax[0].set_title("(a) 1800 s 近表面水分剖面 (薄层)")
429 # 两根柱值几乎相等 (相对差 <1%), 柱宽取 0.32: 默认 0.8 会把绘图区填成两块实心
430 # 色块, 过细则两根之间空出一大片。同时把组中心间距从默认的 1.0 收到 0.5 并收紧
431 # xlim, 让数据占满画幅; 柱顶标出数值, 便于直接核对。
432 _v9 = [abs(grad_num), abs(grad_robin)]
433 _x9 = np.array([0.0, 0.5])
434 ax[1].bar(_x9, _v9, width=0.32, color=["C0", "C1"])
435 ax[1].set_xticks(_x9)
436 ax[1].set_xticklabels(["数值差分\n(Robin 左端)", "Robin 条件\n右端"])
437 ax[1].set_xlim(-0.30, 0.80)
438 for _x, _v in zip(_x9, _v9):
439     ax[1].text(_x, _v, "%.1f" % _v, ha="center", va="bottom", fontsize=10.5)
440 ax[1].set_ylim(0, max(_v9) * 1.14)
441 ax[1].set_ylabel("$\\partial C / \\partial r$ / (kg/kg/m)")
442 ax[1].set_title("(b) 表面梯度: 差分值 vs Robin 预测\n(相对差 %.1f%)"
443               % (abs(abs(grad_num) - abs(grad_robin)) / abs(grad_robin) * 100))
444 save(fig, "fig09_p1_surface.png")
445
446 print("[fig10-11] Bessel 展开与验证")
447 # ----- fig10: 问题 1 闭式解 (Bessel 级数) 的收敛速度 -----
448 # 数据取自 analytic_p1.py 在导入时算出的展开系数 an.BN 与各阶衰减率 an.GAM。
449 # (a) 横轴 阶数 n、纵轴 |b_n| (半对数): 系数随阶数快速衰减;
450 # (b) 横轴 阶数 n、纵轴 gamma_n*t (t=1800 s), 黑虚线为 y=1: n>=2 的阶次 gamma_n*t
451 # 已远大于 1, 对应 e^{-gamma_n t} 可忽略, 故闭式解只取前几阶即可当作基准解。
452 import analytic_p1 as an
453 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
454 n = np.arange(1, 9)
455 ax[0].semilogy(n, np.abs(an.BN[:8]), "o-", lw=1.5)
456 ax[0].set_xlabel("阶数 $n$"); ax[0].set_ylabel("$|b_n|$")
457 ax[0].set_title("(a) 展开系数 $b_n$ 的衰减")
458 ax[1].plot(n, an.GAM[:8] * 1800, "s-", lw=1.5, color="C3")
459 ax[1].axhline(1, color="k", ls="--", lw=1)
460 ax[1].set_xlabel("阶数 $n$"); ax[1].set_ylabel("$\\gamma_n t$ (t=1800 s)")
461 ax[1].set_title("(b) 各阶衰减因子: $n \\geq 2$ 项已被压至 $e^{-13}$ 以下")
462 save(fig, "fig10_bessel_coef.png")
463
464 # ----- fig11: 谱配置法与闭式解的一致性验证 -----

```

```

465 # 基准是 analytic_p1.T_exact 给出的闭式温度解, 被校核的是 HerbSolver 在 t=1800 s 的
466 # 解: 用 _bary 把 u 节点上的解重心插值到 201 个 (r/R_0)^2 处。
467 # (a) 横轴 节点数 N、纵轴 与闭式解的最大偏差/K (半对数): N=8 即降到 1e-10, 红虚线
468 # 是 1e-10 的时间积分误差平台; (b) 横轴 r/cm、纵轴 偏差/(1e-10 K): 偏差沿半径的分布,
469 # 量级已是纯舍入水平, 说明空间离散本身没有引入系统性误差。
470 rr2 = np.linspace(0, cm.R0, 201)
471 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
472 Ns = [4, 6, 8, 12, 16, 24, 32, 48, 64]
473 err = []
474 for NN in Ns:
475     ss = HerbSolver(NN, prob=1)
476     a_ = np.concatenate([np.full(ss.Np, 1e-14), np.full(ss.Np, 1e-12)])
477     so = ss.solve(1800.0, t_eval=[1800.0], rtol=1e-12, atol=a_)
478     Ti = ss._bary(so.y[ss.Np:, -1], (rr2 / cm.R0) ** 2)
479     err.append(np.max(np.abs(Ti - an.T_exact(rr2, 1800.0))))
480 ax[0].semilogy(Ns, err, "o-", lw=1.6)
481 ax[0].axhline(1e-10, color="r", ls="--", lw=1, label="时间积分误差平台  $10^{-10}$ ")
482 ax[0].set_xlabel("节点数 N"); ax[0].set_ylabel("与闭式解的最大偏差 / K")
483 ax[0].set_title("(a) 谱收敛性: N=8 即达  $10^{-10}$ "); ax[0].legend(fontsize=8)
484 ss = HerbSolver(64, prob=1)
485 a_ = np.concatenate([np.full(ss.Np, 1e-14), np.full(ss.Np, 1e-12)])
486 so = ss.solve(1800.0, t_eval=[1800.0], rtol=1e-12, atol=a_)
487 Ti = ss._bary(so.y[ss.Np:, -1], (rr2 / cm.R0) ** 2)
488 res = Ti - an.T_exact(rr2, 1800.0)
489 ax[1].plot(rr2 * 100, res * 1e10, lw=1.4)
490 ax[1].set_xlabel("$r$ / cm"); ax[1].set_ylabel("偏差 / ( $10^{-10}$  K)")
491 ax[1].set_title("(b) 偏差沿半径的分布 (纯舍入量级)")
492 save(fig, "fig11_bessel_validate.png")
493
494 # ----- 第 4 组: 问题 2 的变物性耦合解 (fig12-fig14) -----
495 # 问题 2 的物性取附录 3 (含 C、T 的强非线性关系), 温度场与水分场双向耦合。
496 C2, T2 = _samples(S2, S2)
497 print("[fig12-13] 问题2")
498 # ----- fig12: 问题 2 的温度与水分浓度剖面演化 -----
499 # 数据取自 solutions.npz 的 p2_t [s]、p2_y, 经 _samples() 插值到 RCOLS 得 C2、T2。
500 # 横轴均为 r/cm。(a) 纵轴 温度/degC; (b) 纵轴 C/(kg/kg)。两幅都画 0.5、1.0、1.5、
501 # 2.0、3.0 h 五个时刻; (b) 中表面浓度早早掉到很低的水平而中心仍然很高, 即干壳形成。
502 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
503 hs = []
504 for tt in [0.5, 1.0, 1.5, 2.0, 3.0]:
505     i = int(np.argmax(np.abs(t2 - tt * 3600)))
506     hh, = ax[0].plot(RCOLS * 100, T2[i], "o-", ms=3.5, lw=1.4, label="%.1f h" % tt)
507     ax[1].plot(RCOLS * 100, C2[i], "s-", ms=3.5, lw=1.4, label="%.1f h" % tt)
508     hs.append(hh)
509 ax[0].set_xlabel("$r$ / cm"); ax[0].set_ylabel("温度 /  $^{\circ}\text{C}$ ")
510 ax[0].set_title("(a) 问题2 温度剖面演化")
511 ax[1].set_xlabel("$r$ / cm"); ax[1].set_ylabel("C / (kg/kg)")
512 ax[1].set_title("(b) 问题2 水分剖面演化 (干壳形成)")
513 # 两幅子图的时刻序列完全相同, 故共用一条图例放到底部:
514 # 5 条曲线在面板内铺满整个纵向量程, 面板内已无空位可放图例
515 # (放左下角会盖住 0.5 h 曲线, 而该曲线横跨整个版心)。
516 fig.legend(hs, ["%.1f h" % tt for tt in [0.5, 1.0, 1.5, 2.0, 3.0]],
517           loc="lower center", ncol=5, frameon=False, fontsize=9,
518           bbox_to_anchor=(0.5, -0.015))
519 fig.subplots_adjust(bottom=0.24)
520 save(fig, "fig12_p2_profiles.png")
521
522 # ----- fig13: 问题 2 的时空云图 (0-3 h) -----
523 # 数据同 fig12 (p2_t、p2_y), 密采样 220 个 u 节点。横轴均为 r/cm、纵轴均为 时间/h。
524 # (a) 温度云图, 色标 温度/degC; (b) 水分浓度云图, 色标 C/(kg/kg); (c) 干壳薄层放大。
525 # 与 (a) 对照可见温度场在径向已基本均匀, 水分却只在表面附近形成陡峭的低浓度薄层,
526 # 即所谓干壳。加花与 fig08 同一套: 叠等值线 + 线上标数值; 把"干壳边界在哪"直接摊开
527 # ——这是问题 2 的核心结论, 原先只能从颜色深浅猜。
528 Cg2 = np.array([s2.interp_u(y2[:, i], uu)[0] for i in range(len(t2))])
529 Tg2 = np.array([s2.interp_u(y2[:, i], uu)[1] for i in range(len(t2))])
530 # 三幅并排: (c) 把原先压在 (b) 左上角的内嵌大图拉出来独立成面板——小图不再遮挡
531 # 主图数据, 放大后的干壳薄层也看得清。画幅加宽到 13 in, 按 \textwidth 排入时高度反而
532 # 比原来的 9.6×3.4 更矮, 不会挤占正文版面; 字号相应放大以补偿缩印比例。
533 fig, ax = plt.subplots(1, 3, figsize=(13.0, 3.5))
534 # colorbar 刻度是四位 (47.5/42.5...), 比 fig08 更宽, 故 wspace 留得比默认大。
535 fig.subplots_adjust(wspace=0.42)
536 t2h = t2 / 3600
537 m0 = ax[0].pcolormesh(rr, t2h, Tg2, shading="auto", cmap="inferno")
538 plt.colorbar(m0, ax=ax[0], label="温度 /  $^{\circ}\text{C}$ ")
539 ax[0].set_xlabel("$r$ / cm"); ax[0].set_ylabel("时间 / h")
540 ax[0].set_title("(a) 问题2 温度时空云图")
541 m1 = ax[1].pcolormesh(rr, t2h, Cg2, shading="auto", cmap="viridis")
542 plt.colorbar(m1, ax=ax[1], label="C / (kg/kg)")
543 ax[1].set_xlabel("$r$ / cm"); ax[1].set_ylabel("时间 / h")
544 ax[1].set_title("(b) 问题2 水分时空云图")
545 m2 = ax[2].pcolormesh(rr, t2h, Cg2, shading="auto", cmap="viridis")

```

```

546 plt.colorbar(m2, ax=ax[2], label="C / (kg/kg)")
547 ax[2].set_xlim(1.7, 2.0)
548 ax[2].set_xlabel("$r$ / cm"); ax[2].set_ylabel("时间 / h")
549 ax[2].set_title("(c) 干壳薄层放大" # 放大区间写进图注, 标题不再压到 y 轴刻度上
550 ax[0].grid(False); ax[1].grid(False); ax[2].grid(False)
551 lv_T2 = [32, 38, 44, 48]
552 lv_C2 = [1.2, 1.6, 2.0, 2.3]
553 # 等值线: (a) 温度场标数值; (b) 只留线、不标值——(b) 与 (c) 数据相同, 同一个量标两遍
554 # 既重复, 又会在 (b) 右边界上挤出框外 (实测最外侧那个标值压框 0.063 in); 放大后的
555 # (c) 里标值看得最清楚, 标值统一放那里。
556 for a, Z, lv, do_label in ((ax[0], Tg2, lv_T2, True),
557 (ax[1], Cg2, lv_C2, False),
558 (ax[2], Cg2, lv_C2, True)):
559     cs = a.contour(rr, t2h, Z, levels=lv, colors="white", linewidths=0.9)
560     if not do_label:
561         continue
562     for t in a.clabel(cs, fmt="%g", fontsize=11.5, inline=True, inline_spacing=2):
563         t.set_color("#1a1a1a")
564         t.set_path_effects([pe.withStroke(linewidth=2.0, foreground="white")])
565 for a in ax:
566     a.tick_params(labelsize=11.5)
567     a.xaxis.label.set_size(12); a.yaxis.label.set_size(12)
568     a.title.set_size(12.5)
569 save(fig, "fig13_p2_spacetime.png")
570
571 print("[fig14] 表面传质通量 (降速特征)")
572 # ----- fig14: 表面传质通量的降速特征与表面/中心水分浓度 -----
573 # 数据不由缓存提供: 这里另解一次问题 3 (prob=3, 控制方程、边界与附录 3 物性都与问题 2
574 # 相同) 并积分到 72 h, 再从解的 y 中取 u=1 的表面 C_s 与 u=0 的中心 C_0 两条序列。
575 # 横轴均为 时间/h。(a) 纵轴 表面传质通量 |h_m*(C_s-C_air)|/(kg/(m2 s)), 半对数, 用来看
576 # 干壳形成后通量如何持续衰减; (b) 纵轴 C/(kg/kg), 两条曲线分别标为"中心"与"表面",
577 # 中心浓度的长平台正是 (a) 里通量衰减那么快的原因。
578
579 # 问题 2 只要求输出 0~3 h, 但正文用 6~72 h 的通量衰减来说明降速特征。
580 # 问题 2 与问题 3 的控制方程、边界和附录 3 物性完全相同, 故这里直接用
581 # 问题 3 的解 (它本来就积分到 t_dry 之后) 把通量曲线画满 72 h,
582 # 避免"正文讲 6~72 h、图只画 0~3 h"的口径不一致。
583 FLUX_T_END = 72 * 3600.0
584 s_flux = HerbSolver(64, prob=3)
585 at_f = np.concatenate([np.full(s_flux.Np, 1e-13), np.full(s_flux.Np, 1e-11)])
586 t_flux = np.linspace(0.0, FLUX_T_END, 433)
587 sol_flux = s_flux.solve(FLUX_T_END, t_eval=list(t_flux), rtol=1e-11, atol=at_f)
588 Cs_flux = sol_flux.y[s_flux.Np - 1, :]
589 C0_flux = sol_flux.y[0, :]
590 flux = 8e-7 * (Cs_flux - cm.C_air(t_flux)) # kg/(m^2 s)
591 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
592 ax[0].semilogy(t_flux / 3600, np.abs(flux), lw=1.8, color="C3")
593 ax[0].set_xlabel("时间 / h")
594 ax[0].set_ylabel("表面传质通量 $|h_m(C_s-C_{\mathrm{air}})|$ / (kg/(m$^2\cdot s$))")
595 ax[0].set_title("(a) 表面传质通量: 0~72 h 降速衰减")
596 ax[1].plot(t_flux / 3600, C0_flux, lw=1.8, label="中心")
597 ax[1].plot(t_flux / 3600, Cs_flux, lw=1.8, label="表面")
598 ax[1].set_xlabel("时间 / h"); ax[1].set_ylabel("C / (kg/kg)")
599 ax[1].set_title("(b) 中心与表面水分浓度 (同期)"); ax[1].legend(fontsize=8)
600 save(fig, "fig14_p2_flux.png")
601
602 # ----- 第 5 组: 问题 3 的烘干时长与降速特征 (fig15~fig17) -----
603 C3, T3 = _samples(S3, S3)
604 # 截面平均按论文 S11.6 的定义 C_bar=int_0^1 C(u,t)du 计算 (面积平均)。
605 # 不能用 5 个物理半径采样点的算术平均代替, 达标时刻也不能写死。
606 Cbar3 = y3[:s3.Np].T @ s3.area_weights()
607 t_bar3 = _cross_time(t3, Cbar3, CRIT)
608 lag_h = (td3 - t_bar3) / 3600.0
609 under_pct = (td3 - t_bar3) / td3 * 100.0
610 print(" 面积平均达标 %.4f h (中心 %.4f h); 滞后 %.2f h, 低估 %.2f%"
611 % (t_bar3 / 3600, td3 / 3600, lag_h, under_pct))
612 print("[fig15-16] 问题3")
613 # ----- fig15: 问题 3 的半对数干燥曲线与"平均判据"的迟滞 -----
614 # 数据取自 solutions.npz 的 p3_t [s], p3_y, 经 _samples() 插值到 RCOLS 得 C3 [kg/kg]。
615 # (a) 横轴 时间/h (限定 0~62 h)、纵轴 C/(kg/kg) 半对数: 画 r=0、1.0、2.0 cm 三条
616 # 干燥曲线, 红虚线是判据 0.15, 黑点线是中心达标时刻 t_dry;
617 # (b) 横轴 时间/h、纵轴 C/(kg/kg), 把中心 C_0 (实线) 与截面面积平均 Cbar (虚线) 画在
618 # 一起: Cbar 比 C_0 早达标, 即按平均判据算法会把烘干时长低估 under_pct% (写在子图标题里)。
619 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
620 # 加花: 把"预热平衡 / 恒温干燥"两阶段直接铺成底纹。本题的工艺结构就写在这条
621 # 4 h 分界上, 而原先 0~57 h 的横轴上完全看不出来, 读者只能回正文找。底纹 alpha
622 # 压到 0.075, 低于判据线与曲线的对比度, 不会盖住任何数据。
623 ax[0].axvspan(0, 4, color=COL_PHASE, alpha=0.075)
624 ax[0].axvspan(4, 62, color=COL_MOIST, alpha=0.055)
625 # 阶段名原写在底纹带里 (y=2.05): 4 h 宽的带放不下 4 个中文字, 文字必然横向伸出带外,
626 # 纵向又正好落在 r=0 与 r=1.0 cm 两条曲线上 (实测压住 55 个数据点, 左端还超出坐标框

```

```

627 # 0.15 in)。改写到曲线上方的空白带(y约2.9,曲线最高只到2.55),各接一段短引线指回
628 # 自己的阶段带;引线只经过 y=2.68-2.80 这段无数据的区间。
629 ax[0].annotate("预热平衡", xy=(2.0, 2.68), xytext=(1.0, 2.92), ha="left", va="center",
630               fontsize=8.5, color=COL_PHASE,
631               arrowprops=dict(arrowstyle="-", color=COL_PHASE, lw=0.8,
632                               alpha=0.75, shrinkA=1, shrinkB=1))
633 ax[0].annotate("恒温干燥", xy=(20.0, 2.68), xytext=(19.0, 2.92), ha="left", va="center",
634               fontsize=8.5, color=COL_MOIST,
635               arrowprops=dict(arrowstyle="-", color=COL_MOIST, lw=0.8,
636                               alpha=0.75, shrinkA=1, shrinkB=1))
637 for j, lbl, is_main in [(0, "r=0 中心", True), (2, "r=1.0 cm", False), (4, "r=2.0 cm 表面", False)]:
638     # 中心干燥最慢、决定烘干时长,故 r=0 这条是主结论线,发光并加粗;其余两条灰细
639     if is_main:
640         _glow(ax[0], t3 / 3600, C3[:, j], COL_MOIST, lw=1.9, glow=3.6, label=lbl)
641     else:
642         ax[0].semilogy(t3 / 3600, C3[:, j], lw=1.4, color=COL_SIDE, label=lbl)
643 ax[0].axhline(CRIT, color=COL_CRIT, ls="--", lw=1.3, label="判据  $C \leq 0.15$ ")
644 ax[0].axvline(t3 / 3600, color="k", ls=":", lw=1.2)
645 ax[0].set_xlabel("时间 / h"); ax[0].set_ylabel("C / (kg/kg)")
646 ax[0].set_ylim(0.04, 3.5); ax[0].set_xlim(0, 62)
647 ax[0].set_yscale("log") # _glow 用的是 ax.plot, 纵轴对数必须显式设定
648 # t_dry 的说明原贴在左下角(x=36, y=0.055): 横跨 x=36-56 h 的那一格里,表面干燥
649 # 曲线恰好也在 0.054-0.058 (实测压住 556 个数据点)。改到两条干燥曲线之间的空带
650 # (y约0.10: 中心线与 r=1.0 cm 线在该区间都在 0.15 以上、表面线在 0.06 以下),
651 # 再看箭头指回终点。
652 ax[0].annotate(" $t_{\mathrm{dry}} = 0.4t_3$  h" % (t3 / 3600),
653               xy=(t3 / 3600, CRIT), xytext=(38.0, 0.10), ha="left", va="center",
654               fontsize=9.5, color="k",
655               arrowprops=dict(arrowstyle=">", color="k", lw=1.0,
656                               shrinkA=2, shrinkB=2))
657 ax[0].set_title("(a) 半对数干燥曲线")
658 # 判据的说明原先单独贴在虚线左下方,横向正好横穿表面干燥曲线(压住 10 个数据点);
659 # 收进图例——图例本就是回答"这条线是什么"的地方,且框内没有数据。四条目使图例底边
660 # 落到 y 约 0.9,与上面那行 t_dry 说明(顶边 0.5)仍留有余量。
661 ax[0].legend(fontsize=8, loc="upper right")
662 _glow(ax[1], t3 / 3600, C3[:, 0], COL_MOIST, lw=1.8, label="中心  $C_0$ ")
663 ax[1].semilogy(t3 / 3600, Cbar3, lw=1.6, ls="--", color=COL_SIDE,
664               label="截面平均  $\bar{C} = \int_0^1 C \, du$ ")
665 ax[1].axhline(CRIT, color=COL_CRIT, ls="--", lw=1.3)
666 ax[1].axvline(t_bar3 / 3600, color=COL_PHASE, ls=":", lw=1.2)
667 ax[1].axvline(t3 / 3600, color="k", ls=":", lw=1.2)
668 ax[1].annotate(" $\bar{C}$  达标  $0.1f$  h" % (t_bar3 / 3600),
669               xy=(t_bar3 / 3600 + 1.0, 0.5), fontsize=9.5, color=COL_PHASE)
670 ax[1].annotate(" $C_0$  达标  $0.1f$  h" % (t3 / 3600),
671               xy=(t3 / 3600 - 21, 0.9), fontsize=9.5)
672 ax[1].set_xlabel("时间 / h"); ax[1].set_ylabel("C / (kg/kg)")
673 ax[1].set_yscale("log") # 同上: _glow 走的是线性 plot, 对数轴要显式开
674 ax[1].set_title("(b) 判据迟滞: 平均判据低估  $0.1f\%$ " % under_pct); ax[1].legend(fontsize=8)
675 save(fig, "fig15_p3_drying.png")
676
677 # ----- fig16: 烘干终点附近的放大与中心干燥速率 -----
678 # 数据同 fig15 (C3 取自 p3_y), 横轴均为 时间/h。由相邻两点差分得中心干燥速率
679 # rate = -dC_0/dt [kg/(kg/h)]。
680 # (a) 只保留 52-60 h、纵轴 C/(kg/kg): 画中心与表面两条曲线,红虚线是判据 0.15,红星
681 # 标出 C_0(t_dry)=0.1500 的终点; (b) 纵轴 |rate|/(kg/kg/h) 半对数,用来看中心干燥
682 # 速率在降速期是否按指数衰减。
683
684 # 加花: 这是全文最关键的结论图——烘干时长本身原先只能回正文找。这里把 t_dry 直接
685 # 标注在图上、判据线标出阈值、中心曲线发光(它是判据取的那条线),表面曲线降为灰细。
686 # 注意 t_dry 必须由 t3 变量格式化: 论文摘要用的是 N=128 连续极限 57.1043 h,本图画
687 # 的是交付口径 N=64 的生产解,写死常量就会与图里的曲线对不上。
688 # 画布与布局: 默认间距下 (b) 的纵轴标签会压进 (a) 的绘图区(实测 0.015 in),故用
689 # constrained 布局;画布宽 8.2 in 按"存出 PNG 宽 ÷ 版心宽"反推,印后轴标签约 8 pt。
690 fig = plt.figure(figsize=(8.2, 3.05), layout="constrained")
691 ax = fig.subplots(1, 2)
692 m = (t3 / 3600 > 52) & (t3 / 3600 < 60)
693 _glow(ax[0], t3[m] / 3600, C3[m, 0], COL_MOIST, lw=2.2, glow=4.0, label="中心  $C_0$ ")
694 ax[0].plot(t3[m] / 3600, C3[m, -1], lw=1.4, color=COL_SIDE, label="表面  $C_s$ ")
695 ax[0].axhline(CRIT, color=COL_CRIT, ls="--", lw=1.4)
696 ax[0].axvline(t3 / 3600, color="k", ls=":", lw=1.4)
697 ax[0].plot([t3 / 3600], [CRIT], "*", color=COL_TEMP, ms=14, zorder=6)
698 # 纵轴上下各留一条空带: 下带给图例、上带给"判据"标签,中间那条才是标注框。
699 # 不留的话,标注框的引线会从图例上穿过去。
700 ax[0].set_ylim(0.02, 0.185)
701 # 标注框落在这两行之间的空白带(中心线在 0.15 以上、表面线在 0.054 以下,中间无数据)
702 ax[0].annotate(" $t_{\mathrm{dry}} = 0.4t_3$  h" % (t3 / 3600),
703               xy=(t3 / 3600, CRIT), xytext=(52.25, 0.088), fontsize=8.5,
704               color=COL_TEMP, ha="left", va="center",
705               bbox=dict(boxstyle="round,pad=0.28", fc="white", ec=COL_TEMP, lw=0.8),
706               arrowprops=dict(arrowstyle=">", color=COL_TEMP, lw=1.1))
707 ax[0].annotate("判据  $C \leq 0.15$ ", xy=(56.0, CRIT), xytext=(52.05, 0.181),

```

```

708         fontsize=9.5, color=COL_CRIT, va="top")
709 ax[0].set_xlabel("时间 / h"); ax[0].set_ylabel("C / (kg/kg)")
710 ax[0].set_title("(a) 干燥终点放大:  $C_0(t_{\text{dry}})=0.1500$ ")
711 ax[0].legend(fontsize=8, loc="lower left")
712 dt_ = np.diff(t3 / 3600); dC = np.diff(C3[:, 0])
713 rate = -dC / dt_
714 _glow(ax[1], (t3[:-1] / 3600, np.abs(rate), COL_MOIST, lw=1.6)
715 ax[1].set_yscale("log") # 同上: _glow 走线性 plot, 半对数图的纵轴要显式开
716 # 标出拟合用的降速段 30-54 h: 这段的斜率才是图 15 里那个特征时间
717 ax[1].axvspan(30, 54, color=COL_PHASE, alpha=0.075)
718 ax[1].text(42, 0.06, "降速段 30-54 h", fontsize=9.5, color=COL_PHASE, ha="center")
719 ax[1].set_xlabel("时间 / h"); ax[1].set_ylabel(" $-\ln \dot{C}_0$  / (kg/kg/h)")
720 ax[1].set_title("(b) 中心干燥速率指数衰减 (降速期)")
721 save(fig, "fig16_p3_endpoint.png")
722
723 print("[fig17] 干燥速率对数的线性拟合")
724 # ----- fig17: 干燥速率对数与浓度对数两条斜率的对比 -----
725 # 把 fig16(b) 的速率取自然对数, 再对时间做一次最小二乘 (窗口取 30-54 h)。
726 # (a) 横轴 时间/h、纵轴  $\ln(-dC_0/dt)$ : 散点是数据、红实线是拟合直线, 斜率单位为
727 #  $h^{-1}$ , 其负倒数即图上标的速率衰减特征时间;
728 # (b) 横轴 时间/h、纵轴  $\ln C_0$ : 浓度自身对数的斜率, 由 30 h 之后的数据拟合得到。
729 # 两幅斜率相差很多, 说明"速率按指数衰减"与"浓度按指数衰减"不是同一件事。
730 mm = (t3[:-1] / 3600 > 30) & (t3[:-1] / 3600 < 54) & (np.abs(rate) > 0)
731 xs = t3[:-1][mm] / 3600
732 ys = np.log(np.abs(rate[mm]))
733 co = np.polyfit(xs, ys, 1)
734 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
735 ax[0].plot(xs, ys, ".", ms=3, alpha=.5, label="数据")
736 ax[0].plot(xs, np.polyval(co, xs), "r-", lw=1.8,
737          label="拟合斜率  $.4f h^{-1}$ " % co[0])
738 ax[0].set_xlabel("时间 / h"); ax[0].set_ylabel(" $-\ln(-\dot{C}_0)$ ")
739 ax[0].set_title("(a) 30-54 h: 速率衰减的特征时间  $.1f h$ " % (-1 / co[0]))
740 ax[0].legend(fontsize=8)
741 ax[1].plot(t3 / 3600, np.log(C3[:, 0]), lw=1.8, color="C0")
742 ax[1].set_xlabel("时间 / h"); ax[1].set_ylabel(" $-\ln C_0$ ")
743 ax[1].set_title("(b) 浓度自身的对数斜率仅  $.4f h^{-1}$ " % np.polyfit(
744     t3[t3 / 3600 > 30] / 3600, np.log(C3[t3 / 3600 > 30, 0]), 1)[0])
745 save(fig, "fig17_p3_rate.png")
746
747 # ----- 第 6 组: 问题 4 的收缩效应 (fig18-fig20) -----
748 # 问题 4 物性取附录 4, 并叠加附件 2 的实测收缩  $R(t)$ 。 $u=xi^2$  是物质坐标: 收缩过程
749 # 中求解域在该坐标下固定为 0-1, 换算回物理距离要乘当前半径  $R(t)$ 。
750 C4, T4 = _samples(S4, S4)
751 Rt4 = np.array([s4.R_of(x) for x in t4])
752 print("[fig18-19] 问题4")
753 # ----- fig18: 问题 4 在物理坐标下的剖面与收缩中的表面半径 -----
754 # 数据取自 p4_t [s]、p4_y: 先用 Rt4[i] 取出该时刻的当前半径, 再把 u 节点换算成物理
755 # 距离  $r=\sqrt{u} \cdot R(t)$ , 浓度由 interp_u 取回。
756 # (a) 横轴 当前物理距离 r/cm、纵轴 C/(kg/kg): 画 6、12、18、24、48 h 五条剖面, 图例
757 # 括号里给的是该时刻的 R (cm), 可见剖面整体向左移动;
758 # (b) 横轴 时间/h、纵轴 R(t)/cm: 收缩中的表面半径本身, 输入来自附件 2。
759
760 # 加花: 这张图就是"模型到底有没有真的做收缩"的直接证据, 而原先每条剖面只是默默
761 # 左移了 13%, 视觉上几乎注意不到。改进三处: (1) 每条剖面的右端点 (= 当时的药材表面)
762 # 加实心圆点, 并用由浅到深的水分蓝区分早晚时刻; (2) 在 48 h 的端点直标当时的半径;
763 # (3) 用一条弧形箭头把 6 h 与 48 h 的端点连起来, 把"表面左移了多少"写成数字。
764 # 左移量由 Rt4 现算, 不写死——图上任何数字都必须对应代码里的一次真实计算。
765 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
766 shades = ["#9ecae1", "#6baed6", "#4292c6", "#2171b5", "#08306b"]
767 ends = {}
768 for k, tt in enumerate([6, 12, 18, 24, 48]):
769     i = int(np.argmax(np.abs(t4 - tt * 3600)))
770     Rn = np.sqrt(s4.u) * Rt4[i]
771     Cn, _ = s4.interp_u(y4[:, i], s4.u)
772     ax[0].plot(Rn * 100, Cn, lw=1.6, color=shades[k],
773              label="%.0f h (R=%.3f)" % (tt, Rt4[i] * 100))
774     ax[0].plot([Rn[-1] * 100], [Cn[-1]], "o", ms=5.5, color=shades[k],
775              mec="white", mew=0.7, zorder=5)
776     ends[tt] = float(Rn[-1] * 100) # 该时刻的表面半径 [cm]
777 r6, r48 = ends[6], ends[48]
778 # 收缩的视觉证据: 把 6 h 与 48 h 的两个表面半径用一条双向箭头连起来。
779 # 版面: 纵轴上方专门留一条空带放结论文字与图例, 双向箭头放在 6 h 与 12 h 两条
780 # 剖面之间的空白带 (实测该带内无数据), 三者的位置都按"不与任何曲线相交"挑的。
781 ax[0].set_ylim(-0.05, 2.45)
782 # 结论文字分三行并左对齐: 单行时它有 30 多个字符、宽度超过绘图区, 左端超出坐标框
783 # 0.36 in (印后 26 pt)、压到 y 轴刻度上; 右对齐拆两行后, 第一行宽 2.10 in 仍伸出
784 # 右上角的图例框 (实测重叠 0.077 × 0.133 in)。三行后最长一行只有约 1.1 in。
785 ax[0].text(0.02, 2.38,
786          "收缩: 表面半径  $n\%.3f \to .3f \text{ cm}$  (左移  $\$.3f \text{ cm}$ ,  $\$.1f \%$ )"
787          % (r6, r48, r6 - r48, (r6 - r48) / r6 * 100),
788          fontsize=9.5, color="#08306b", ha="left", va="top", linespacing=1.35)

```

```

789 ybar = 0.95
790 ax[0].annotate("", xy=(r48, ybar), xytext=(r6, ybar),
791               arrowprops=dict(arrowstyle="<|->", color="#08306b", lw=1.5,
792                               mutation_scale=11, shrinkA=0, shrinkB=0))
793 ax[0].text((r6 + r48) / 2, ybar + 0.05, "$\\Delta R$", fontsize=9.5,
794           color="#08306b", ha="center", va="bottom")
795 ax[0].set_xlabel("当前物理距离 $r$ / cm"); ax[0].set_ylabel("C / (kg/kg)")
796 ax[0].set_title("(a) 物理坐标下的水分剖面 (随收缩移动)")
797 ax[0].legend(fontsize=7.5, loc="upper right", bbox_to_anchor=(1.0, 0.92),
798             title="干燥时间 (当时半径)", title_fontsize=7.5)
799 _glow(ax[1], t4 / 3600, Rt4 * 100, COL_MOIST, lw=1.9)
800 # 实测半径按 0.001 cm 步进下降, 20 h 之后基本落定; 这一段是问题 4 的几何前提
801 ax[1].axvspan(20, t4[-1] / 3600, color=COL_PHASE, alpha=0.075)
802 ax[1].text((20 + t4[-1] / 3600) / 2, Rt4.min() * 100 + 0.035,
803           "20 h 后收缩基本停止", fontsize=9.5, color=COL_PHASE, ha="center")
804 ax[1].set_xlabel("时间 / h"); ax[1].set_ylabel("$R(t)$ / cm")
805 ax[1].set_title("(b) 收缩中的表面半径")
806 save(fig, "fig18_p4_physics.png")
807
808 # ----- fig19: 物质坐标下的剖面与论文表 6 的采样点 -----
809 # (a) 横轴 物质坐标 $u=xi^2$ (域固定为 0~1)、纵轴 C/(kg/kg): 画 6、12、24、48 h 四条
810 # 剖面, 与 fig18(a) 是同一批数据的两种坐标画法;
811 # (b) 横轴 时间/h、纵轴 C/(kg/kg): 取 $r=0.5、1.0、1.5$ cm 四个物理距离在四个时刻的
812 # 浓度值, 即论文表 6 的采样; $r > R(t)$ 处该位置不存在, 不存点故曲线留空。
813 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
814 for tt in [6, 12, 24, 48]:
815     i = int(np.argmax(np.abs(t4 - tt * 3600)))
816     Rn = np.sqrt(s4.u) * Rt4[i]
817     Cn, _ = s4.interp_u(y4[:, i], s4.u)
818     ax[0].plot(s4.u, Cn, lw=1.5, label="%.0f h" % tt)
819 ax[0].set_xlabel("物质坐标 $u=\\xi^2$"); ax[0].set_ylabel("C / (kg/kg)")
820 ax[0].set_title("(a) 物质坐标下的剖面 (域固定)")
821 ax[0].legend(fontsize=8)
822 # 表 6 采样
823 rcs = np.array([0.0, 0.005, 0.010, 0.015])
824 vals = np.full((len(rcs), 4), np.nan)
825 for tt in [6, 12, 24, 48]:
826     i = int(np.argmax(np.abs(t4 - tt * 3600)))
827     for j, r in enumerate(rcs):
828         if r <= Rt4[i]:
829             v, _ = s4.interp_u(y4[:, i], (r / Rt4[i]) ** 2)
830             vals[j, [6, 12, 24, 48].index(tt)] = v[0]
831 for j, r in enumerate(rcs):
832     ax[1].plot([6, 12, 24, 48], vals[j], "o-", ms=4, lw=1.4,
833               label="r=%.1f cm" % (r * 100))
834 ax[1].set_xlabel("时间 / h"); ax[1].set_ylabel("C / (kg/kg)")
835 ax[1].set_title("(b) 表 6 各物理距离处的干燥曲线"); ax[1].legend(fontsize=8)
836 save(fig, "fig19_p4_profiles.png")
837
838 print("[fig20] 问题3 与问题4 对比 (备用图, 论文未采用)")
839 # ----- fig20 (备用): 问题 3 与问题 4 的净差异 -----
840 # (a) 横轴 时间/h、纵轴 中心水分浓度 C/(kg/kg) 半对数: 实线是问题 3, 虚线是问题 4,
841 # 红线为判据 0.15, 两条竖点线分别是 td3 (蓝) 与 td4 (橙);
842 # (b) 纵轴 $t_{dry}/h$, 两根柱对比两问的烘干时长, 柱顶标出小时数。
843
844 # 该图把问题 3 与问题 4 直接相减, 而两者的物性关系与几何同时改变,
845 # 其差值不是“纯收缩效应”; 论文正式使用 fig20_p4_controlled.png 的受控对照。
846 # 此处标题只陈述“净差异”, 不做因果表述。
847 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
848 ax[0].semilogy(t3 / 3600, C3[:, 0], lw=1.8, label="问题3 (附录3物性、不收缩)")
849 ax[0].semilogy(t4 / 3600, C4[:, 0], lw=1.8, ls="--", label="问题4 (附录4物性、含收缩)")
850 ax[0].axhline(CRIT, color="r", ls=":", lw=1.2)
851 ax[0].axvline(td3 / 3600, color="C0", ls=":", lw=1.2)
852 ax[0].axvline(td4 / 3600, color="C1", ls=":", lw=1.2)
853 ax[0].set_xlabel("时间 / h"); ax[0].set_ylabel("中心水分浓度 / (kg/kg)")
854 ax[0].set_title("(a) 物性与几何同时改变后的净差异 %.1f%%"
855               % ((td3 - td4) / td3 * 100)); ax[0].legend(fontsize=8)
856 ax[1].bar(["问题3\\equiv 2$ cm", "问题4\\equiv 2$ cm"],
857           [td3 / 3600, td4 / 3600], color=["C0", "C1"])
858 for i, v in enumerate([td3 / 3600, td4 / 3600]):
859     ax[1].text(i, v + 0.6, "%.2f h" % v, ha="center", fontsize=10)
860 ax[1].set_ylabel("$t_{\\mathrm{dry}}$ / h"); ax[1].set_ylim(0, 65)
861 ax[1].set_title("(b) 烘干时长对比 (净差异, 非纯收缩)")
862 save(fig, "fig20_p4_vs_p3.png")
863
864 # ----- 第 7 组: 三层验证、收敛性与误差分析 (fig21~fig25) -----
865 print("[fig21] 三族离散互证")
866 # ----- fig21: 谱配置法 / 有限体积 / 闭式解三族离散互证 -----
867 # 前两族都在 $t=1800$ s 报出表面水分浓度 $C(R_0, 1800)$ [kg/kg], 闭式解仍是基准。
868 # (a) 横轴 四组均匀网格 FV 的配置 ($N=200/200/400/400$, $dt=1.0/0.5/0.5/0.25$ s)、纵轴
869 # $C(R_0, 1800)/(kg/kg)$: 四种网格都落在红虚线 (谱配置法的 1.510918) 上;
870 # (b) 横轴 与闭式解析解的最大偏差/K (对数)、纵轴 三条方法: 闭式解按定义为基准,

```

```

870 # 谱配置法 1.1e-10、均匀网格 FV 2.6e-5。
871 from fv_uniform import solve_fv, sample_xi
872 lab, vals_spec, vals_fv = [], [], []
873 xis = []; fvv = []
874 for Nn, dtv in [(200, 1.0), (200, 0.5), (400, 0.5), (400, 0.25)]:
875     xi, Ctab, Ttab = solve_fv(Nn, dtv, 1800.0, prob=1, t_report=[1800.0])
876     fvv.append(float(np.atleast_1d(sample_xi(xi, Ctab[0], 1.0))[0]))
877     lab.append("$N$=%d\\Delta t$=%.2f" % (Nn, dtv))
878 # 画布与布局: (b) 的纵轴刻度是两行中文方法名 ("均匀网格/守恒型 FV"、"闭式解析解/
879 # (问题1 温度)", 默认间距下会压进 (a) 的绘图区 (实测 0.153 in), 故用 constrained
880 # 布局自动按最宽的刻度留位; 画布宽 8.2 in 按"存出 PNG 宽 ÷ 版心宽"反推, 印后约 8 pt。
881 fig = plt.figure(figsize=(8.2, 3.05), layout="constrained")
882 ax = fig.subplots(1, 2)
883 SPEC = 1.5109181 # 本文谱配置法在该点的值 (与表 7 同口径)
884 ax[0].plot(range(len(fvv)), fvv, "o-", lw=1.5, label="均匀网格守恒型 FV")
885 ax[0].axhline(SPEC, color="r", ls="--", lw=1.6, label="本文谱配置法 1.510918")
886 # 刻度原来是 45° 倾斜的长串 ("N=200,dt=1.00"), 是全篇最挤的一处; 拆成两行短标签
887 ax[0].set_xticks(range(len(lab))); ax[0].set_xticklabels(lab, rotation=0, fontsize=8.5)
888 ax[0].set_ylabel("$C(R_0,1800)$ / (kg/kg)")
889 ax[0].set_title("(a) 两族离散收敛到同一值"); ax[0].legend(fontsize=8)
890 ax[0].set_ylim(1.51085, 1.51115)
891 # 把结论量直接写在图上: 四组网格与谱解的最大相对差, 用当前 fvv 现算, 不写死。
892 # 用"相对差"而不是绝对极差: 绝对极差 7.9e-5 kg/kg 单看数字会比实际收敛质量显得差。
893 rel = max(abs(np.array(fvv) - SPEC)) / SPEC
894 ax[0].annotate("四组网格与谱解\\n最大相对差 $%.1f\\times 10^{-5}$" % (rel * 1e5),
895               xy=(0.02, 0.97), xycoords="axes fraction", fontsize=9.5,
896               ha="left", va="top", color="#08306b",
897               bbox=dict(boxstyle="round,pad=0.28", fc="white", ec="#08306b", lw=0.8))
898 # 闭式解析解是"偏差"的基准, 其偏差恰为 0; 对数轴上 0 高的柱不可见,
899 # 故改用水平柱 + 一个下限刻度, 并把"0 (基准)"直接写在图上。
900 FLOOR = 1e-12
901 names = ["闭式解析解\\n(问题1 温度)", "谱配置法", "均匀网格\\n守恒型 FV"]
902 errs = [FLOOR, 1.1e-10, 2.6e-5]
903 ax[1].barh(names, errs, color=["0.7", "C0", "C1"], height=0.55)
904 ax[1].set_xscale("log"); ax[1].set_xlim(FLOOR / 2, 1e-3)
905 ax[1].set_xlabel("与闭式解的最大偏差 / K")
906 ax[1].set_title("(b) 三族方法的精度量级")
907 # 纵轴上限抬到 2.7: 数值标签改为"条右端上方 6 pt", 需要这 0.15 的顶部余量。
908 ax[1].set_ylim(-0.6, 2.7)
909 ax[1].tick_params(axis="y", pad=5.0) # 方法名与色条左端之间留出约 5 pt
910 # 数值标签原来贴在色条右端外侧: 对数轴上 1.3 倍的位置偏移折算下来离条端只有约
911 # 1.6 pt (看上去与色条粘在一起), 而最长的 $2.6\\times 10^{-5}$ 还超出坐标框 0.18 in。
912 # 改成"居中于条右端上方 7 pt": 标签不再出框 (窄的谱配置法那条左端距框也还有余量),
913 # 文字中心正对条端, 数值与条长的对应关系比外侧更直接。
914 ax[1].text(FLOOR * 1.6, 0, "0 (定义基准)", va="center", fontsize=9.5, color="0.35")
915 for yy, v, s in [(1, 1.1e-10, "$1.1\\times 10^{-10}$"), (2, 2.6e-5, "$2.6\\times 10^{-5}$")]:
916     ax[1].annotate(s, xy=(v, yy), xytext=(0, 7), textcoords="offset points",
917                   ha="center", va="bottom", fontsize=9.5)
918 save(fig, "fig21_three_methods.png")
919
920 print("[fig22] 收敛性 (数据来自 中间数据/verification.json, 不写死)")
921 # ----- fig22: 空间与时间收敛性 (数据取自 中间数据/verification.json) -----
922 # L 取最细网格的 t_dry 当作连续极限; 经典 FV 的 O(dt) 时间误差由 VD.time_slope() 的
923 # 斜率乘 dt 扣掉, 不扣的话空间阶会被时间误差盖住。
924 # (a) 横轴 节点数 N、纵轴 L-t_dry/s (双对数): 三角是谱解、方块是经典 FV+theta, 另有
925 # 斜率 -2 的参考线; (b) 横轴 时间步 dt/s (坐标轴反向)、纵轴 L-t_dry/s: 一阶外推极限
926 # L_fv 由 t(dt)=L-c*dt 对最细三档做最小二乘得到, 黑虚线是斜率 1 的参考线。
927 vd = VD.load()
928 VD.check_consistency(vd, tdry_cached=td3)
929 Ns = list(vd["space"]["N"])
930 tds = np.asarray(vd["space"]["tdry"], dtype=float)
931 Nf = list(vd["fv_space"]["N"])
932 tdf = np.asarray(vd["fv_space"]["tdry"], dtype=float)
933 c_fv = VD.time_slope(vd) # 该格式的 O(dt) 系数, 用于扣时间误差
934 dt_fv = vd["fv_space"]["dt"]
935 LIM = float(tds.max()) # 谱解的连续极限 (取最细网格值)
936 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
937 ax[0].loglog(Ns, np.maximum(LIM - tds, 0.01), "--", lw=1.7, ms=6,
938             label="本文谱配置法")
939 ax[0].loglog(Nf, LIM - (tdf + c_fv * dt_fv), "s-", lw=1.7, ms=6,
940             label="经典 FV+$\\theta$ 格式 (扣去时间误差 %.2f s)" % (c_fv * dt_fv))
941 ax[0].loglog([Nf[0], Nf[-1]],
942             (LIM - (tdf[-1] + c_fv * dt_fv)) * (np.array([Nf[0], Nf[-1]],
943                 dtype=float) / Nf[-1]) ** -2,
944             "k--", lw=1.1, label="斜率 $-2$")
945 ax[0].set_xlabel("节点数 $N$"); ax[0].set_ylabel("$L-t_{\\mathrm{dry}}$ / s")
946 ax[0].set_title("(a) 空间收敛: 谱解远快于二阶")
947 # 谱解的最后一点已压到时间误差平台 (纵轴下端), 默认下半轴没有空位,
948 # 图例放左下角会盖住该点; 把纵轴下限再放低约一个半数量级腾出空白。
949 ax[0].set_ylim(bottom=3e-4)
950 ax[0].legend(fontsize=8, loc="lower left")

```

```

951 # 把结论量直接写在图上：谱解最后一点已落进时间误差平台，而经典 FV 还差三个数量级。
952 # 数值取 (a) 里谱解曲线真正画出的末点值——注意不能直接用 LIM - tds[-1]：LIM 就是取
953 # tds 的最大值，那一项恒为 0，而 y=0 在对数轴上不可表示，标注会整条画不出来（实测）。
954 plateau = float(np.maximum(LIM - tds, 0.01)[-1])
955 ax[0].annotate("谱解误差已落到\n%.1f\textimes 10^{-2} s 的平台" % (plateau * 1e2),
956               xy=(0.97, 0.96), xycoords="axes fraction", fontsize=9.5,
957               ha="right", va="top", color="#08306b",
958               bbox=dict(boxstyle="round,pad=0.28", fc="white", ec="#08306b", lw=0.8))
959 dtl = np.asarray(vd["time"][:, "dt"], dtype=float)
960 tdl = np.asarray(vd["time"][:, "tdry"], dtype=float)
961 # 一阶外推极限：由 t(dt)=L-c*dt 的最小二乘截距给出，而不是写死的常数
962 sl, L_fv = np.polyfit(dtl[-3:], tdl[-3:], 1)
963 print(" 经典 FV+theta 外推极限 L=%.3f s, c=%.4f s/s" % (L_fv, -sl))
964 ax[1].loglog(dtl, L_fv - tdl, "o-", lw=1.7, ms=6, label="经典 FV+\theta$ 格式")
965 ax[1].loglog(dtl, (L_fv - tdl[-1]) * (dtl / dtl[-1]), "k--", lw=1.1, label="斜率 1")
966 ax[1].set_xlabel("时间步 $\Delta t$ / s"); ax[1].set_ylabel("$L-t_{\mathrm{dry}}$ / s")
967 ax[1].set_title("(b) 时间收敛：严格一阶")
968 ax[1].invert_xaxis()
969 # 实测斜率直接标在图上（与 -2 参考线对照）。单位必须写成 s/s：这个 1.42 是
970 # t(dt)=L-c*dt 的斜率 c，而正文另有一个“相邻增量-步长双对数斜率 1.01”（收敛阶），
971 # 两个数不是一回事，标上单位才不会让人以为图上写错了。
972 ax[1].annotate("最小二乘斜率 $c=%.4f$ s/s" % (-sl), xy=(0.03, 0.96),
973               xycoords="axes fraction", fontsize=9.5, ha="left", va="top",
974               color="#08306b",
975               bbox=dict(boxstyle="round,pad=0.28", fc="white", ec="#08306b", lw=0.8))
976 ax[1].legend(fontsize=8, loc="lower left")
977 save(fig, "fig22_convergence.png")
978
979 print("[fig23] 积分器与容差稳定性")
980 # ----- fig23: 求解器与容差配置的稳定性 -----
981 # 数据取自 verification.json 的 integ 组，共 6 种配置 (BDF 1e-9/1e-10/1e-12、Radau
982 # 1e-11、LSODA 1e-10 以及 1 s 密采样) 各自报出的 t_dry [s]。
983 # (a) 横轴 六种配置、纵轴 t_dry/s，数值直接标在点上；(b) 横轴 t_dry 相对均值的偏差/s、
984 # 纵轴 频数。两幅标题都只报极差。
985 vvv = [r["tdry"] for r in vd["integ"]]
986 cfigs = ["BDF\n1e-9", "BDF\n1e-10", "BDF\n1e-12", "Radau\n1e-11", "LSODA\n1e-10", "1s密采样"]
987 spread = max(vvv) - min(vvv)
988 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
989 ax[0].plot(range(len(vvv)), vvv, "o-", lw=1.5, ms=7)
990 ax[0].set_xticks(range(len(cfigs))); ax[0].set_xticklabels(cfigs, fontsize=8)
991 ax[0].set_ylabel("$t_{\mathrm{dry}}$ / s")
992 ax[0].set_title("(a) 六种积分器/容差配置：极差 %.3f s" % spread)
993 ax[0].set_ylim(min(vvv) - 0.02, max(vvv) + 0.02)
994 for i, v in enumerate(vvv):
995     # 首末标签靠内对齐，否则会压到 y 轴刻度数字上；
996     # 相邻两点数值只差 0.0001 s，标签上下错开以免互相叠字。
997     ha = "left" if i == 0 else ("right" if i == len(vvv) - 1 else "center")
998     ax[0].text(i, v + (0.004 if i % 2 == 0 else 0.009), "%.3f" % v,
999               ha=ha, fontsize=6.0)
1000 ax[0].margins(x=0.07)
1001 ax[1].hist(np.array(vvv) - np.mean(vvv), bins=8, color="C0", alpha=.8)
1002 ax[1].set_xlabel("$t_{\mathrm{dry}}$ 相对均值的偏差 / s"); ax[1].set_ylabel("频数")
1003 # 注意：极差 0.007 s 不等于“各点落在均值 ±0.004 s 内”——6 个配置里 5 个聚在
1004 # 一侧、1 个偏离，相对均值的最大偏差可达 0.0056 s。此处只陈述极差。
1005 ax[1].set_title("(b) 相对均值的偏差：极差 %.3f s" % spread)
1006 save(fig, "fig23_integrator.png")
1007
1008 print("[fig24] 谱解-闭式解残差统计")
1009 # ----- fig24: 谱解与闭式解偏差的统计特征 -----
1010 # 残差 res 沿用 fig11 里算出的“谱解 - 闭式解”（单位 K），这里放大 1e10 后统计。
1011 # (a) 横轴 残差/(1e-10 K)、纵轴 概率密度：柱状为直方图，红实线为同均值同标准差的
1012 # 正态密度；(b) Q-Q 图，横轴 理论分位、纵轴 样本分位。
1013 res2 = res * 1e10
1014 fig, ax = plt.subplots(1, 2, figsize=(9.6, 3.4))
1015 ax[0].hist(res2, bins=26, color="C0", alpha=.8, density=True)
1016 xs = np.linspace(res2.min(), res2.max(), 200)
1017 from scipy import stats as st
1018 ax[0].plot(xs, st.norm.pdf(xs, res2.mean(), res2.std()), "r-", lw=1.8)
1019 ax[0].set_xlabel("残差 / ($10^{-10}$ K)"); ax[0].set_ylabel("概率密度")
1020 ax[0].set_title("(a) 谱解与闭式解偏差的分布")
1021 st.probplot(res2, dist="norm", plot=ax[1])
1022 ax[1].set_xlabel("理论分位"); ax[1].set_ylabel("样本分位")
1023 # 残差由时间积分的系统性偏差主导（201 个点同号、Shapiro-Wilk p=1e-12），
1024 # 不应表述为“与正态无偏离”；此处只描述其确定性特征。
1025 ax[1].set_title("(b) Q-Q 图：确定性离散误差，非随机噪声")
1026 ax[1].get_lines()[0].set_marker("."); ax[1].get_lines()[0].set_markersize(5)
1027 ax[1].get_lines()[1].set_color("r")
1028 save(fig, "fig24_residual_qc.png")
1029
1030 print("[fig25] 方法流程图")
1031 # ----- fig25: 技术路线流程图 -----

```

```

1032 # 本图没有数据来源, 纯手工排版的方框 + 箭头, 坐标都用 axes fraction 给出, 与任何
1033 # 物理量无关。方框自上而下是: 题目条件 + 附件 1/2 数据 → 几何简化 → 控制方程 →
1034 # 环境标定 / 坐标正则化 / 物性经验式 → 问题 1-4 的求解路线 → 三层验证 →
1035 # 灵敏度与结论。
1036 # 底色承载语义 (三类含义固定的浅底, 与全篇语义色一致; 同一类节点在任何一版里同色):
1037
1038 # #cfe2f3 蓝 = 数据、模型与求解流程 (主干)
1039 # #9dc3e6 深蓝 = 两个关键节点 (控制方程、三层验证), 边框加粗以形成视觉层级
1040 # #fdf1cf 米黄 = 三项建模准备 (中间产物, 对应参考图里的黄色强调框)
1041 # 三处必须守住的版面底线:
1042 # (1)"最后粉色的框框下面的黑线没有了"——原代码最后一个框中心 y=0.04、高 0.11,
1043 # 底边落在 y=-0.015 已被坐标轴裁掉 (Rectangle 默认 clip_on=True)。新版把整列
1044
1045 # 重排成 7 行 (0.940 / 0.812 / 0.684 / 0.535 / 0.370 / 0.212 / 0.062),
1046 # 最低一行的底边在 y=0.018, 四周都留出余量, 不会再被裁。
1047 # (2)"这个箭头必须有 - 再箭头"——原相邻框间距只有 0.03 axes fraction (约 0.16 in),
1048 # 箭头只看得见一个三角。新版行距放到 0.04-0.077, 箭头 lw=1.6, mutation_scale=13,
1049 # 杆与头都清楚可见。
1050 # (3)"不要怕占地方、要圆角"——直角矩形换 FancyBboxPatch 圆角框, 方框文字 9.5-10.5 pt。
1051
1052 # 画布宽仍是 9.6 in (不加大): 插图按 0.92\textwidth 排版, 画布一加宽缩放系数就变小,
1053 # 印后字号反而更小; 要"字更大"只能把字号做大于画布。画布只加高到 5.9 in, 给行距让位。
1054 fig, ax = plt.subplots(figsize=(9.6, 5.9))
1055 ax.axis("off")
1056 BLUE, BLUE_D, YELLOW = "#cfe2f3", "#9dc3e6", "#fdf1cf"
1057 EDGE, ACCENT = "#2e75b6", "#bf9000"
1058 ROUND = "round,pad=0,rounding_size=0.012"
1059 rows = [
1060     (0.5, 0.940, 0.76, "题目条件 + 附件 1/2 实测数据", BLUE, EDGE, 1.6),
1061     (0.5, 0.812, 0.76, "几何简化: $L/R_0=12.5$ \\ \\Rightarrow$ 一维轴对称", BLUE, EDGE, 1.6),
1062     (0.5, 0.684, 0.76, "控制方程: 热传导 + Fick 扩散 + Robin 边界", BLUE_D, EDGE, 2.2),
1063     (0.185, 0.535, 0.26, "环境标定\\n一阶惯性拟合", YELLOW, ACCENT, 1.4),
1064     (0.500, 0.535, 0.26, "坐标正则化\\n$u=(r/R)^2$", YELLOW, ACCENT, 1.4),
1065     (0.815, 0.535, 0.26, "物性经验式\\n附录 2/3/4", YELLOW, ACCENT, 1.4),
1066     (0.185, 0.370, 0.26, "问题 1\\n解耦 + 闭式解", BLUE, EDGE, 1.6),
1067     (0.500, 0.370, 0.26, "问题 2\\n变物性耦合", BLUE, EDGE, 1.6),
1068     (0.815, 0.370, 0.26, "问题 3/4\\n事件求交 + 收缩", BLUE, EDGE, 1.6),
1069     (0.5, 0.212, 0.76, "三层验证: 闭式解析解 / 谱配置 / 两族有限体积",
1070      BLUE_D, EDGE, 2.2),
1071     (0.5, 0.062, 0.76, "灵敏度与误差分析 $\\Rightarrow$ 结论与工艺建议", BLUE, EDGE, 1.6),
1072 ]
1073 BH = 0.088 # 方框高度 (axes fraction), 全列统一
1074 for (x, yv, w, txt, fc, ec, lw) in rows:
1075     ax.add_patch(FancyBboxPatch((x - w / 2, yv - BH / 2), w, BH,
1076                                boxstyle=ROUND, facecolor=fc, edgecolor=ec,
1077                                lw=lw, transform=ax.transAxes, zorder=2))
1078     ax.text(x, yv, txt, ha="center", va="center", fontsize=10.5, linespacing=1.35,
1079            transform=ax.transAxes, zorder=3)
1080 # 三个"问题"框用虚线分组框围起来, 与参考图的分组画法一致 (纯装饰, 不含数值);
1081 # 标签放在分组框左下外, 避开所有箭头 (最左的收敛箭头从 x=0.185 才起步)
1082 ax.add_patch(FancyBboxPatch((0.045, 0.370 - BH / 2 - 0.030), 0.915, BH + 0.060,
1083                              boxstyle=ROUND, facecolor="none", edgecolor="0.55",
1084                              lw=1.1, ls=(0, (5, 3)), transform=ax.transAxes, zorder=1))
1085 ax.text(0.050, 0.370 - BH / 2 - 0.038, "四问求解路线", fontsize=9.5, color="0.40",
1086        ha="left", va="top", transform=ax.transAxes, zorder=1)
1087 arr = dict(arrowstyle="-|>", color=ACCENT, lw=1.6, shrinkA=0, shrinkB=0,
1088            mutation_scale=13)
1089 # 箭头一律从上一行的底边画到下一行的顶边, 故杆长就是行距, 肉眼可见
1090 A = [(0.5, 0.896, 0.5, 0.856), (0.5, 0.768, 0.5, 0.728),
1091      (0.5, 0.640, 0.185, 0.579), (0.5, 0.640, 0.5, 0.579), (0.5, 0.640, 0.815, 0.579),
1092      (0.185, 0.491, 0.185, 0.414), (0.5, 0.491, 0.5, 0.414), (0.815, 0.491, 0.815, 0.414),
1093      (0.185, 0.326, 0.5, 0.256), (0.5, 0.326, 0.5, 0.256), (0.815, 0.326, 0.5, 0.256),
1094      (0.5, 0.168, 0.5, 0.106)]
1095 for (x1, y1_, x2, y2_) in A:
1096     ax.annotate("", xy=(x2, y2_), xytext=(x1, y1_),
1097                arrowprops=arr, xycoords="axes fraction")
1098 ax.set_title("本文技术路线", fontsize=13.5, pad=6)
1099 save(fig, "fig25_flow.png")
1100
1101 print("[fig26] 物理模型与坐标系示意图")
1102 # ----- fig26: 物理模型与坐标系示意图 (无数据, 纯示意) -----
1103 # 全篇此前没有一张物理示意图——翻到第五章会想知道"这个问题被想象成什么形状、
1104 # 边界加在哪里、收缩体现在哪个量上", 原先只能靠读文字。本图把这栋建模主线画出来:
1105 # (1)药材横截面 (热风对流 + 表面蒸发 + 收缩后的 R(t)): (2)取 1/4 截面对称化, 得到
1106 # 一维径向坐标 r 属于 [0, R(t)]; (3)控制方程与 Robin 边界写在右侧。
1107 # 本图不含任何数值计算, 画的都是模型假设本身; 图上虚线圆用的 0.60 是附件 2 实测
1108
1109 # 收缩的终值比例 (R 由 2.00 cm 降到 1.20 cm), 不是拟合或外推出来的。
1110 # 坐标一律用 data 单位并 set_aspect("equal"): axes fraction 下图会被拉成椭圆。
1111 # 版面纪律: 三块内容各自留出"文字带", 说明文字一律放在图形外, 不与图元交叠。
1112 fig, ax = plt.subplots(figsize=(8.2, 3.45))

```

```

1113 ax.set_xlim(0, 12.6); ax.set_ylim(0, 5.30)
1114 ax.set_aspect("equal"); ax.axis("off")
1115
1116 # ---- (1) 药材横截面: 热风、蒸发、收缩 ----
1117 # 外圆由 1.05 放大到 1.15: 内圆半径固定为 0.60 R (附件 2 实测收缩终值比例, 不能改),
1118 # 原尺寸下内圆直径只有 47.8 pt, 而圆内" (湿物料) "一行在 9.5 pt 下就要 47.5 pt,
1119 # 两端正好压在红虚线圆上。放大外圆并把该行降到 9 pt (印后 8.5 pt, 仍过 7 pt 线) 后,
1120 # 文字两端各留约 3.6 pt: 再加一层白色描边, 即使与虚线相切也读得清。
1121 cx, cy, Ro = 1.85, 3.20, 1.15
1122 Ri = Ro * 0.60 # R(t_end)/R_0 = 1.20/2.00, 取自附件 2 实测收缩
1123 ax.add_patch(plt.Circle((cx, cy), Ro, facecolor="#eaf3fb", edgecolor="#2e75b6",
1124 lw=1.8, zorder=2))
1125 ax.add_patch(plt.Circle((cx, cy), Ri, facecolor="none", edgecolor="#c0392b",
1126 lw=1.6, ls=(0, (4, 3)), zorder=3))
1127 ax.text(cx, cy, "药材\n (湿物料)", ha="center", va="center", fontsize=9.0, zorder=4,
1128 path_effects=[pe.withStroke(linewidth=2.0, foreground="white")])
1129 # 热风: 从左斜向吹来, 箭头止于表面
1130 for (dx, dy) in [(-1.60, 0.75), (-1.75, -0.10), (-1.30, -0.90)]:
1131     s = np.hypot(dx, dy)
1132     ax.annotate("", xy=(cx + dx / s * Ro, cy + dy / s * Ro),
1133 xytext=(cx + dx, cy + dy),
1134 arrowprops=dict(arrowstyle="->", color=COL_TEMP, lw=1.8,
1135 mutation_scale=13, shrinkA=0, shrinkB=0))
1136 # 蒸发: 从表面向外
1137 for ang in (-30, 5, 45):
1138     a = np.deg2rad(ang)
1139     ax.annotate("", xy=(cx + np.cos(a) * (Ro + 0.55), cy + np.sin(a) * (Ro + 0.55)),
1140 xytext=(cx + np.cos(a) * Ro, cy + np.sin(a) * Ro),
1141 arrowprops=dict(arrowstyle="->", color=COL_MOIST, lw=1.6,
1142 mutation_scale=12, shrinkA=0, shrinkB=0))
1143 ax.text(0.10, 5.15, "热风:  $T_{\text{air}}(t), C_{\text{air}}(t)$ ",
1144 ha="left", va="top", fontsize=9.5, color=COL_TEMP)
1145 ax.text(4.60, 5.15, "表面蒸发\n (Robin 边界)", ha="right", va="top",
1146 fontsize=9.5, color=COL_MOIST)
1147 # 收缩: 外圆 → 内虚线圆, 箭头画在右下 45° (该方向既无热风入射也无蒸发射出)
1148 ax.annotate("", xy=(cx + Ri * 0.70, cy - Ri * 0.70),
1149 xytext=(cx + Ro * 0.70, cy - Ro * 0.70),
1150 arrowprops=dict(arrowstyle="->", color="#c0392b", lw=1.6,
1151 mutation_scale=12, shrinkA=0, shrinkB=0))
1152 ax.text(cx, 1.82, " $R(t)$ : 表面半径随含水率收缩", ha="center", va="top",
1153 fontsize=9.5, color="#c0392b")
1154 ax.text(cx, 1.34, "(a) 圆柱药材横截面 (长  $L=25$  cm)", ha="center", va="top",
1155 fontsize=9.5, color="0.25")
1156
1157 # ---- (2) 取 1/4 截面对称化 → 一维径向 ----
1158 # 圆心高度与 (a) 对齐 (cy=3.20), 两块示意图才在同一水平线上
1159 mx, my, Rm = 5.95, 3.20, 1.15
1160 ax.add_patch(plt.Circle((mx, my), Rm, facecolor="none", edgecolor="0.55",
1161 lw=1.3, ls=(0, (4, 3)), zorder=2))
1162 # 两条对称轴用点线画出: 1/4 截面就是靠它们切出来的
1163 ax.plot([mx, mx], [my, my + Rm], ls=":", lw=1.0, color="0.6")
1164 ax.plot([mx, mx + Rm], [my, my], ls=":", lw=1.0, color="0.6")
1165 ax.annotate("", xy=(mx + Rm, my), xytext=(mx, my),
1166 arrowprops=dict(arrowstyle="->", color="k", lw=1.8,
1167 mutation_scale=13, shrinkA=0, shrinkB=0))
1168 ax.plot([mx], [my], "o", ms=5, color="k", zorder=4)
1169 ax.text(mx + Rm + 0.10, my + 0.12, " $r=R(t)$ ", fontsize=9.5, va="bottom")
1170 ax.text(mx + Rm * 0.45, my - 0.16, " $r$ ", fontsize=11, ha="center", va="top")
1171 ax.text(mx, 1.82, " $r=0$ : 对称 ( $\partial_r T = \partial_r C = 0$ )", ha="center",
1172 va="top", fontsize=9.5, color="0.35")
1173 ax.text(mx, 1.34, "(b) 化为轴对称一维径向问题", ha="center", va="top",
1174 fontsize=9.5, color="0.25")
1175
1176 # ---- (3) 控制方程与边界条件 ----
1177 ax.add_patch(FancyBboxPatch((7.75, 0.30), 4.70, 4.70,
1178 boxstyle="round,pad=0,rounding_size=0.16",
1179 facecolor="#f4f8fc", edgecolor="#2e75b6", lw=1.5))
1180 ex = 10.10
1181 ax.text(ex, 4.74, "(c) 控制方程与边界条件", ha="center", va="center",
1182 fontsize=9.5, color="0.25")
1183 # 每一节"小标题在上、公式在下", 左右都不加标签, 避免长公式越过小标题的左边界
1184 eqs = [
1185     ("表观体积热容模型",
1186      r" $\rho_v(C) \partial_t T = \partial_r (r \partial_r T)$ ",
1187      ("干物质基准守恒",
1188       r" $\rho_d \partial_t C = \partial_r (r \partial_r C)$ ",
1189      ("边界条件",
1190       r" $r=0$ :  $\partial_r T = \partial_r C = 0$ ",
1191       ("",
1192        r" $r=R(t)$ :  $-k \partial_r T = h (T_{\text{air}} - T_s)$ ",
1193

```

```

1194         r"$-D\,\partial_rC=h_m(C_{\mathrm{air}})-C_s$"),
1195     ]
1196     yy = 4.40
1197     # 公式块整体上移 0.12: 原点 4.28 使最后一行 $-D\,\partial_rC=\dots$ 距框底边只剩
1198     # 2 pt (实测), 紧得像是被裁掉; 上移后首行距 (c) 标题 12 px (约 3.6 pt)、末行距框底
1199     # 净留白约 25 px, 上下都不再贴边。
1200     for (tag, eq) in eqs:
1201         if tag:
1202             ax.text(ex, yy, tag, fontsize=9.5, ha="center", va="center", color="0.30")
1203             yy -= 0.46
1204             ax.text(ex, yy, eq, fontsize=9.5, ha="center", va="center")
1205             yy -= 0.60
1206     save(fig, "fig26_model_schematic.png")
1207
1208     print("全部图完成, 输出目录: ", FIGDIR)
1209     return td3, td4
1210
1211
1212 if __name__ == "__main__":
1213     main()

```

3.11 问题 4 受控对照: make_controlled_comparison.py

```

1  # -*- coding: utf-8 -*-
2  # =====
3  # make_controlled_comparison.py —— 受控对照: 把"物性效应"与"收缩效应"拆开
4  # =====
5  # 做什么: 从问题 3 到问题 4, 物性关系 (附录 3 -> 附录 4) 与几何 (半径固定 -> 实测收缩)
6  #         同时改变, 两者直接相减只能得到"净差异", 无法归因。本脚本补算一个受控算例
7  #         ——物性仍取附录 4, 但半径固定为 R_0 不收缩——于是可以把总差异拆成两段纯效应:
8  #         A 到 B: 只换物性 (附录 3 -> 附录 4), 半径都固定 -> 物性效应
9  #         B 到 C: 只加收缩, 物性都用附录 4 -> 收缩效应
10 #         A 到 C 即两问的净差异, 画在同一张图上便于逐段对照。
11 # 输入: 不读文件。算例由 spectral.HerbSolver 求解 (其几何、环境标定与物性系数取自
12 #        common.py: R0、h、hm、T_air(t)、C_air(t) 与 props() 的附录 4 关系);
13 #        基准值 TD_PROBLEM3、TD_PROBLEM4_SHRINK 是问题 3、4 的烘干时刻记录值。
14 # 输出: figure/fig20_p4_controlled.png 受控对照图, 240 dpi, 左右两面板
15 # 关键变量:
16 #   CRIT 烘干判据阈值 0.15 kg/kg (干基含水率), 与其余脚本口径一致
17 #   n 谱节点数; 正式出图取 64, 另跑一次 48 做网格敏感性对照
18 #   t_dry 由事件求交得到的烘干时刻 [s]
19 # =====
20
21 from pathlib import Path
22
23 import matplotlib
24 import numpy as np
25
26 matplotlib.use("Agg")
27 import matplotlib.pyplot as plt
28
29 from spectral import HerbSolver
30
31
32 # ----- 算例常数与基准值 -----
33 CRIT = 0.15 # kg/kg (干基) 烘干判据阈值
34 TD_PROBLEM3 = 205575.5 # s 问题 3: 附录 3 物性、半径固定不收缩
35 TD_PROBLEM4_SHRINK = 182778.3 # s 问题 4: 附录 4 物性、含实测收缩 R(t)
36
37
38 def fixed_radius_time(n=64):
39     """补算"固定半径 + 附录 4 物性"这个受控算例, 返回烘干时刻 t_dry [s]。
40
41     只把 HerbSolver 的 shrink 置 False (半径恒为 R_0 = 2 cm), 物性仍取 prob=4 的附录 4
42     关系, 于是它和问题 3 的差别就只剩物性一项。事件定义、容差与积分器 (BDF) 都与其他
43     脚本一致, 故算出的差值不是求解设置造成的。
44     """
45     solver = HerbSolver(n, prob=4, shrink=False)
46     # atol 按状态分块: 前 Np 个是水分浓度 C [kg/kg] (量级 1), 后 Np 个是温度 T [degC]
47     # (量级 3e2)。两块量级不同, 同一个绝对容差对两者并不等价。
48     atol = np.concatenate(
49         [np.full(solver.Np, 1.0e-13), np.full(solver.Np, 1.0e-11)]
50     )
51
52     def event(t, y):
53         """事件函数: y 前半段是水分浓度场 [kg/kg], 取其最大值 (干燥最慢的中心) 减判据。"""

```

```

54     return np.max(y[: solver.Np]) - CRIT
55
56     event.terminal = True          # 命中阈值立即停止积分，不必白算到 600000 s
57     event.direction = -1          # 只认由大变小穿越，避免初值恰好等于阈值时误判
58     # 600000 s (约 167 h) 是足够宽的上界，事件必定在此之前触发
59     sol = solver.solve(
60         600000.0, rtol=1.0e-10, atol=atol, events=event, method="BDF"
61     )
62     if not sol.t_events[0].size:
63         raise RuntimeError("Drying threshold was not reached.")
64     return float(sol.t_events[0][0])
65
66
67 def draw(output_path, td_fixed):
68     """画受控对照图并存到 output_path, 返回 None。
69
70     (a) 横轴为 A/B/C 三个算例、纵轴 烘干时长 t_dry/h, 柱顶标出小时数: A = 问题 3
71         (附录 3 物性、不收缩), B = 固定半径 + 附录 4 物性 (本脚本补算, 即入参
72         td_fixed [s]), C = 问题 4 (附录 4 物性、含实测收缩)。
73     (b) 横轴 相对变化/%、纵轴三根水平柱, 柱端标百分数: A 到 B 是纯物性效应、B 到 C 是
74         纯收缩效应、A 到 C 是净差异。
75     配色语义固定 (两面板一致, 便于横向对照): #287271 深青 = A 问题 3 基准、
76     #6C757D 灰 = B 固定半径附录 4、#D95F43 砖红 = C 问题 4 含收缩; (b) 中 A 到 B 用灰、
77     B 到 C 用砖红、A 到 C 用深青。
78     """
79     plt.rcParams.update(
80         {
81             "font.sans-serif": ["Microsoft YaHei", "SimHei", "DengXian"],
82             "axes.unicode_minus": False,
83             "font.size": 14,
84             "axes.titlesize": 15,
85             "axes.labelsize": 14,
86             "xtick.labelsize": 12,
87             "ytick.labelsize": 12,
88             "axes.linewidth": 1.1,
89             "savefig.dpi": 240,
90         }
91     )
92
93     labels = ["A", "B", "C"]          # 代号: A=问题3, B=固定半径+附录4, C=问题4含收缩
94     seconds = np.array([TD_PROBLEM3, td_fixed, TD_PROBLEM4_SHRINK])
95     hours = seconds / 3600.0
96     colors = ["#287271", "#6C757D", "#D95F43"]
97
98     # 画布按 0.86\textwidth = 5.42 in 排版: 取 8.2 in 宽时 14 pt 的正文字号印后只剩
99     # 6.98 pt, 低于 7 pt 的可读下限; 缩到 8.0 in 可到 7.16 pt, 故宽度定为 8.0 in。
100    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(8.0, 3.27))
101
102    bars = ax1.bar(np.arange(3), hours, width=0.62, color=colors, edgecolor="white")
103    ax1.bar_label(bars, labels=[f"{v:.2f} h" for v in hours], padding=4, fontsize=12)
104    ax1.set_xticks(np.arange(3), labels)
105    ax1.set_ylabel("烘干时长 / h")
106    ax1.set_title("(a) 烘干时长")
107    ax1.set_ylim(0, 145)
108    ax1.grid(axis="y", alpha=0.28, linewidth=0.8)
109    ax1.spines[["top", "right"]].set_visible(False)
110
111    # 三段效应的相对变化 [%]: 正号表示烘干时长变长、负号变短, A 到 C 是净差异
112    effects = np.array([125.8, -60.6, -11.1])
113    effect_labels = [
114        "A 到 B 物性",
115        "B 到 C 收缩",
116        "A 到 C 净差",
117    ]
118    y = np.arange(3)
119    effect_colors = [colors[1], colors[2], colors[0]]
120    ebar = ax2.barh(y, effects, height=0.56, color=effect_colors)
121    ax2.axvline(0, color="#333333", linewidth=1.2)
122    ax2.set_yticks(y, effect_labels)
123    ax2.invert_yaxis()          # 反转纵轴, 让 "A 到 B" 一行显示在最上方
124    ax2.set_xlim(-75, 140)
125    ax2.set_xlabel("相对变化 / %")
126    ax2.set_title("(b) 效应分解")
127    ax2.grid(axis="x", alpha=0.28, linewidth=0.8)
128    ax2.spines[["top", "right"]].set_visible(False)
129    for rect, value in zip(ebar, effects):
130        # 柱端百分数一律写在柱内: 正向柱靠右端内对齐, 负向柱靠左端内对齐,
131        # 绝对值小于 30 的短柱若也写白色就会压在色块外, 故改用深灰并移出一小点
132        if value > 30:
133            x, ha, color = value - 5, "right", "white"
134            elif value < -30:

```

```

135         x, ha, color = value + 5, "left", "white"
136     else:
137         x, ha, color = value - 4, "right", "#222222"
138     ax2.text(
139         x,
140         rect.get_y() + rect.get_height() / 2,
141         f"{value:+.1f}%",
142         va="center",
143         ha=ha,
144         color=color,
145         fontsize=12,
146     )
147
148     # 手工设定边距: wspace=0.34 是留给 (b) 三条较长的纵轴标签 ("A 到 B 物性" 等),
149     # 否则它们会顶到 (a) 的柱子上
150     fig.subplots_adjust(left=0.085, right=0.985, top=0.88, bottom=0.19, wspace=0.34)
151     fig.savefig(output_path, bbox_inches="tight", pad_inches=0.06)
152     plt.close(fig)
153
154
155 def main():
156     """跑两档网格的固定半径算例、画出受控对照图, 并把三个数值打到屏幕上。
157
158     两档网格只用来检查空间离散够不够细: rel_gap 应远小于要归因的那几个百分点,
159     出图用 N=64 的结果。
160     """
161     td48 = fixed_radius_time(48)
162     td64 = fixed_radius_time(64)
163     rel_gap = abs(td64 - td48) / td64
164     out = Path(__file__).resolve().parents[1] / "figure" / "fig20_p4_controlled.png"
165     draw(out, td64)
166     print(f"N=48 fixed-R Appendix-4: {td48:.3f} s")
167     print(f"N=64 fixed-R Appendix-4: {td64:.3f} s")
168     print(f"relative grid gap: {rel_gap:.3e}")
169     print(f"figure: {out}")
170
171
172 if __name__ == "__main__":
173     main()

```

3.12 验证数据复跑与一致性校验: verify_data.py

```

1  # -*- coding: utf-8 -*-
2  # =====
3  # verify_data.py —— fig22/fig23 所用验证数据的来源与一致性检查
4  # =====
5  # 做什么: 那两张"收敛性 / 积分器稳定性"图要用四类耗时复跑的结果, 不可能每画一次图就重算;
6  # 把数字写在绘图脚本里又会让图与代码脱钩(代码改了、图还是旧数)。故本模块负责
7
8  # "真实复跑 → 落盘 → 校验": recompute() 产生数据, 绘图脚本只读数、且绘图前必过校验。
9  # 输入: common.py (物性、环境、初值)、spectral.HerbSolver、fv_theta.find_tdry 现场复跑;
10 # 一致性检查另读 中间数据/solutions.npz 的 p3_tdry 做交叉核对。
11 # 输出: <交付根>/中间数据/verification.json (带版本号 DATA_VERSION, 供 fig22/fig23 使用)。
12 # 运行方式与依赖顺序:
13 #     python verify_data.py                只校验数据文件(秒级, run_all.py 与
14
15 #     make_figures.py 内部也是这样调用它的);
16 #     python verify_data.py --recompute 真实复跑四类扫描并重写 json(约 22 min);
17 #     四类扫描耗时: 空间扫描约 2 min、时间扫描约 10 min、经典 FV 对照约 9 min、
18 #     积分器与容差约 1 min。数据文件缺失或版本不符时直接抛错并给出补救命令。
19 # 关键变量:
20 #     SPACE_N      谱节点数扫描 24/32/48/64/96/128 (fig22 的自变量之一)
21
22 #     TIME_DT      定步长 theta 法的步长 40-0.25 (fig22 的另一个自变量) [s]
23 #     FV_N         该步长扫描用的空间网格 1600 (论文表 11 的网格)
24 #     FV_SPACE_N   经典 FV 的空间对照 800/1600/3200/6400, 固定步长 1.0 [s]
25 #     INTEG_CFG    积分器/容差配置 5 组 (BDF×3、Radau、LSODA), 加 1 s 密采样求交
26 #     共 6 个点, 即 fig23 的 6 个刻度
27 #     CRIT         烘干判断阈值 0.15 [kg/kg]
28
29 # =====
30 import json
31 import os
32 import sys
33
34 import numpy as np

```

```

35
36 from fig_core import DATADIR
37
38 DATA_PATH = os.path.join(DATADIR, "verification.json")
39 DATA_VERSION = "2026-09-13a"
40
41 CRIT = 0.15
42 # 空间扫描的网格; 时间扫描的步长 (s)。两者都是论文 fig22 的自变量。
43 SPACE_N = [24, 32, 48, 64, 96, 128]
44 TIME_DT = [40.0, 20.0, 10.0, 5.0, 2.0, 1.0, 0.5, 0.25]
45 SPACE_RTOL = 1e-11 # 与生产设置一致
46 FV_N = 1600 # 定步长 theta 法的空间网格 (论文表 11)
47 FV_SPACE_N = [800, 1600, 3200, 6400] # fig22(a) 的经典 FV 空间对照
48 FV_SPACE_DT = 1.0 # 该对照固定用的时间步 (时间误差按 c*dt 扣除)
49 INTEG_CFG = [("BDF", 1e-9), ("BDF", 1e-10), ("BDF", 1e-12),
50              ("Radau", 1e-11), ("LSODA", 1e-10)]
51
52
53 def _tdry(prob, N, method="BDF", rtol=1e-11, shrink=False):
54     """用谱配置法求 t_dry, 返回 (求解器实例, t_dry [s])。
55
56     prob 为问题号 (本模块只用到 3: 附录 3 物性、半径固定); N 是谱节点数;
57     method 与 rtol 是时间积分器及其相对容差, 用来考察积分器/容差带来的差异;
58     shrink 只在含收缩的问题 4 下为 True。终止事件取全场最大含水率 (出现在轴心 u=0)
59     降到 CRIT 以下的时刻, 与生产脚本同口径。
60     """
61     from spectral import HerbSolver
62     s = HerbSolver(N, prob=prob, shrink=shrink)
63     at = np.concatenate([np.full(s.Np, 1e-13), np.full(s.Np, 1e-11)])
64
65     def ev(t, y):
66         return np.max(y[:s.Np]) - CRIT
67     ev.terminal = True
68     ev.direction = -1
69     e = s.solve(300000.0, rtol=rtol, atol=at, events=ev, method=method)
70     return s, float(e.t_events[0][0])
71
72
73 def recompute(verbose=True):
74     """真实复跑四类扫描并写进 verification.json, 返回数据字典。
75
76     verbose 控制是否边跑边打印进度。落盘的字典有四个键: space (谱网格扫描)、
77     time (定步长时间扫描)、fv_space (经典 FV 空间对照)、integ (积分器与容差,
78     另含一条 1 s 密采样求交的独立口径), 并记录版本号与生成命令。
79     """
80     import common as cm
81     from fv_theta import find_tdry
82
83     def say(s):
84         if verbose:
85             print(s, flush=True)
86
87     say("[verify] 1/4 空间扫描: 问题 3 的 t_dry vs N")
88     tdry_space = []
89     for N in SPACE_N:
90         _, td = _tdry(3, N, rtol=SPACE_RTOL)
91         tdry_space.append(td)
92         say(" N=%-4d tdry = %.4f s" % (N, td))
93
94     say("[verify] 2/4 时间步扫描: 定步长 theta 法 (N=%d), 最耗时" % FV_N)
95     tdry_time = []
96     for d in TIME_DT:
97         plan = [(7200.0, min(1.0, d)), (1e18, d)]
98         td = float(find_tdry(FV_N, 3, plan))
99         tdry_time.append(td)
100        say(" dt=%-6.2f tdry = %.4f s" % (d, td))
101
102     say("[verify] 3/4 经典 FV 的空间对照 (N=%s, dt=%.1f s) "
103         % (FV_SPACE_N, FV_SPACE_DT))
104     tdry_fvspace = []
105     for N in FV_SPACE_N:
106         plan = [(7200.0, FV_SPACE_DT), (1e18, FV_SPACE_DT)]
107         td = float(find_tdry(N, 3, plan))
108         tdry_fvspace.append(td)
109         say(" N=%-5d tdry = %.4f s" % (N, td))
110
111     say("[verify] 4/4 时间积分器与容差")
112     rows = []
113     for meth, tol in INTEG_CFG:
114         _, td = _tdry(3, 64, method=meth, rtol=tol)
115         rows.append({"method": meth, "rtol": tol, "tdry": td})

```

```

116     say("    %-6s rtol=%.0e tdry = %.4f s" % (meth, tol, td))
117
118     # 第 6 个点: 换一种与 solve_ivp 事件机制无关的口径复核 t_dry——先在邻域内按 1 s
119     # 输出求解, 再对中心含水率 C(0,t) 与 CRIT 做线性求交。含水率的最大值始终出现在
120     # 轴心 u=0, 故事件函数取的就是 C(0,t), 这条口径与事件法等价, 但"求交点"的方式
121     # 完全不同, 两者一致才说明 t_dry 不依赖数值求交的实现。
122     s, td0 = _tdry(3, 64, rtol=1e-11)
123     at = np.concatenate([np.full(s.Np, 1e-13), np.full(s.Np, 1e-11)])
124     tgrid = np.arange(max(0.0, td0 - 120.0), td0 + 61.0, 1.0)
125     sol = s.solve(td0 + 60.0, t_eval=list(tgrid), rtol=1e-11, atol=at)
126     c0 = sol.y[0, :]
127     idx = np.where(c0 <= CRIT)[0]
128     k = int(idx[0])
129     td_dense = float(tgrid[k - 1] + (c0[k - 1] - CRIT) / (c0[k - 1] - c0[k])
130                     * (tgrid[k] - tgrid[k - 1]))
131     rows.append({"method": "dense1s", "rtol": 1e-11, "tdry": td_dense})
132     say("    1s 密采样求交 tdry = %.4f s" % td_dense)
133
134     data = {
135         "version": DATA_VERSION,
136         "produced_by": "python verify_data.py --recompute",
137         "space": {"N": SPACE_N, "tdry": tdry_space, "rtol": SPACE_RTOL},
138         "time": {"dt": TIME_DT, "tdry": tdry_time, "N": FV_N},
139         "fv_space": {"N": FV_SPACE_N, "tdry": tdry_fvspace, "dt": FV_SPACE_DT},
140         "integ": rows,
141     }
142     os.makedirs(DATA_DIR, exist_ok=True)
143     with open(DATA_PATH, "w", encoding="utf-8") as fh:
144         json.dump(data, fh, ensure_ascii=False, indent=1)
145     say("[verify] 已写出 %s" % DATA_PATH)
146     return data
147
148
149 def load(verbose=True):
150     """读入 verification.json 并返回其字典, 附带版本号校验。
151
152     verbose 控制是否打印一行载入信息 (run_all.py 用 verbose=False 静默调用)。
153     文件缺失或版本号与 DATA_VERSION 不符时直接抛 RuntimeError, 消息里给出重新生成
154     的命令——原因是不画图也比画一张来源不明的图好。
155     """
156     if not os.path.isfile(DATA_PATH):
157         raise RuntimeError(
158             "缺少验证数据文件: %s\n"
159             " 它由 fig22/fig23 使用, 需要先真实复跑一次: \n"
160             "    python verify_data.py --recompute      (约 22 min) \n"
161             "    或运行完整复跑: python run_all.py % DATA_PATH"
162         )
163     with open(DATA_PATH, "r", encoding="utf-8") as fh:
164         data = json.load(fh)
165     if data.get("version") != DATA_VERSION:
166         raise RuntimeError(
167             "验证数据文件版本不符 (文件 %r / 期望 %r): %s\n"
168             " 请重新生成: python verify_data.py --recompute"
169             % (data.get("version"), DATA_VERSION, DATA_PATH))
170     if verbose:
171         print("[verify] 载入 %s (版本 %s, 由 %s 生成)"
172               % (DATA_PATH, data["version"], data["produced_by"]))
173     return data
174
175
176 def time_slope(data):
177     """由时间扫描拟合 t(dt) = L - c*dt 的斜率系数 c (s per s), 供扣时间误差用。
178
179     用最细的三个步长做最小二乘, 避免粗步长段的高阶项污染一阶系数。
180     """
181     dt = np.asarray(data["time"]["dt"], dtype=float)
182     td = np.asarray(data["time"]["tdry"], dtype=float)
183     k = min(3, len(dt))
184     slope, _intercept = np.polyfit(dt[-k:], td[-k:], 1)
185     return float(-slope)
186
187
188 def check_consistency(data, tdry_cached=None, verbose=True):
189     """把数据文件与本轮真实算出的量逐条对照, 任何一条不成立就抛 RuntimeError。
190
191     data 是 load() 返回的字典; tdry_cached 是解缓存里的问题 3 烘干时长 [s], 给了就
192     额外核对空间扫描的 N=64 项 (这是"图的数据"与"这次算的解"之间唯一的硬连接);
193     verbose 控制是否打印每条的核对结果。五条校验见下面的 (1)-(5), 都是数据自洽的
194     必要条件; 全部通过返回 True。
195     """
196     problems = []
197     Ns = list(data["space"]["N"])

```

```

197 ts = np.asarray(data["space"]["tdry"], dtype=float)
198 dts = list(data["time"]["dt"])
199 td_dt = np.asarray(data["time"]["tdry"], dtype=float)
200
201 def note(s):
202     if verbose:
203         print("[verify] " + s)
204
205 # (1) 空间扫描必须单调不减 (请解随 N 增大而收敛到同一极限)
206 if not np.all(np.diff(ts) >= -1e-9):
207     problems.append("空间扫描的 t_dry 随 N 非单调: %s" % np.round(ts, 3).tolist())
208 # (2) 与解缓存交叉核对: N=64 项必须等于缓存里真实算出的 t_dry
209 if tdry_cached is not None:
210     i64 = Ns.index(64) if 64 in Ns else None
211     if i64 is None:
212         problems.append("空间扫描缺少 N=64 项, 无法与解缓存核对")
213     else:
214         gap = abs(ts[i64] - float(tdry_cached))
215         note("空间扫描 N=64 = %.4f s vs 解缓存 %.4f s (差 %.4f s) "
216             % (ts[i64], tdry_cached, gap))
217         # 0.05 s 只是"数据是否过期"的防呆阈值: 口径一致时两者逐位相等 (差 0.0000 s)。
218         if gap > 0.05:
219             problems.append(
220                 "空间扫描 N=64 的 t_dry 与解缓存相差 %.3f s, 数据文件已过期"
221                 % gap)
222 # (3) 时间扫描必须随 dt 减小而单调增大, 且增量与 dt 成正比 (严格一阶)
223 if not np.all(np.diff(td_dt) >= -1e-9):
224     problems.append("时间扫描的 t_dry 随 dt 减小非单调: %s" % np.round(td_dt, 3).tolist())
225 inc = np.diff(td_dt) # dt 由大到小, 增量应为正且递减
226 if inc.size >= 3:
227     # 一阶收敛的判据: log(相邻增量) 对 log(dt 的几何中点) 的斜率应为 1。
228     # 注意增量是**减半**的 (比值约 0.5), 不是加倍——用比值判据容易反向。
229     # 容差取 0.8~1.25: 既容得下离散噪声, 又能把斜率约 2 的二阶数据或非单调数据挡掉。
230     dts_arr = np.asarray(dts, dtype=float)
231     mid = np.sqrt(dts_arr[1:] * dts_arr[:-1])
232     slope = float(np.polyfit(np.log(mid), np.log(inc), 1)[0])
233     if not (0.8 < slope < 1.25):
234         problems.append("时间扫描不满足 0(dt) 一阶收敛: 增量-步长双对数斜率 %.3f (应接近 1) "
235             % slope)
236     else:
237         note("时间扫描一阶收敛: 增量-步长双对数斜率 %.3f (应接近 1) " % slope)
238 else:
239     note("时间扫描只有 %d 个点, 跳过收敛阶检查" % len(dts))
240
241 # (4) 经典 FV 的空间对照应随 N 增大单调收敛, 且扣去时间误差后仍不高于连续极限
242 fs = np.asarray(data["fv_space"]["tdry"], dtype=float)
243 c_fv = time_slope(data) # 由时间扫描拟合出的 0(dt) 系数 c
244 if not np.all(np.diff(fs) >= -1e-9):
245     problems.append("经典 FV 空间对照的 t_dry 随 N 非单调: %s" % np.round(fs, 3).tolist())
246 # 留 0.5 s 余量: 扣掉时间误差后若仍比请解极限高出 0.5 s 以上, 说明系数 c 不合理,
247 # 或者这批数据已经过期。
248 if len(fs) and fs[-1] + c_fv * data["fv_space"]["dt"] > ts[-1] + 0.5:
249     problems.append("经典 FV 扣去时间误差后超过了请解连续极限, 说明时间误差系数 c=%.3f 不合理"
250         % c_fv)
251 else:
252     note("经典 FV 最细网格扣时间误差 = %.3f s, 请解极限 %.3f s (差 %.3f s) "
253         % (fs[-1] + c_fv * data["fv_space"]["dt"], ts[-1],
254            ts[-1] - fs[-1] - c_fv * data["fv_space"]["dt"]))
255
256 # (5) 积分器与容差: fig23 的"极差"必须真的小
257 iv = np.asarray([r["tdry"] for r in data["integ"]], dtype=float)
258 spread = float(iv.max() - iv.min())
259 if len(iv) != len(INTEG_CFG) + 1:
260     problems.append("积分器配置数 %d 与 fig23 的 6 个刻度不符" % len(iv))
261 # fig23 的结论是"六种配置的极差只有 0.007 s", 故超过 0.05 s 就说明该结论不成立。
262 if spread > 0.05:
263     problems.append("积分器/容差极差 %.4f s 过大, fig23 的结论不成立" % spread)
264 else:
265     note("积分器与容差极差 %.4f s (%d 个配置)" % (spread, len(iv)))
266
267 if problems:
268     raise RuntimeError(
269         "验证数据不一致, 已中止绘图 (避免画出与代码结果脱钩的图): \n - "
270         + "\n - ".join(problems)
271         + "\n 请重新生成: python verify_data.py --recompute")
272 note("一致性检查通过")
273 return True
274
275
276 def _cached_tdry():
277     """从解缓存里读问题 3 的 t_dry, 用于与空间扫描的 N=64 项交叉核对。"""

```

```

278 path = os.path.join(DATADIR, "solutions.npz")
279 if not os.path.isfile(path):
280     return None
281 try:
282     z = np.load(path)
283     return float(z["p3_tdry"][0])
284 except Exception as exc: # 缓存损坏或缺失: 跳过核对即可
285     print("[verify] 读解缓存失败, 跳过 N=64 交叉核对: %s" % exc)
286     return None
287
288
289 def main(argv=None):
290     """命令行入口: 带 --recompute 就先重跑四类扫描, 然后统一做一次一致性检查。
291
292     argv 不给时取 sys.argv[1:]。返回 0 表示校验通过; 不通过时异常抛出,
293     由 sys.exit 带出非零退出码。
294     """
295     argv = sys.argv[1:] if argv is None else argv
296     if "--recompute" in argv:
297         recompute()
298     check_consistency(load(), tdry_cached=_cached_tdry())
299     return 0
300
301
302 if __name__ == "__main__":
303     sys.exit(main())

```

3.13 控制台编码修复: console_utf8.py

```

1  # -*- coding: utf-8 -*-
2  # =====
3  # console_utf8.py —— 把控制台输出切到 UTF-8 (Windows 默认 GBK 会把中文打成乱码)
4  # =====
5  # 做什么: 只做一件事——把当前进程的 stdout/stderr 重新配置成 UTF-8。Windows 控制台
6  # 默认按 GBK 解码, 脚本 print 出来的中文进度信息会变成乱码, 而一键复跑到
7  # 哪一步、哪一节报错, 全靠这些中文行判断。
8  # 输入: 不读数据文件、不依赖本项目任何模块, 只调用 sys.stdout/stderr.reconfigure()。
9  # 输出: 不写文件; 唯一效果是改掉本进程这两个标准流的字符编码。
10 # 运行方式: 不单独运行。在调用方脚本的最前面 (任何 print 之前) 写两行:
11 #     from console_utf8 import fix_console
12 #     fix_console()
13 # 它不参与任何数值计算, 放在哪个脚本前面都不影响结果。
14 # 关键变量: 无模块级常量; 两个标准流的 errors 一律取 "replace" (原因见 fix_console)。
15 # 文件本身保持 UTF-8 无 BOM、行尾 LF、行尾不留空格, 以便任何平台按纯文本读取。
16 # =====
17 import sys
18
19
20 def fix_console():
21     """把 stdout/stderr 切到 UTF-8; 失败时静默跳过, 不阻断主流程。
22
23     参数与返回: 均为空 (就地重配两个标准流)。
24     errors="replace" 是关键: 即使下游终端不接受 UTF-8, 也只是把个别字显示成问号,
25     不会让整段复跑因为一次 print 抛 UnicodeEncodeError 而中断。
26     """
27     try:
28         sys.stdout.reconfigure(encoding="utf-8", errors="replace")
29         sys.stderr.reconfigure(encoding="utf-8", errors="replace")
30     except Exception:
31         # 需要 Python 3.7+ 才有的 reconfigure, 且个别环境的重定向流不支持;
32         # 这种情况退回平台默认编码即可, 不影响任何计算结果。
33         pass

```